

PROMPT TO PROFIT™

WORKBOOK 01

Expense Tracker

Build a Complete Expense Tracking Application
with AI

TECHNOLOGY STACK

HTML

CSS

Vanilla JavaScript

Supabase

DIFFICULTY

Beginner

ESTIMATED COMPLETION TIME

10-15 Hours

PRODUCED BY

Tochukwu Tech and AI Academy

www.tochukwunkwocha.com

PROMPT TO PROFIT™

Workbook 01 · Expense Tracker

© 2026 Tochukwu Tech and AI Academy. All rights reserved.

Produced and published by Tochukwu Tech and AI Academy.

www.tochukwunkwocha.com

This workbook is designed for personal learning and guided software-building practice.

Technology names and product names remain the property of their respective owners.

Contents

Workbook 01 · Expense Tracker

LEARNER SUPPORT TOOLKIT	5
VISUAL GLOSSARY	6
WORK SAFELY: MAKE A BACKUP	8
SOMETHING WENT WRONG – WHAT SHOULD I CHECK?	9
MY ERROR LOG	11
CHAPTER 1 · BUILDING THE PUBLIC WEBSITE	13
LESSON 1 · UNDERSTANDING THE PROJECT	15
LESSON 2 · BUILDING THE LANDING PAGE	18
LESSON 3 · STYLING THE LANDING PAGE	25
LESSON 4 · ADDING THE RESPONSIVE NAVIGATION	36
LESSON 5 · COMPLETING THE LANDING PAGE	46
CHAPTER SUMMARY	55
CHAPTER MILESTONE	56
CHAPTER 2 · CONNECTING TO SUPABASE	59
LESSON 1 · CREATING YOUR SUPABASE PROJECT	61
LESSON 2 · UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY	65
LESSON 3 · CREATING YOUR SUPABASE CONNECTION FILE	70
LESSON 4 · BUILDING A CONNECTION TEST PAGE	77
LESSON 5 · FINAL CONNECTION REVIEW	85
CHAPTER SUMMARY	90
CHAPTER MILESTONE	91
CHAPTER 3 · BUILDING THE AUTHENTICATION SYSTEM	93
LESSON 1 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	96
LESSON 2 · AUDITING YOUR AUTHENTICATION SYSTEM	116
CHAPTER SUMMARY	122

CHAPTER MILESTONE	123
CHAPTER 4 · BUILDING THE TRANSACTIONS DATABASE	125
LESSON 1 · CREATING THE TRANSACTIONS TABLE	127
LESSON 2 · UNDERSTANDING ROW LEVEL SECURITY	134
LESSON 3 · CREATING THE SELECT POLICY	139
LESSON 4 · CREATING THE INSERT POLICY	143
LESSON 5 · CREATING THE UPDATE POLICY	148
LESSON 6 · CREATING THE DELETE POLICY	152
LESSON 7 · TESTING YOUR DATABASE SECURITY	157
CHAPTER SUMMARY	162
CHAPTER MILESTONE	163
CHAPTER 5 · BUILDING THE EXPENSE TRACKER DASHBOARD	165
LESSON 1 · BUILDING THE TRANSACTION ENTRY FORM	167
LESSON 2 · SAVING TRANSACTIONS TO SUPABASE	177
LESSON 3 · DISPLAYING TRANSACTIONS	187
LESSON 4 · CALCULATING TOTAL INCOME, TOTAL EXPENSES AND CURRENT BALANCE	196
CHAPTER MILESTONE	205
CHAPTER 6 · VIEWING TRANSACTIONS	208
LESSON 1 · IMPROVING THE TRANSACTION LIST	209
LESSON 2 · AUDITING THE TRANSACTION VIEW	221
CHAPTER SUMMARY	231
CHAPTER MILESTONE	232
CHAPTER 7 · EDITING AND DELETING TRANSACTIONS	234
LESSON 1 · EDITING TRANSACTIONS	236
LESSON 2 · DELETING TRANSACTIONS	249
LESSON 3 · TESTING OWNERSHIP AND RECORD MANAGEMENT	262
CHAPTER SUMMARY	272
CHAPTER MILESTONE	273
CHAPTER 8 · MAKING THE DASHBOARD MORE POWERFUL	275
LESSON 1 · ADDING SEARCH AND FILTERS	277
LESSON 2 · ADDING SORTING CONTROLS	293

LESSON 3 · FINAL USER EXPERIENCE REVIEW	303
CHAPTER SUMMARY	312
CHAPTER MILESTONE	313
CHAPTER 9 · FINAL TESTING AND PROJECT COMPLETION	314
LESSON 1 · COMPLETE PROJECT AUDIT	316
LESSON 2 · GOING LIVE	326
LESSON 3 · PORTFOLIO DESCRIPTION	332
REFLECTION QUESTIONS	337
EXTENSION CHALLENGES	338
NEXT WORKBOOK	339

WELCOME

Welcome to Prompt to Profit™ Workbook 01.

This workbook will guide you step by step as you build a complete Expense Tracker using Artificial Intelligence.

By the end of this workbook, you will have built a real software application that people can use to register, log in, record their income and expenses, monitor their spending, and manage their own financial records securely.

You are not expected to know how to write code before starting this workbook.

Instead, you will learn how to work with AI as your software development partner.

Throughout this workbook, AI will generate the code while you learn how software is put together, how different files work together, how to organise a project properly, and how to test each feature as you build it.

You will be using only:

- Notepad
- A modern web browser
- ChatGPT
- Supabase

You do not need any programming experience.

You do not need any special software.

You do not need to install anything on your computer.

Everything will be built one step at a time.

As the workbook progresses, your software will gradually become more powerful. Every chapter builds on the previous one, so it is important to complete each lesson in order.

Never skip ahead.

Always test your work before moving to the next lesson.

If something does not work, fix it before continuing.

Remember:

Software is not built by writing one huge file.

It is built by adding one working piece at a time until everything works together.

That is exactly how professional software developers work, and that is the approach you will follow throughout this workbook.

WHAT YOU WILL BUILD

By the time you complete this workbook, your Expense Tracker will include:

PUBLIC WEBSITE

- Responsive navigation
- Mobile hamburger menu
- Hero section
- Problem section
- Features section
- How It Works section
- Dashboard preview
- Call-to-action section
- Footer
- Login button
- Register button

SUPABASE

- Supabase project
- Secure project connection
- Publishable Key
- Connection testing
- Proper script loading order

AUTHENTICATION

- User registration
- Email verification
- User login
- Protected dashboard
- Logout

- Friendly success messages
- Friendly error messages
- Loading indicators

DATABASE

- Secure transactions table
- Row Level Security
- User-owned records only
- Protected SELECT
- Protected INSERT
- Protected UPDATE
- Protected DELETE

EXPENSE TRACKER

- Add transactions
- View transactions
- Edit transactions
- Delete transactions
- Search
- Filters
- Running balance
- Total income
- Total expenses
- Responsive dashboard
- Professional design

FINAL PROJECT

- Complete testing
- Security testing
- Folder review

- Portfolio-ready submission

WORKBOOK STRUCTURE

This workbook is organised into nine chapters.

Chapter 1

Building the Public Website

Chapter 2

Connecting to Supabase

Chapter 3

Building the Authentication System

Chapter 4

Building the Transactions Database

Chapter 5

Building the Expense Tracker Dashboard

Chapter 6

Viewing Transactions

Chapter 7

Editing and Deleting Transactions

Chapter 8

Making the Dashboard More Powerful

Chapter 9

Final Testing and Project Completion

Complete each chapter before moving to the next.

Do not jump ahead.

Each chapter assumes everything built previously is already working.

LEARNER SUPPORT TOOLKIT

Keep this part of the workbook close while you build.

It explains common words, shows you how to protect your work, gives you a simple way to investigate problems, and provides a place to record errors and solutions.

You are not expected to remember everything immediately.

Return to these pages whenever you need them.

VISUAL GLOSSARY

HTML	The file that gives a web page its content and structure.
CSS	The file that controls colours, spacing, layout and appearance.
VANILLA JAVASCRIPT	JavaScript used without a framework. It controls what the application does.
BROWSER	The program used to open and test the application, such as Chrome or Edge.
FILE	One saved part of the project, such as index.html or dashboard.js.
FOLDER	The place where all files for one project are kept together.
FUNCTION	A named set of instructions that performs one task.
EVENT	An action the browser can notice, such as a click, form submission or key press.
VARIABLE	A named place used to hold information while the application is running.
URL	The address of a page, website or online service.
SUPABASE	The online service used for the database, authentication and security.
DATABASE	An organised place where the application stores information.
TABLE	A named part of the database that stores one type of information.
RECORD	One complete item saved inside a database table.
AUTHENTICATION	The process of creating accounts, signing in and identifying the current user.
SESSION	The saved sign-in state that allows the application to remember the current user.
ROW LEVEL SECURITY	Supabase rules that control which database records each user can access.

PUBLISHABLE KEY	The Supabase project key that browser-based applications are allowed to use.
DEPLOY	To place the complete project online so it has a live HTTPS website address.
DEBUGGING	The careful process of finding, understanding and correcting a problem.

WORK SAFELY: MAKE A BACKUP

A backup is a separate copy of your complete project folder.

Create a backup before replacing files that already contain working code.

1.

Close any project files that are open in Notepad.

2.

Find your complete project folder.

3.

Copy the folder.

4.

Paste the copy outside the working project folder.

Do not place the backup inside the folder you deploy.

5.

Rename the copy clearly.

For example:

Project Backup - Before Chapter 3 Lesson 2

6.

Open the backup folder.

Confirm that the expected files are inside it.

If a new change causes a serious problem, you can return to this working copy.

SOMETHING WENT WRONG — WHAT SHOULD I CHECK?

Problems are a normal part of building software.

Do not replace random pieces of code and do not continue to the next lesson.

Work through this checklist in order.

- ☐ Confirm that you opened the correct project folder.
- ☐ Confirm that every updated file was saved in Notepad.
- ☐ Check every filename carefully.
- ☐ Make sure an HTML, CSS or JavaScript file does not end with .txt.
- ☐ Confirm that you pasted the complete file returned by AI.
- ☐ Confirm that you did not paste an explanation, a code-block label or only part of a file.
- ☐ Check that stylesheet and JavaScript filenames match the names used inside the HTML.
- ☐ On Supabase pages, check that the scripts load in the order taught in the lesson.
- ☐ Read the exact message shown on the page.
- ☐ Open the browser Console and look for the first red error.

To open the Console:

Right-click the page.

Select:

Inspect

Then select:

Console

- ☐ Copy the complete first red error into your error log.
- ☐ Think about the last file you changed before the problem appeared.
- ☐ Compare that file with the lesson requirements.
- ☐ If necessary, restore the most recent working backup.

When asking AI for help, provide:

-
- What you were trying to do
 - What you expected to happen
 - What actually happened
 - The complete error message
 - The names of the files involved

Ask AI to return complete corrected files.

Do not ask for snippets.

MY ERROR LOG

Use this page whenever something does not work.

Writing down the problem helps you investigate it carefully and remember the solution.

ERROR 1

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

ERROR 2

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

CHAPTER INTRODUCTION

Every software product needs a public face.

Before users create an account or sign in, they first visit a public website where they learn what the software does and why they should use it.

That is what you will build in this chapter.

By the end of this chapter, you will have created a modern, responsive landing page for your Expense Tracker.

Visitors will be able to learn about the software and use clearly visible Login and Register links to continue into the application later in this workbook.

Although those pages have not yet been built, the navigation will already be designed to support them.

You will also learn one of the most important ideas in software development:

Different files have different jobs.

HTML provides the structure.

CSS controls the appearance.

JavaScript adds behaviour and interaction.

Keeping these responsibilities separate makes software easier to maintain and improve.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter you will build:

- A project folder
- index.html

- styles.css
- script.js
- Responsive navigation
- Three-line hamburger menu
- Hero section
- Problem section
- Features section
- How It Works section
- Dashboard preview section
- Call-to-action section
- Footer
- Mobile-friendly layout
- Login and Register navigation links

At the end of this chapter, your landing page will be fully responsive and ready for the authentication system that will be built in Chapter 3.

LESSON 1

UNDERSTANDING THE PROJECT

Estimated Time

10 minutes

WHAT YOU ARE BUILDING

Before writing any files, it is important to understand the software you are about to build.

The project in this workbook is more than a simple webpage.

It is a complete web application with user accounts, a secure online database, and a dashboard where each user can safely manage their own financial records.

Over the coming chapters, every feature will be added one step at a time until the entire application works together as one complete system.

WHY THIS MATTERS

One mistake many beginners make is rushing straight into building without understanding the overall project.

Professional software developers always begin by understanding the complete system before creating individual pages.

When you understand where the project is going, every lesson makes much more sense.

You will know why each file exists, how different files work together, and why every new feature is added in a particular order.

BEFORE YOU CONTINUE

Create a new folder anywhere on your computer.

Name the folder:

Expense Tracker

This folder will contain every file you create throughout this workbook.

Do not rename this folder later.

Keeping your files together makes the project much easier to manage.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with AI in this lesson.

Instead, prepare your project folder and read through the workbook structure so you understand the journey ahead.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson prepares your workspace.

SAVE YOUR FILES

No files are created in this lesson.

Simply make sure your Expense Tracker folder has been created successfully.

TEST YOUR WORK

Open your Expense Tracker folder.

Confirm that it exists and is empty.

This is exactly what you should see before starting the next lesson.

CHECKPOINT

Before moving on, you should have:

- ☒ Created your Expense Tracker folder.
- ☒ Read the workbook overview.
- ☒ Understood what you will build during this workbook.

COMMON BEGINNER MISTAKES

Creating multiple folders for the same project can make it difficult to find the correct files later.

Keep everything for this workbook inside your Expense Tracker folder.

BEHIND THE SCENES

Professional software projects usually contain many files.

Organising those files properly from the beginning saves time and prevents confusion as the project grows.

Even though your project starts with an empty folder, that folder will gradually become a complete software application.

THINK LIKE A SOFTWARE DESIGNER

Software developers rarely begin by writing code immediately.

They first understand the problem they are solving and the structure of the application they intend to build.

Good planning almost always produces better software.

WHAT YOU LEARNED

In this lesson you learned:

- what you will build
- how the workbook is organised
- why projects are built step by step
- why keeping files organised matters

In the next lesson, you will begin building the public website that visitors will see when they first open your Expense Tracker.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 2

BUILDING THE LANDING PAGE

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the first version of your public website.

This page will become the home page of your Expense Tracker. Visitors will use it to learn what the software does before deciding to register for an account.

At this stage, the page will already include the main sections of a professional landing page. In later lessons, you will improve its appearance, make it responsive, and add interactive features.

WHY THIS MATTERS

The home page is often the first thing people see when they visit a website.

A good landing page quickly explains what the software does, who it is for, and why someone should use it.

Although the dashboard is where users will eventually manage their expenses, the landing page is what encourages them to create an account in the first place.

BEFORE YOU CONTINUE

Open Notepad.

You are going to create your first HTML file.

When you save the file:

- Click **File**.
- Click **Save As**.
- Browse to your **Expense Tracker** folder.

- Change **Save as type** to **All Files**.

- Enter the filename exactly as:

index.html

Make sure the file is not saved as:

index.html.txt

Saving the file with the correct name is very important because web browsers recognise HTML files by their ".html" extension.

BUILD PROMPT

PROMPT STARTS HERE

Generate a complete HTML5 file named **index.html** for a modern, responsive Expense Tracker landing page.

This is the public website for the application.

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

Requirements:

- Use proper HTML5 structure.
- Include a meaningful page title.
- Include appropriate meta tags.
- Include this stylesheet link inside the <head> section:

```
<link rel="stylesheet" href="styles.css">
```

- Include this JavaScript file immediately before the closing </body> tag:

```
<script src="script.js"></script>
```

The page must contain the following sections in this exact order:

1. Header

Inside the header include:

- Logo text:

Expense Tracker

- Navigation menu with these links:

Home

Features

How It Works

Login

Register

The Login and Register links should temporarily point to:

login.html

register.html

These pages have not been built yet, but the links should already use those filenames because they will exist later in this workbook.

Also include a mobile hamburger button built using exactly three empty `<span>` elements.

2. Hero Section

Include:

- A strong headline.
- A short paragraph explaining the software.
- A "Get Started" button linking to register.html.
- A "Learn More" button linking to the Features section.

3. Problem Section

Explain common problems people face when trying to manage their finances manually.

4. Features Section

Include six feature cards describing the software.

5. How It Works Section

Explain the process from creating an account to tracking expenses.

6. Dashboard Preview Section

Include a realistic placeholder dashboard layout built entirely with HTML.

Do not use images.

Use simple cards and boxes to represent what the dashboard will look like.

7. Final Call-to-Action Section

Encourage visitors to create an account.

Include another button linking to:

register.html

8. Footer

Include:

- Copyright text

- Quick navigation links
 - A short description of the software
- Use clean, semantic HTML throughout.
- Use meaningful class names.
- Everything should be prepared for styling in a separate CSS file.
- Do not include any CSS.
- Do not include any JavaScript.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

index.html

The file should already contain:

- The stylesheet link to styles.css.
- The JavaScript link to script.js.
- All required landing page sections.
- Login and Register links.
- A three-line hamburger menu built with three span elements.

SAVE YOUR FILES

Copy the complete code returned by ChatGPT.

Open Notepad.

Paste the code into the empty document.

Click **File**.

Click **Save As**.

Browse to your **Expense Tracker** folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

index.html

Click **Save**.

TEST YOUR WORK

Open your Expense Tracker folder.

Double-click **index.html**.

Your web browser should open the page.

At this stage, the page will look plain because the stylesheet has not been created yet.

That is completely normal.

Scroll through the page and confirm that you can see:

☒ Header

☒ Navigation

☒ Hero section

☒ Problem section

☒ Features section

☒ How It Works section

☒ Dashboard preview

☒ Final call-to-action

☒ Footer

Click the Login and Register links.

The browser will show a page-not-found message because those pages have not been created yet.

This is expected and confirms that the links are already pointing to the correct filenames.

CHECKPOINT

Before moving on, confirm that:

- ✓ index.html opens successfully.
- ✓ Every section appears on the page.
- ✓ The browser does not display HTML code as plain text.
- ✓ The Login and Register links point to login.html and register.html.

COMMON BEGINNER MISTAKES

One of the most common mistakes is saving the file as:

index.html.txt

If this happens, the browser will not recognise it as a web page.

Another common mistake is forgetting to change **Save as type** to **All Files** before saving.

If your browser displays the code instead of the webpage, check the filename carefully.

BEHIND THE SCENES

Even though styles.css and script.js do not exist yet, your HTML file already references them.

Professional developers often build software this way.

They first create the structure of the application, then add the styling and behaviour afterwards.

By linking those files now, you avoid having to modify the HTML structure later.

THINK LIKE A SOFTWARE DESIGNER

Good software is designed to grow.

Instead of building everything at once, developers create a solid foundation that allows new features to be added without rebuilding the entire project.

That is exactly what you are doing here.

CODE-READING QUESTION

Open index.html. Find the part of the page added for building the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create your first HTML file
- save an HTML file correctly using Notepad
- build the structure of a professional landing page
- prepare a page for separate CSS and JavaScript files
- create navigation that supports future pages

In the next lesson, you will create the stylesheet that transforms this plain HTML page into a modern, professional website.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 3

STYLING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the stylesheet for your landing page.

The stylesheet will control the colours, spacing, typography, buttons, cards, section layouts, dashboard preview, and footer.

Your page already contains the correct HTML structure. You will now give that structure a professional appearance.

The responsive mobile navigation will be completed in the next lesson when you create the JavaScript file.

WHY THIS MATTERS

HTML gives a webpage its structure, but CSS determines how that structure looks.

Without CSS, a webpage appears as plain text and basic links arranged from top to bottom.

A well-written stylesheet helps users understand the page more easily. It creates clear sections, readable text, visible buttons, balanced spacing, and a layout that works across different screen sizes.

Good design is not only about making a website attractive. It also makes the website easier to use.

BEFORE YOU CONTINUE

Confirm that your Expense Tracker folder already contains:

index.html

Open index.html in your browser and make sure the complete landing page still appears.

Do not continue if index.html is missing or does not open correctly.

You will now ask ChatGPT to create styles.css.

The Build Prompt refers to the HTML structure created in the previous lesson. To help ChatGPT match the stylesheet to your exact HTML classes, paste your complete index.html code at the end of the prompt where instructed.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete HTML file named index.html for an Expense Tracker landing page.

Create one complete CSS file named styles.css that correctly styles the exact HTML structure I will paste below.

Return the complete styles.css file only.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return JavaScript.

Everything already built in index.html must continue working.

The stylesheet must be compatible with this existing link inside index.html:

```
<link rel="stylesheet" href="styles.css">
```

Design requirements:

GENERAL DESIGN

- Create a clean, modern, professional financial software design.
- Use a light background with strong contrast.
- Use a restrained colour palette suitable for a trusted financial application.
- Use CSS custom properties inside :root for the main colours, spacing values, border radius, shadows, and content width.
- Apply box-sizing: border-box to all elements.
- Remove unnecessary default browser margins and padding.
- Use a clear system font stack that does not require an internet connection.
- Give the page smooth scrolling.
- Make paragraphs easy to read.
- Make all buttons and links visually clear.

LAYOUT

- Give major sections generous but balanced vertical spacing.
- Keep page content centred inside a maximum-width container.
- Ensure content does not touch the edges of the screen.
- Use flexible layouts that can adjust across screen sizes.

HEADER AND NAVIGATION

- Make the header clean and professional.
- Keep the logo clearly visible.
- Style the logo as a clickable brand element if it is inside a link.
- Arrange the desktop navigation links horizontally.
- Style Login as a secondary action.
- Style Register as the main action.
- Add clear hover and focus states.
- Keep the header visible and organised on smaller screens.

HAMBURGER BUTTON

- Style the mobile hamburger button.
- The button contains exactly three empty span elements.
- Stack the three span elements vertically so they appear as three horizontal lines.
- Hide the hamburger button on large screens.
- Show it on smaller screens.
- Do not use CSS-generated lines through `::before` or `::after`.
- Do not add JavaScript.
- Prepare the navigation menu to work with an active class that JavaScript will add in the next lesson.
- Use a class such as `.active` on the navigation menu to display it on smaller screens.

HERO SECTION

- Create a strong two-column desktop layout where appropriate.
- Make the main headline prominent.

- Style the supporting paragraph for readability.
- Place the main buttons neatly together.
- Make the primary and secondary buttons visually different.
- Make the layout stack properly on smaller screens.

PROBLEM SECTION

- Give the section a clear visual difference from the hero section.
- Make the heading and explanatory text easy to scan.
- Use balanced spacing.

FEATURES SECTION

- Display the six feature cards in a responsive grid.
- Give each card a professional border, subtle shadow, rounded corners, and comfortable spacing.
- Include hover movement that is subtle and does not distract the user.

HOW IT WORKS SECTION

- Display the steps clearly.
- Use cards, numbered circles, or another simple visual structure.
- Keep the process easy to follow.

DASHBOARD PREVIEW SECTION

- Make the HTML dashboard placeholder look like a realistic software dashboard.
- Style its summary cards, transaction rows, panels, headings, buttons, and other existing elements.
- Do not depend on any images.
- Make the preview responsive.

FINAL CALL-TO-ACTION SECTION

- Give the section a strong but professional appearance.
- Make the Register button easy to notice.
- Keep the text readable.

FOOTER

- Use a clean footer layout.
- Arrange its content neatly on large screens.
- Stack the content properly on smaller screens.
- Style the footer navigation links.

RESPONSIVE DESIGN

- Add media queries for tablets and mobile phones.
- At smaller screen sizes, hide the normal navigation menu by default.
- Allow the navigation menu to appear when it receives the active class.
- Display the hamburger button on smaller screens.
- Stack multi-column layouts when there is not enough space.
- Make buttons easy to tap.
- Prevent horizontal scrolling.
- Make cards, dashboard preview content, headings, and footer content fit properly on narrow screens.

ACCESSIBILITY

- Include visible keyboard focus styles for links, buttons, and form-like controls.
- Do not remove outlines unless you replace them with a clear focus style.
- Maintain readable colour contrast.
- Respect reduced-motion preferences by reducing unnecessary animations when the user has enabled reduced motion on their device.

TECHNICAL REQUIREMENTS

- Match the class names and structure in the supplied index.html exactly.
- Do not invent a completely different HTML structure.
- Do not use external CSS libraries.
- Do not use frameworks.
- Do not use imported fonts.
- Do not include unfinished comments or placeholder instructions.
- Do not omit any required styling.

- Return one complete styles.css file from beginning to end.

Here is my complete existing index.html file:

[PASTE YOUR COMPLETE INDEX.HTML CODE HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

styles.css

The returned file should:

- Match the classes in your existing index.html.
- Style every major section of the landing page.
- Include desktop, tablet, and mobile layouts.
- Hide the hamburger button on larger screens.
- Show the hamburger button on smaller screens.
- Prepare the mobile menu to open when it receives an active class.
- Contain no HTML or JavaScript.

Read through the response before saving it.

The response should begin with CSS code, such as:

```
:root {
```

It should not begin with:

```
<html>
```

```
<script>
```

or a written explanation.

SAVE YOUR FILES

Copy the complete CSS code returned by ChatGPT.

Open Notepad.

Paste the complete code into a new empty document.

Click File.

Click Save As.

Browse to your Expense Tracker folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

styles.css

Click Save.

Check the folder carefully.

It should now contain:

index.html

styles.css

Make sure the new file is not named:

styles.css.txt

TEST YOUR WORK

Close Notepad after saving.

Open your Expense Tracker folder.

Double-click index.html.

If index.html was already open in your browser, refresh the page.

The landing page should now look styled.

Check the page from top to bottom.

Confirm that:

- ☒ The header has a clear layout.
- ☒ The logo and navigation are visible.
- ☒ The hero section looks professional.

- ✓ The buttons are styled.
- ✓ The Problem section is easy to read.
- ✓ The six feature cards appear in a grid.
- ✓ The How It Works section is organised.
- ✓ The dashboard preview resembles a software dashboard.
- ✓ The final call-to-action is clearly visible.
- ✓ The footer is styled.

Now make the browser window narrower.

The layout should adjust as the available space becomes smaller.

On a narrow screen, the normal navigation links may disappear and the hamburger button should appear.

The hamburger button will not open the menu yet because script.js has not been created. That behaviour will be added in the next lesson.

CHECKPOINT

Before moving on, confirm that:

- ✓ styles.css is inside the Expense Tracker folder.
- ✓ index.html still opens successfully.
- ✓ The landing page is no longer plain.
- ✓ Every major section has visible styling.
- ✓ The page adjusts when the browser window becomes narrower.
- ✓ The hamburger button appears on smaller screens.
- ✓ The file is named styles.css and not styles.css.txt.

COMMON BEGINNER MISTAKES

If the page remains plain after creating styles.css, first check that both files are inside the same Expense Tracker folder.

The folder should contain:

index.html

styles.css

Next, open index.html in Notepad and confirm that the following line appears inside the head section:

```
<link rel="stylesheet" href="styles.css">
```

The spelling must match the filename exactly.

These names are different:

styles.css

style.css

Styles.css

styles.css.txt

Another common mistake is saving the CSS file in a different folder. The browser can only find styles.css through the current link when it is stored beside index.html.

If only some parts of the page are styled, the CSS may not match the class names in your index.html. Return to ChatGPT, paste your complete index.html again, and ask it to regenerate the complete styles.css file using the exact existing class names.

Do not ask for a small correction or a changed snippet.

Ask for the complete updated styles.css file.

BEHIND THE SCENES

The browser reads index.html and finds this line:

```
<link rel="stylesheet" href="styles.css">
```

That line tells the browser to look for a file named styles.css in the same folder.

The browser then reads the CSS rules and applies them to matching HTML elements.

For example, an HTML element may contain a class such as:

```
class="feature-card"
```

The CSS file may contain a rule such as:

```
.feature-card {
```

```
padding: 24px;
```

```
}
```

The full stop before feature-card tells the browser that the rule belongs to elements using that class.

This is why the class names in the HTML and CSS must match exactly.

THINK LIKE A SOFTWARE DESIGNER

A professional design should help users move through a page naturally.

The headline should receive attention first.

The supporting text should explain the offer.

The main button should show the next action.

Later sections should answer the visitor's questions in a sensible order.

When reviewing your page, do not judge it only by colour. Ask whether the page is easy to understand and whether the important actions are easy to find.

A design can look attractive and still be difficult to use.

Good software design balances appearance with clarity.

CODE-READING QUESTION

Open styles.css. Find one style rule added for styling the landing page. Which selector does the rule use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a separate CSS file
- connect CSS to an existing HTML page
- save a CSS file correctly using Notepad

- style the main sections of a landing page
- prepare a responsive mobile navigation menu
- test a website at different browser widths
- identify common filename and file-location problems

In the next lesson, you will create `script.js` and make the three-line hamburger menu open and close on smaller screens.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 4

ADDING THE RESPONSIVE NAVIGATION

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create `script.js`.

This file will control the mobile navigation menu.

When the page becomes narrow, the normal navigation links are hidden and the hamburger button appears. JavaScript will make the menu open when the hamburger button is clicked and close when it is clicked again.

You will also make the menu close after a visitor selects a navigation link.

WHY THIS MATTERS

A website must remain easy to use on different screen sizes.

Desktop screens usually have enough space to display all navigation links across the header. Mobile screens have much less room, so the links are normally placed inside a menu that can be opened when needed.

HTML creates the hamburger button.

CSS controls how the button and menu look.

JavaScript controls what happens when the button is clicked.

The three files work together to create one complete feature.

BEFORE YOU CONTINUE

Confirm that your Expense Tracker folder contains:

`index.html`

`styles.css`

Open index.html in your browser.

Make the browser window narrow.

You should see the hamburger button with three horizontal lines.

The button may not do anything yet because script.js has not been created.

Before using the Build Prompt, open index.html in Notepad and copy the complete code.

You will paste it into the prompt so ChatGPT can identify the exact class names or IDs used for the hamburger button and navigation menu.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete Expense Tracker landing page with these files:

index.html

styles.css

The existing index.html already includes this script immediately before the closing body tag:

```
<script src="script.js"></script>
```

Create one complete JavaScript file named script.js.

Return the complete script.js file only.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

Everything already built must continue working.

The JavaScript must work with the exact HTML structure I will paste below.

MOBILE NAVIGATION REQUIREMENTS

- Find the existing hamburger button in index.html.
- Find the existing navigation menu in index.html.
- Use the exact existing class names or IDs.
- When the hamburger button is clicked, add or remove the same active class that styles.css expects on the navigation menu.

- Do not invent a different menu structure.
- Do not replace the existing three span elements.
- Update the hamburger button's aria-expanded value correctly.
- Set aria-expanded to true when the menu is open.
- Set aria-expanded to false when the menu is closed.
- If the hamburger button does not already have an accessible label, the JavaScript should not rewrite the HTML. Use only the existing structure.

CLOSING THE MENU

- Close the menu when a visitor clicks any navigation link inside the mobile menu.
- Close the menu when a visitor clicks outside the header navigation area.
- Close the menu when the Escape key is pressed.
- Close the menu when the browser window becomes wider than the mobile navigation breakpoint.
- After closing the menu, set aria-expanded back to false.

SMOOTH PAGE NAVIGATION

- For internal links that point to sections on index.html, allow the browser to move smoothly to the correct section.
- Do not interfere with links to login.html or register.html.
- Do not prevent normal navigation to another page.

TECHNICAL REQUIREMENTS

- Wait until the HTML page has loaded before selecting elements.
- Check that required elements exist before adding event listeners.
- Do not produce errors if an expected element is missing.
- Use clear variable names.
- Use plain JavaScript only.
- Do not use any library.
- Do not use a framework.
- Do not add Supabase code.
- Do not add authentication code.
- Do not add transaction code.
- Do not include unfinished comments.

- Return one complete script.js file from beginning to end.

IMPORTANT

The active class used in JavaScript must match the active class already prepared in styles.css.

Do not guess the navigation selectors.

Use the exact hamburger button and navigation menu selectors found in my existing index.html.

Here is my complete existing index.html file:

[PASTE YOUR COMPLETE INDEX.HTML CODE HERE]

Here is my complete existing styles.css file:

[PASTE YOUR COMPLETE STYLES.CSS CODE HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

script.js

The returned file should:

- Use the exact existing hamburger button selector.
- Use the exact existing navigation menu selector.
- Add and remove the correct active class.
- Open and close the mobile menu.
- Update aria-expanded.
- Close the menu after a navigation link is clicked.
- Close the menu when the user clicks outside it.
- Close the menu when the Escape key is pressed.
- Close the menu when the browser becomes wide again.

- Leave Login and Register links working normally.

The response should contain JavaScript only.

It should not begin with:

`<html>`

`<style>`

or a written explanation.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the complete code into a new empty document.

Click File.

Click Save As.

Browse to your Expense Tracker folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

`script.js`

Click Save.

Check your Expense Tracker folder.

It should now contain:

`index.html`

`styles.css`

`script.js`

Make sure the new file is not named:

`script.js.txt`

TEST YOUR WORK

Close Notepad after saving.

Open index.html in your browser.

If the page is already open, refresh the browser.

Make the browser window narrow until the normal navigation links disappear and the hamburger button appears.

Click the hamburger button.

The navigation menu should open.

Click the hamburger button again.

The navigation menu should close.

Open the menu again and click one of the page-section links, such as Features or How It Works.

The menu should close and the browser should move to the correct section.

Open the menu again and press the Escape key.

The menu should close.

Open the menu again and click outside the navigation area.

The menu should close.

Open the menu and then make the browser window wider.

The desktop navigation should return and the mobile menu should no longer remain open.

Finally, test the Login and Register links.

They should still point to:

login.html

register.html

The browser may still show that those files do not exist. This is expected because they will be created in Chapter 3.

CHECKPOINT

Before moving on, confirm that:

- ✓ script.js is inside the Expense Tracker folder.
- ✓ The hamburger button opens the mobile menu.
- ✓ Clicking the button again closes the menu.
- ✓ Clicking a navigation link closes the menu.
- ✓ Pressing Escape closes the menu.
- ✓ Clicking outside the menu closes it.
- ✓ Widening the browser resets the navigation.
- ✓ Login and Register links still point to the correct pages.
- ✓ The browser does not show visible JavaScript errors or stop responding.
- ✓ The file is named script.js and not script.js.txt.

COMMON BEGINNER MISTAKES

If the hamburger button does nothing, first confirm that script.js is saved inside the same folder as index.html.

Next, open index.html in Notepad and confirm that this line appears immediately before the closing body tag:

```
<script src="script.js"></script>
```

The filename must match exactly.

These names are different:

script.js

scripts.js

Script.js

script.js.txt

Another common problem is a mismatch between the active class in styles.css and the class added by script.js.

For example, if styles.css expects:

.navigation.active

but script.js adds:

open

the menu will not appear.

The same class name must be used in both files.

The selectors must also match the actual HTML.

If index.html uses:

`class="menu-toggle"`

but script.js searches for:

`hamburger-button`

JavaScript will not find the button.

When this happens, return to ChatGPT and provide the complete versions of index.html and styles.css again.

Ask for a complete corrected script.js file.

Do not ask for a small replacement line.

BEHIND THE SCENES

JavaScript can change the classes attached to an HTML element.

When the user clicks the hamburger button, script.js adds an active class to the navigation menu.

The CSS file already contains rules that show the menu when that active class is present.

When the user closes the menu, JavaScript removes the class and the menu returns to its hidden state.

JavaScript is not creating a second navigation menu.

It is changing the state of the menu that already exists in index.html.

The aria-expanded value gives additional information to people using assistive technology.

When the menu is closed, the value should be:

false

When the menu is open, the value should be:

true

This small detail makes the navigation clearer and more accessible.

THINK LIKE A SOFTWARE DESIGNER

A feature is complete only when it works in more than one simple situation.

A mobile menu should not merely open.

It should also close properly after a link is selected, when the user presses Escape, when the user clicks elsewhere, and when the page returns to desktop size.

These details may appear small, but they determine whether the website feels finished or frustrating.

When testing software, think beyond the main action.

Ask what should happen before, during, and after the user performs that action.

CODE-READING QUESTION

Open script.js. Find one function that supports adding the responsive navigation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a separate JavaScript file
- connect JavaScript to an existing HTML page
- make a hamburger button control a mobile menu
- add and remove an active class
- update aria-expanded
- close a menu in several sensible ways
- preserve links that lead to other pages
- test interaction at different browser widths

Your landing page now has structure, styling, and working mobile navigation.

In the next lesson, you will review and improve the complete public website while preserving all three connected files.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 5

COMPLETING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will improve the landing page you have already built.

Rather than creating new files, you will ask ChatGPT to review your existing website and make it more polished while preserving everything that already works.

By the end of this lesson, your landing page will look and feel like a professional software product that is ready for visitors.

WHY THIS MATTERS

Software is rarely perfect after the first version.

Professional developers spend time refining what they have already built.

They improve layouts, wording, spacing, responsiveness, accessibility, and consistency without breaking existing functionality.

Learning how to improve software safely is just as important as learning how to build it.

BEFORE YOU CONTINUE

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

Open index.html in your browser.

Confirm the following before continuing:

- ☒ The landing page opens correctly.
- ☒ The page is styled.
- ☒ The hamburger menu opens and closes on smaller screens.
- ☒ The Login and Register links point to:

login.html

register.html

Do not continue until everything above is working.

You will now ask ChatGPT to improve the existing files while keeping everything that already works.

BUILD PROMPT

PROMPT STARTS HERE

I have already built the public website for my Expense Tracker.

Everything already built must continue working.

Do not remove any existing functionality.

Return complete updated versions of every affected file.

Do not return snippets.

Do not return explanations.

Return these complete files only if changes are required:

index.html

styles.css

script.js

The following functionality must continue working exactly as before:

- Responsive navigation
- Three-line hamburger menu
- Mobile navigation
- Hero section
- Problem section
- Features section
- How It Works section

- Dashboard preview section
- Final call-to-action
- Footer
- Login link pointing to login.html
- Register link pointing to register.html

Improve the website by making the following enhancements:

DESIGN

- Improve spacing where necessary.
- Improve alignment.
- Improve typography.
- Improve colour consistency.
- Improve button appearance.
- Improve card layouts.
- Improve section spacing.

CONTENT

- Improve the wording so that it sounds more professional.
- Keep the writing clear and beginner-friendly.
- Do not make the text unnecessarily long.

ACCESSIBILITY

- Improve keyboard accessibility.
- Improve focus states where needed.
- Improve semantic HTML where appropriate.

RESPONSIVENESS

- Improve the mobile experience if necessary.
- Ensure all sections remain responsive.
- Prevent unnecessary horizontal scrolling.

CODE QUALITY

- Keep HTML well organised.
- Keep CSS organised into logical sections.
- Keep JavaScript clean and easy to follow.

IMPORTANT

Do not rename files.

Do not rename existing IDs.

Do not rename existing class names unless absolutely necessary.

Do not remove existing sections.

Do not remove existing links.

Do not remove the existing stylesheet link:

```
<link rel="stylesheet" href="styles.css">
```

Do not remove the existing JavaScript link:

```
<script src="script.js"></script>
```

The Login link must continue pointing to:

```
login.html
```

The Register link must continue pointing to:

```
register.html
```

Return complete updated files only.

Here are my existing files.

```
index.html
```

[PASTE YOUR COMPLETE INDEX.HTML]

```
styles.css
```

[PASTE YOUR COMPLETE STYLES.CSS]

```
script.js
```

[PASTE YOUR COMPLETE SCRIPT.JS]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of any files that require changes.

Each returned file should be complete from beginning to end.

Nothing that already works should stop working.

The Login and Register links must still point to:

login.html

register.html

The responsive navigation should continue working correctly.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

If ChatGPT returns an updated version of any file:

Open that file in Notepad.

Delete the existing contents.

Paste the complete updated code.

Click File.

Click Save.

Repeat this process for every updated file.

Do not create duplicate files.

Always replace the complete contents of the existing file.

After saving, your Expense Tracker folder should still contain only:

index.html

styles.css

script.js

TEST YOUR WORK

Refresh index.html in your browser.

Carefully review the entire page.

Confirm that:

- ☒ The header still appears correctly.
- ☒ The navigation still works.
- ☒ The hamburger menu still opens and closes.
- ☒ Every section is still present.
- ☒ Buttons still look correct.
- ☒ The dashboard preview is still visible.
- ☒ The footer still appears correctly.

Make the browser window narrower.

Confirm that:

- ☒ The layout adjusts correctly.
- ☒ The mobile navigation still works.
- ☒ No text runs off the screen.
- ☒ No horizontal scrolling appears.

Finally, click:

Home

Features

How It Works

Login

Register

The internal links should continue working.

The Login and Register links should still point to:

login.html

register.html

CHECKPOINT

Before moving to the next chapter, confirm that:

- ☒ The landing page looks professional.
- ☒ Every section is still present.
- ☒ The navigation still works.
- ☒ The hamburger menu still works.
- ☒ The page remains responsive.
- ☒ The Login and Register links remain unchanged.
- ☒ The browser displays no obvious problems.

COMMON BEGINNER MISTAKES

Some beginners ask ChatGPT to improve only part of a file.

This often causes important sections to disappear.

Always ask for the complete updated file.

Another common mistake is accepting changes without testing them.

Even a small improvement can accidentally remove an existing feature.

After every update, refresh the browser and test the complete page.

Professional developers test after every meaningful change.

BEHIND THE SCENES

This lesson introduces an important software development habit.

As projects become larger, developers rarely rebuild them from scratch.

Instead, they improve existing files while preserving the features that already work.

That is why your Build Prompt repeatedly tells ChatGPT to keep everything already built working.

These reminders reduce the chances of AI accidentally removing features during later updates.

You will continue using this approach throughout the rest of the workbook.

THINK LIKE A SOFTWARE DESIGNER

Good software evolves.

Each improvement should make the application better without making it unstable.

Before accepting any change, ask yourself two questions:

"Did this make the software better?"

"What else might this change have affected?"

Developing this habit will help you build reliable software as your projects become more advanced.

CODE-READING QUESTION

Open index.html. Find the part of the page added for completing the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- improve an existing project safely

- ask ChatGPT for complete updated files
- preserve existing functionality
- test a project after making improvements
- recognise why software refinement is part of the development process

Congratulations.

You have completed Chapter 1.

In the next chapter, you will connect your project to Supabase and prepare it for user accounts and online data storage.

CHAPTER SUMMARY

Congratulations.

You have completed the first chapter of your Expense Tracker project.

Although your software cannot yet register users or store expenses, you have already built something important.

You now have a professional public website that introduces your software, explains its benefits, and encourages visitors to create an account.

Along the way, you also learned one of the most important principles in software development: each file has a specific responsibility.

Your HTML file provides the structure of the page.

Your CSS file controls how the page looks.

Your JavaScript file controls how the page behaves.

Keeping these responsibilities separate makes software easier to understand, maintain, and improve.

You also experienced something that professional software developers do every day.

Instead of trying to build everything at once, you built one working piece at a time, tested it carefully, and then improved it before moving forward.

That same approach will continue throughout the rest of this workbook.

CHAPTER MILESTONE

By the end of Chapter 1, you should have successfully built:

- ✓ A complete landing page
- ✓ Responsive navigation
- ✓ A three-line hamburger menu
- ✓ Hero section
- ✓ Problem section
- ✓ Features section
- ✓ How It Works section
- ✓ Dashboard preview section
- ✓ Final call-to-action section
- ✓ Footer
- ✓ Login navigation link
- ✓ Register navigation link
- ✓ Mobile-friendly layout
- ✓ Working HTML, CSS and JavaScript files

Your Expense Tracker folder should now contain exactly these files:

index.html

styles.css

script.js

Nothing else has been created yet.

BEFORE MOVING TO CHAPTER 2

Before continuing, open your project one more time and complete this final review.

Open your Expense Tracker folder.

Double-click index.html.

Your browser should display a complete, professional landing page.

Check the following carefully.

Navigation

- ✓ The logo is visible.
- ✓ The navigation links appear correctly.
- ✓ The Login link points to login.html.
- ✓ The Register link points to register.html.

Mobile Navigation

- ✓ The hamburger button appears on smaller screens.
- ✓ The menu opens.
- ✓ The menu closes.
- ✓ The menu closes after selecting a navigation link.

Landing Page

- ✓ Every section appears in the correct order.
- ✓ Buttons look correct.
- ✓ Cards display properly.
- ✓ The dashboard preview looks organised.
- ✓ The footer appears correctly.

Responsiveness

- ✓ The layout adjusts as the browser becomes narrower.
- ✓ No content disappears unexpectedly.
- ✓ No horizontal scrolling appears.

Files

- ✓ index.html
- ✓ styles.css
- ✓ script.js

Each file should be saved inside the same Expense Tracker folder.

TRANSITION TO CHAPTER 2

Your website now has a professional public face, but it is still a static website.

Visitors can click the Login and Register links, but those pages do not exist yet.

Your software also has nowhere to store user accounts or financial information.

The next step is to connect your project to an online database.

You will use Supabase to handle user authentication and data storage throughout the rest of this workbook.

Before users can register, log in, or save transactions, your website must know how to communicate securely with your Supabase project.

That is exactly what you will build in the next chapter.

CHAPTER 2

CONNECTING TO SUPABASE

CHAPTER INTRODUCTION

Modern software applications usually store information online instead of inside the visitor's browser.

This allows users to sign in from different devices, retrieve their information later, and securely manage their own data.

In this workbook, Supabase will provide that online service.

It will manage user accounts, authentication, and the database that stores each person's financial transactions.

During this chapter, you will create your first Supabase project, obtain the information needed to connect your website to it, and confirm that the connection works correctly.

Although no users will register during this chapter, you are laying the foundation for everything that follows.

Think of this chapter as installing the plumbing before turning on the water.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Create a Supabase account
- Create a new Supabase project
- Understand the Project URL
- Understand the Publishable Key
- Create a reusable connection file
- Build a connection test page
- Verify that your website can communicate with Supabase

By the end of this chapter, your project will be fully prepared for the authentication system you will build in Chapter 3.

LESSON 1

CREATING YOUR SUPABASE PROJECT

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create your own Supabase project.

Every student should create a separate project.

Your project will contain your own users, your own database, and your own application settings.

No one else should use your project, and you should never use someone else's project.

WHY THIS MATTERS

Your website needs a secure place to store information.

Without an online database, users would not be able to create accounts, log in, or save their expense records.

Supabase provides all of these services.

Once your project has been created, your website will be able to communicate with it using a secure connection.

BEFORE YOU CONTINUE

Make sure you have:

- ☒ A working internet connection.
- ☒ A web browser.
- ☒ Your Expense Tracker project folder.

No changes will be made to your project files during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete the following steps.

1. Open your web browser.

2. Visit:

<https://supabase.com>

3. Create a free Supabase account if you do not already have one.

4. Sign in to your account.

5. Click **New Project**.

6. Choose your organisation.

7. Enter a project name.

You can use:

Expense Tracker

8. Create a strong database password.

Store this password somewhere safe.

You may need it later if you choose to manage your database directly.

Do not share your password with anyone.

9. Choose the region closest to you.

10. Click **Create New Project**.

Supabase will begin creating your project.

This may take a few minutes.

Wait until the project status shows that it is ready before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed directly inside your web browser.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

When your project finishes creating, confirm that you can open its dashboard.

You should be able to see your new Supabase project.

Do not continue until the project has finished loading successfully.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your Supabase account has been created.
- ☒ Your new project has been created.
- ☒ The project dashboard opens successfully.
- ☒ The project status shows that it is ready.

COMMON BEGINNER MISTAKES

Some students close the browser while the project is still being created.

Wait until Supabase finishes preparing your project before moving to the next lesson.

Another common mistake is forgetting the database password.

Store it somewhere safe before leaving this page.

Remember that your website will not use this password.

It is only for managing your project.

Later in this workbook, your website will connect using a Publishable Key instead.

BEHIND THE SCENES

When you created your project, Supabase automatically prepared several services for you.

These include a PostgreSQL database, user authentication, storage, security settings, and APIs that allow your website to communicate with your project.

You do not need to install or configure these services yourself.

Supabase prepares them automatically.

THINK LIKE A SOFTWARE DESIGNER

Every software application needs its own secure environment.

Creating a separate project means your users and data remain completely independent from everyone else's.

As your software grows, keeping projects separate makes them easier to manage, test, and secure.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a Supabase account
- create a new Supabase project
- understand why every project has its own database
- prepare your online backend for the rest of this workbook

In the next lesson, you will locate the two pieces of information every Supabase application needs to connect securely: the Project URL and the Publishable Key.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 2

UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will locate the two pieces of information your website needs to communicate with Supabase.

These are:

- The Project URL
- The Publishable Key

In the next lesson, you will place these values inside a reusable connection file that every page in your application will use.

WHY THIS MATTERS

Your website cannot simply guess where your database is located.

It needs to know exactly which Supabase project belongs to you.

The Project URL tells your website where your Supabase project lives.

The Publishable Key tells Supabase that your website is allowed to communicate with that project.

These two values work together whenever your application creates user accounts, signs users in, or stores data.

BEFORE YOU CONTINUE

Make sure:

✓ Your Supabase project has finished creating.

✓ You are signed in to Supabase.

✓ Your project dashboard is open.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, locate your Project URL and Publishable Key.

Follow these steps carefully.

1. Open your Supabase project.

2. In the left-hand menu, click:

Project Settings

3. Click:

Data API

You should see several pieces of information about your project.

Locate:

Project URL

It will look similar to:

<https://your-project-id.supabase.co>

Copy this value somewhere safe.

Do not change it.

Next, locate:

Publishable Key

This is the key your website will use.

Copy it somewhere safe.

You will need both values in the next lesson.

IMPORTANT

Your Project URL should end with:

.supabase.co

Do not add:

/rest/v1/

The correct format is:

<https://your-project-id.supabase.co>

Not:

<https://your-project-id.supabase.co/rest/v1/>

IMPORTANT

Use the Publishable Key only.

Do NOT use:

- Service Role Key
- Secret Key
- Database password

Your website should never contain secret keys.

Only the Publishable Key belongs inside your project files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your Supabase dashboard.

SAVE YOUR FILES

No project files are created during this lesson.

Keep your Project URL and Publishable Key somewhere safe because you will use them in the next lesson.

TEST YOUR WORK

Confirm that you have copied:

- ☒ Your Project URL

☒ Your Publishable Key

Check the Project URL carefully.

It should look similar to:

`https://your-project-id.supabase.co`

There should be nothing after:

`.supabase.co`

CHECKPOINT

Before moving on, confirm that:

☒ You have located your Project URL.

☒ You have located your Publishable Key.

☒ You copied the Publishable Key and not a secret key.

☒ Your Project URL does not contain:

`/rest/v1/`

COMMON BEGINNER MISTAKES

One of the most common mistakes is copying the wrong key.

Supabase displays several keys.

For this workbook, you should always use the Publishable Key.

Never use the Service Role Key.

Never use a Secret Key.

Those keys are intended for secure server environments and should never be placed inside files that visitors can download through their web browser.

Another common mistake is copying the REST API address instead of the Project URL.

Always use the base Project URL that ends with:

`.supabase.co`

BEHIND THE SCENES

When your website communicates with Supabase, it sends requests to your Project URL.

Those requests include your Publishable Key.

Supabase then checks whether the request is allowed before processing it.

Even though the Publishable Key is visible inside your website, it does not allow visitors to bypass your security.

That protection comes from authentication and Row Level Security, which you will build later in this workbook.

THINK LIKE A SOFTWARE DESIGNER

Professional developers separate public information from secret information.

The Project URL and Publishable Key are designed to be used by websites.

Secret keys are designed for secure servers.

Understanding the difference is an important step towards building secure applications.

WHAT YOU LEARNED

In this lesson you learned:

- what the Project URL is
- what the Publishable Key is
- where to find both values
- why the Publishable Key is safe to use in your website
- why secret keys should never be placed inside public files

In the next lesson, you will create a reusable connection file that allows every page in your project to communicate with your Supabase project using these two values.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 3

CREATING YOUR SUPABASE CONNECTION FILE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a reusable JavaScript file that connects your Expense Tracker to your Supabase project.

Rather than writing the connection code on every page, you will place it in a single file named:

`supabase-config.js`

Every page that needs to communicate with Supabase will use this file.

This approach keeps your project organised and makes future updates much easier.

WHY THIS MATTERS

As software grows, the same code is often needed in several places.

Instead of copying the same connection code into every JavaScript file, professional developers place it in one reusable file.

Whenever another page needs to communicate with Supabase, it simply loads this connection file.

This reduces mistakes and keeps the project easier to maintain.

BEFORE YOU CONTINUE

Before starting this lesson, make sure you have:

- ☒ Created your Supabase project.
- ☒ Copied your Project URL.

✓ Copied your Publishable Key.

Your Expense Tracker folder should currently contain:

index.html

styles.css

script.js

You are about to add a fourth file.

BUILD PROMPT

PROMPT STARTS HERE

Create one complete JavaScript file named:

supabase-config.js

Return the complete file only.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

Requirements

Create a reusable Supabase connection file.

Use the official Supabase browser client.

Create the client using this format exactly:

```
const supabaseClient = window.supabase.createClient(
```

```
"https://your-project-id.supabase.co",
```

```
"YOUR_PUBLISHABLE_KEY"
```

```
);
```

Replace the placeholder values with these:

Project URL:

PASTE YOUR PROJECT URL HERE

Publishable Key:

PASTE YOUR PUBLISHABLE KEY HERE

Requirements

- Create only one Supabase client.
- Store it in a constant named:

`supabaseClient`

- Do not use:

`/rest/v1/`

inside the Project URL.

- Do not use:

Service Role Key

- Do not use:

Secret Key

- Do not use:

Database password

- Include a short comment at the top of the file explaining that this file creates the shared Supabase connection used throughout the project.
- Return one complete JavaScript file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`supabase-config.js`

The file should:

- ☒ Create a Supabase client.
- ☒ Use your Project URL.
- ☒ Use your Publishable Key.
- ☒ Create the client using:

```
const supabaseClient = window.supabase.createClient(...)
```

- ☒ Contain JavaScript only.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Browse to your Expense Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

supabase-config.js

Click Save.

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

Make sure the new file is not named:

supabase-config.js.txt

TEST YOUR WORK

Open your Expense Tracker folder.

Confirm that the new file appears.

At this stage, the file is not yet connected to any web page.

That is expected.

The connection will be tested in the next lesson.

CHECKPOINT

Before moving on, confirm that:

✓ supabase-config.js has been created.

✓ The filename is correct.

✓ The Project URL ends with:

.supabase.co

✓ The Project URL does not contain:

/rest/v1/

✓ The Publishable Key is being used.

✓ The Service Role Key is not being used.

COMMON BEGINNER MISTAKES

One common mistake is accidentally copying the REST API address instead of the Project URL.

Always use the base URL that ends with:

.supabase.co

Another common mistake is copying the wrong key.

Only the Publishable Key belongs inside this file.

Never use the Service Role Key or any secret key.

Some beginners also create multiple Supabase clients in different files.

Throughout this workbook, you will create only one shared client.

Every page that needs Supabase will use this same connection file.

BEHIND THE SCENES

The line:

```
const supabaseClient = window.supabase.createClient(...)
```

creates a reusable connection between your website and your Supabase project.

Think of it as opening a secure communication channel.

Later, when users register, sign in, add transactions, edit records, or delete records, those actions will all use this same Supabase client.

This is why keeping the connection in one file makes the project much easier to maintain.

THINK LIKE A SOFTWARE DESIGNER

One of the signs of well-designed software is avoiding unnecessary duplication.

If the same code appears in several different places, it usually becomes harder to maintain.

Whenever possible, create something once and reuse it everywhere.

That is exactly what you are doing by creating a shared Supabase connection file.

CODE-READING QUESTION

Open `supabase-config.js`. Which line creates the Supabase client, and which two saved values does that line use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a reusable connection file
- create a Supabase client
- use the correct Project URL
- use the correct Publishable Key
- avoid using secret keys
- organise shared code professionally

In the next lesson, you will connect your website to this file and build a simple connection test page to confirm that your website can successfully communicate with your Supabase project.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 4

BUILDING A CONNECTION TEST PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a simple page that checks whether your website can successfully communicate with your Supabase project.

This page is only for testing.

It will not become part of the finished Expense Tracker.

Instead, it allows you to confirm that your connection is working before you begin building user registration and login.

Testing now helps you discover connection problems early, when they are much easier to fix.

WHY THIS MATTERS

Professional software developers test important systems before building on top of them.

Imagine spending several hours building registration and login, only to discover that the application was never connected to Supabase correctly.

Finding the problem at that stage would be much more difficult.

By testing the connection first, you know that the foundation is working before adding more features.

BEFORE YOU CONTINUE

Before starting this lesson, your Expense Tracker folder should contain:

index.html

styles.css

script.js

supabase-config.js

You should also have:

☒ Your Supabase project

☒ Your Project URL

☒ Your Publishable Key

BUILD PROMPT

PROMPT STARTS HERE

I already have these files in my project:

index.html

styles.css

script.js

supabase-config.js

Everything already built must continue working.

Do not modify any existing files.

Create one new HTML file named:

test-connection.html

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

Requirements

Use proper HTML5 structure.

Include an appropriate page title.

Inside the <head> section, include simple CSS that creates a clean, centred layout suitable for a test page.

Inside the <body>, display:

- A heading that says:

Supabase Connection Test

- A short paragraph explaining that the page is checking the connection.

- A status box that initially displays:

Checking connection...

Give this status box the ID:

connection-status

At the end of the body, load the following scripts in this exact order:

1.

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

2.

```
<script src="supabase-config.js"></script>
```

3.

A JavaScript script inside the page that:

- Confirms that the variable:

supabaseClient

exists.

- If it exists, change the text inside the status box to:

✔ Connected successfully to Supabase.

- Change the status box to a success colour.

- If it does not exist, display:

✘ Unable to connect to Supabase.

- Change the status box to an error colour.

Requirements

- Use the existing supabase-config.js file.
- Do not create another Supabase client.
- Do not hard-code the Project URL.
- Do not hard-code the Publishable Key.
- Do not use authentication.
- Do not create database tables.
- Do not create users.
- Return one complete HTML file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

test-connection.html

The page should:

- ☒ Load the Supabase CDN.
- ☒ Load supabase-config.js.
- ☒ Load the scripts in the correct order.
- ☒ Check whether supabaseClient exists.
- ☒ Display a clear success or error message.
- ☒ Return only one complete HTML file.

SAVE YOUR FILES

Copy the complete HTML returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Browse to your Expense Tracker folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

test-connection.html

Click Save.

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TEST YOUR WORK

Open your Expense Tracker folder.

Double-click:

test-connection.html

Your browser should open the page.

After a moment, the message should change from:

Checking connection...

to:

✓ Connected successfully to Supabase.

If you see this message, your shared connection file is working correctly.

If you instead see:

✗ Unable to connect to Supabase.

do not continue to the next chapter.

Return to the previous lessons and carefully check:

- Your Project URL
- Your Publishable Key
- Your supabase-config.js file
- The script loading order

CHECKPOINT

Before moving on, confirm that:

- ✓ test-connection.html opens correctly.
- ✓ The page loads without errors.
- ✓ The Supabase CDN loads before supabase-config.js.
- ✓ supabase-config.js loads successfully.
- ✓ The page displays:
- ✓ Connected successfully to Supabase.

COMMON BEGINNER MISTAKES

One common mistake is loading the files in the wrong order.

The correct order is always:

1. Supabase CDN
2. supabase-config.js
3. The page's own JavaScript

If supabase-config.js loads before the Supabase library, the browser will not recognise:

`window.supabase`

and the connection cannot be created.

Another common mistake is creating a second Supabase client inside the page.

Do not do this.

Your project should always use the shared client created inside:

`supabase-config.js`

A third common mistake is accidentally editing the Project URL or Publishable Key.

If the page reports that it cannot connect, compare both values carefully with the ones shown inside your Supabase project.

BEHIND THE SCENES

Notice that this page never creates another connection.

Instead, it loads the existing connection file.

That file creates:

supabaseClient

The page simply checks whether that shared object is available.

Later in this workbook, every page that needs Supabase will follow the same pattern.

Each page will load:

1. The Supabase browser library.
2. supabase-config.js.
3. Its own JavaScript file.

Because every page follows the same structure, the project stays organised and much easier to maintain.

THINK LIKE A SOFTWARE DESIGNER

Reusable code is one of the foundations of professional software development.

Instead of solving the same problem repeatedly, developers create one reliable solution and use it throughout the application.

By creating a shared connection file, every future page can communicate with Supabase in exactly the same way.

That consistency makes large projects much easier to build, debug, and maintain.

CODE-READING QUESTION

Open test-connection.html. Find the part of the page added for building a connection test page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a dedicated connection test page
- load the Supabase browser library
- use your shared Supabase connection file
- load JavaScript files in the correct order
- confirm that your website can communicate with Supabase
- identify common connection problems before building authentication

In the next lesson, you will perform a final review of your Supabase connection before moving on to build the complete authentication system.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 5

FINAL CONNECTION REVIEW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

This lesson is your final checkpoint before building the authentication system.

You are not creating any new files.

Instead, you will carefully review everything you have built so far to make sure your project is ready for user registration and login.

Taking a few minutes to verify your work now can save hours of troubleshooting later.

WHY THIS MATTERS

As software projects grow, small mistakes become harder to find.

A missing file, an incorrect Project URL, or loading scripts in the wrong order may not seem serious now, but they can prevent your authentication system from working correctly.

Professional developers regularly stop to review their work before adding major new features.

You are about to build the most important part of your application so this is the right time to make sure your foundation is solid.

BEFORE YOU CONTINUE

Your Expense Tracker folder should now contain exactly these files:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

If any of these files are missing, return to the earlier lessons before continuing.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete the following review.

1. Open your Expense Tracker folder.
2. Confirm that every file is present.
3. Open:

test-connection.html

4. Confirm that the page displays:

✓ Connected successfully to Supabase.

5. Open:

supabase-config.js

Check that:

- The Project URL ends with:

.supabase.co

- The URL does not contain:

/rest/v1/

- The Publishable Key is being used.
- The Supabase client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

6. Open:

test-connection.html

Confirm that the scripts are loaded in this exact order:

First

The Supabase CDN

Second

supabase-config.js

Third

The page's own JavaScript

Do not continue until every item has been checked.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by reviewing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Landing Page

- ☒ index.html opens correctly.
- ☒ The website is fully styled.
- ☒ The navigation works.
- ☒ The hamburger menu works.
- ☒ Login points to login.html.
- ☒ Register points to register.html.

Supabase

- ☒ The Supabase project exists.
- ☒ The Project URL is correct.
- ☒ The Publishable Key is correct.
- ☒ supabase-config.js creates one shared Supabase client.

Connection Test

- ☒ test-connection.html opens successfully.

- ✓ The page displays:
- ✓ Connected successfully to Supabase.

Project Folder

- ✓ index.html
- ✓ styles.css
- ✓ script.js
- ✓ supabase-config.js
- ✓ test-connection.html

CHECKPOINT

Before moving to Chapter 3, confirm that:

- ✓ Every required file exists.
- ✓ The landing page still works.
- ✓ The connection test succeeds.
- ✓ Your Project URL is correct.
- ✓ Your Publishable Key is correct.
- ✓ Script loading order is correct.
- ✓ No duplicate Supabase clients exist.

COMMON BEGINNER MISTAKES

- Some students continue building even after their connection test fails.
- Do not do that.
- If the connection test reports an error, solve that problem first.
- Authentication depends on the connection working correctly.
- Another common mistake is editing supabase-config.js while trying to solve another problem.
- That file should only contain your shared Supabase connection.
- Avoid adding unrelated code to it.

BEHIND THE SCENES

At this point, your project has everything it needs to begin communicating with Supabase.

What it does not yet have is a way for people to identify themselves.

The next chapter introduces authentication.

Authentication answers an important question every secure application must ask:

"Who is using this software?"

Once the application knows who the current user is, it can safely show that person's own information while protecting everyone else's.

That is exactly what you will build next.

THINK LIKE A SOFTWARE DESIGNER

Every secure application is built in layers.

The first layer is the public website.

The second layer is the connection to the backend.

The third layer is authentication.

Only after these layers are complete does it make sense to start storing user data.

Building software in this order keeps the project organised and reduces the number of problems you need to solve at any one time.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an application before adding major features
- verify your shared Supabase connection
- confirm the correct script loading order
- identify common configuration problems
- understand why authentication comes before database operations

CHAPTER SUMMARY

Congratulations.

You have successfully connected your Expense Tracker to Supabase.

Although visitors still cannot register or log in, your application now knows how to communicate with your online backend.

You created a reusable connection file that every future page will use.

You also learned an important principle that will continue throughout this workbook.

Whenever a page needs Supabase, it should always load the required scripts in the same order:

1. The Supabase browser library
2. `supabase-config.js`
3. The page's own JavaScript file

Following this order consistently prevents many common connection problems.

You also confirmed that your website can communicate successfully with your Supabase project before moving on to more advanced features.

That verification gives you confidence that the foundation of your application is working correctly.

CHAPTER MILESTONE

By the end of Chapter 2, you have successfully completed:

- ✓ Created a Supabase project
- ✓ Located your Project URL
- ✓ Located your Publishable Key
- ✓ Created supabase-config.js
- ✓ Built a connection test page
- ✓ Verified your Supabase connection
- ✓ Learned the correct script loading order
- ✓ Prepared your project for authentication

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TRANSITION TO CHAPTER 3

Your application now has a professional public website and a working connection to Supabase.

The next step is to give people their own accounts.

In Chapter 3, you will build the complete authentication system in one coordinated build.

By the end of that chapter, visitors will be able to:

- Create an account
- Verify their email address
- Sign in securely

- Be redirected automatically to the correct page
- Access a protected dashboard
- Sign out safely

Because the authentication system is built as one complete unit, every page already knows where to send the user after each step.

No temporary pages or broken redirects will exist.

Everything will work together as one complete authentication flow from the very beginning.

CHAPTER 3

BUILDING THE AUTHENTICATION SYSTEM

CHAPTER INTRODUCTION

Until now, anyone can visit your Expense Tracker website, but nobody can create an account or sign in.

That changes in this chapter.

You are about to build the complete authentication system for your application.

Instead of building one page at a time, you will build the entire authentication flow as one connected system.

This is exactly how professional developers approach authentication.

Every page in the authentication process depends on another page.

If one page is missing, users reach a dead end.

By building everything together, every redirect already has a valid destination and every page understands its role in the overall system.

By the end of this chapter, a visitor will be able to:

- Register for an account
- Receive a verification email
- Verify their email address
- Sign in
- Be redirected to the dashboard
- Be prevented from opening the dashboard without signing in
- Sign out safely

Everything will work together as one complete authentication system.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- register.html
- login.html
- dashboard.html
- auth.css
- register.js
- login.js
- dashboard.js

You will also update:

- index.html

These pages will work together using:

- The Supabase JavaScript library
- supabase-config.js

The authentication flow will work like this:

Visitor opens:

index.html

↓

Clicks Register

↓

register.html

↓

Creates an account

↓

Verifies email

↓

login.html

↓

Signs in

↓

dashboard.html

↓

Signs out

↓

login.html

If someone tries to open dashboard.html without signing in, they will automatically be redirected to login.html.

LESSON 1

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete authentication system for your Expense Tracker.

Rather than generating one page at a time, ChatGPT will generate every authentication page together.

This ensures that:

- Every page links correctly.
- Every redirect already has a destination.
- Every JavaScript file loads correctly.
- Every stylesheet is linked correctly.
- Every Supabase script loads in the correct order.
- Everything already built continues working.

WHY THIS MATTERS

Authentication is one of the most connected parts of any software application.

The registration page redirects to the login page.

The login page redirects to the dashboard.

The dashboard redirects back to the login page after logout.

Because these pages depend on one another, they should be generated together.

Doing so greatly reduces errors and missing links.

BEFORE YOU CONTINUE

Before continuing, confirm that your Expense Tracker folder contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Also confirm that:

☒ test-connection.html displays:

Connected successfully to Supabase.

Do not continue until everything above is working correctly.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker project.

Everything already built must continue working.

My project currently contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

The public landing page is already complete.

The responsive navigation already works.

The Supabase connection has already been tested successfully.

Do not remove or break any existing functionality.

Generate the complete authentication system.

Return complete files only.

Do not return snippets.

Do not return explanations.

Return the following complete files:

1.

Updated index.html

2.

register.html

3.

login.html

4.

dashboard.html

5.

auth.css

6.

register.js

7.

login.js

8.

dashboard.js

GENERAL REQUIREMENTS

Everything already built must continue working.

Every HTML page must be complete.

Every HTML page must include proper HTML5 structure.

Every HTML page must include appropriate titles and meta tags.

Use semantic HTML throughout.

Use professional, modern styling.

The logo on every page should display:

Expense Tracker

The logo should link to:

index.html

INDEX.HTML

Return a complete updated version of index.html.

Preserve:

- Hero section
- Features
- Navigation
- Footer
- Responsive menu
- Existing stylesheet link
- Existing JavaScript link

Ensure Login points to:

login.html

Ensure Register points to:

register.html

REGISTER.HTML

Create a professional registration page.

Include:

- Logo linking to index.html
- Full Name
- Email Address
- Password
- Confirm Password
- Register button
- Link to login.html

Friendly validation messages.

Loading state while creating account.

Disable Register button while processing.

Successful registration should display a clear success message explaining that the user should verify their email before signing in.

After successful registration, redirect users to:

login.html

Do not automatically sign the user in.

LOGIN.HTML

Create a professional login page.

Include:

- Logo linking to index.html
- Email
- Password
- Login button
- Link to register.html

Friendly error messages.

Loading state.

Disable Login button while processing.

If login succeeds:

Redirect immediately to:

dashboard.html

If a user is already signed in and opens login.html directly:

Automatically redirect to:

dashboard.html

DASHBOARD.HTML

Create a professional dashboard page.

Include:

- Logo linking to index.html
- Welcome heading
- Placeholder area where transactions will later appear
- Summary cards for:

Income

Expenses

Balance

The values can temporarily display:

№0.00

Include:

Logout button

If no authenticated user exists:

Immediately redirect to:

login.html

AUTH.CSS

Create one shared stylesheet for:

register.html

login.html

dashboard.html

Use:

Responsive layout

Professional card design

Accessible form styling

Professional buttons

Loading styles

Success message styles

Error message styles

Responsive dashboard layout

REGISTER.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

const supabaseClient

already created inside:

supabase-config.js

Do not create another Supabase client.

Validate:

- Full Name

- Email

- Password

- Confirm Password

Passwords must match.

Create the account using Supabase Authentication.

When calling:

`supabaseClient.auth.signUp`

include an email confirmation destination.

Create it from the website that is currently open:

``${window.location.origin}/login.html``

Pass that address as:

`emailRedirectTo`

inside the sign-up options.

Do not hardcode a Netlify website name.

Using:

`window.location.origin`

allows the same complete files to work when the website address changes.

Display friendly success and error messages.

Disable the button while processing.

After successful registration:

Inform the user to open the verification email.

Redirect to:

`login.html`

Do not automatically sign the user in from the registration form.

LOGIN.JS

Requirements:

Load after:

1.

Supabase CDN

2.

supabase-config.js

Use the existing:

supabaseClient

Do not create another Supabase client.

Authenticate users.

Display friendly loading messages.

Display friendly error messages.

Disable Login button while processing.

If login succeeds:

Redirect to:

dashboard.html

DASHBOARD.JS

Requirements

Load after:

1.

Supabase CDN

2.

supabase-config.js

Use:

supabaseClient

Check whether the current user is authenticated.

If not authenticated:

Redirect immediately to:

login.html

Display the authenticated user's email address somewhere on the dashboard.

Implement Logout.

After logout:

Redirect to:

login.html

SCRIPT LOADING ORDER

register.html

must load scripts in this exact order:

1.

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

2.

```
<script src="supabase-config.js"></script>
```

3.

```
<script src="register.js"></script>
```

login.html

must load scripts in this exact order:

1.

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

2.

```
<script src="supabase-config.js"></script>
```

3.

```
<script src="login.js"></script>
```

dashboard.html

must load scripts in this exact order:

1.

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

2.

```
<script src="supabase-config.js"></script>
```

3.

```
<script src="dashboard.js"></script>
```

IMPORTANT

Return every requested file completely.

Do not omit any file.

Do not return partial updates.

Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return eight complete files:

- ☒ Updated index.html
- ☒ register.html
- ☒ login.html
- ☒ dashboard.html
- ☒ auth.css
- ☒ register.js
- ☒ login.js
- ☒ dashboard.js

Every HTML page should already contain the correct stylesheet links and JavaScript files.

Every page should load the Supabase scripts in the correct order.

No file should depend on a page that does not exist.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

For each file returned by ChatGPT:

Open Notepad.

If the file already exists, replace its entire contents with the updated version.

If the file is new, create a new document.

Click File.

Click Save As.

Browse to your Expense Tracker folder.

Choose:

Save as type:

All Files

Save each file using its exact filename.

After saving, your Expense Tracker folder should contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

Make sure none of the new files end with:

.txt

CHAPTER 3

BUILDING THE AUTHENTICATION SYSTEM

LESSON 1 (CONTINUED)

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

TEST YOUR WORK

Do not assume the authentication system works.

Authentication must be tested from a website with an HTTPS address.

Do not test this chapter by double-clicking the HTML files in your project folder.

The files still remain inside your project folder.

You will simply place a test copy of the complete folder online before testing.

BEFORE YOU START THE TESTS

1.

Save every updated file in Notepad.

2.

Deploy the complete project folder to Netlify.

If you already created a Netlify test website for this project, deploy the updated folder to the same website.

3.

Copy the website address supplied by Netlify.

It should begin with:

https://

For example:

`https://your-site-name.netlify.app`

Your own website address will be different.

4.

Open your Supabase project.

Go to:

Authentication

Then open:

URL Configuration

5.

Set the Site URL to your Netlify website address.

Do not add a file name to the Site URL.

6.

Add this complete address to Redirect URLs:

`https://your-site-name.netlify.app/login.html`

Replace the example website name with your real Netlify website name.

7.

Make sure the latest version of every file has been deployed.

8.

Open the project from its Netlify HTTPS address.

Keep using that address throughout every test below.

TEST 1

PUBLIC WEBSITE

Open the deployed index.html page.

Confirm that:

- ☒ The landing page opens correctly.
- ☒ The page is fully styled.
- ☒ The responsive navigation still works.
- ☒ Login opens the deployed login.html page.
- ☒ Register opens the deployed register.html page.

Look at the browser address bar.

The address must begin with:

https://

Nothing already built should have stopped working.

TEST 2

REGISTRATION PAGE

Open the deployed register.html page.

Attempt to submit the form without entering any information.

Friendly validation messages should appear.

Complete the form using an email address that has not already been used for this test.

Click:

Register

The Register button should become disabled while the request is being processed.

A loading message should appear.

After successful registration:

You should see a message asking you to verify your email.

TEST 3

EMAIL VERIFICATION

Open your email inbox.

Locate the verification email from Supabase.

Open the email.

Click the verification link.

Supabase should verify the email and return the browser to your deployed website.

The returned address should use your Netlify website name.

It should not begin with:

file://

Depending on the authentication response, you may briefly see login.html or the application may recognise the new session and open the dashboard.

Both results confirm that the browser returned to the correct website.

If the login page remains open, sign in using the email address and password you registered.

If the verification email does not arrive immediately:

Wait a few minutes.

Refresh your inbox.

Check your Spam or Junk folder.

If the verification link opens the wrong website:

Return to Supabase URL Configuration.

Check the Site URL and Redirect URLs carefully.

Correct them before creating another test account.

TEST 4**LOGIN**

Open the deployed login.html page.

Enter an incorrect password.

A friendly error message should appear.

Now enter the correct password.

Click:

Login

The Login button should become disabled while signing in.

A loading message should appear.

After a successful login:

The browser should open the deployed dashboard.html page.

TEST 5**AUTHENTICATED SESSION**

After signing in successfully:

Confirm that the protected dashboard opens.

Confirm that your email address appears.

Refresh the page.

The dashboard should remain open because the same website still has your authenticated session.

Confirm that the Income, Expenses and Balance cards appear.

Confirm that the Logout button appears.

Click:

Logout

The browser should return to the deployed login.html page.

While signed out, open the complete deployed dashboard.html address.

The application should immediately return you to the deployed login.html page.

Sign in again.

While signed in, open the complete deployed login.html address.

The application should immediately return you to the deployed dashboard.html page.

CHECKPOINT

Before moving to the next lesson, confirm that:

- ☒ Registration works.
- ☒ Validation messages appear.
- ☒ Loading states appear.
- ☒ Register button becomes disabled while processing.
- ☒ Verification email is received.
- ☒ Email verification succeeds.
- ☒ Login works.
- ☒ Friendly error messages appear when appropriate.
- ☒ Login redirects to dashboard.html.
- ☒ Dashboard is protected.
- ☒ Logout works correctly.
- ☒ Logged-out users cannot access dashboard.html.
- ☒ Logged-in users are redirected away from login.html.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to verify the email address before attempting to sign in.

If email verification is enabled in your Supabase project, users must complete this step before they can use their account.

Another common mistake is loading JavaScript files in the wrong order.

Every authentication page must load scripts in exactly this order:

1.

Supabase Browser Library

2.

supabase-config.js

3.

The page's own JavaScript file

Changing this order can prevent the authentication system from working correctly.

Some students also accidentally create another Supabase client inside register.js, login.js or dashboard.js.

Do not do this.

Every page should use the shared client already created in:

supabase-config.js

Another common mistake is allowing users to open dashboard.html without checking whether they are signed in.

Every protected page should verify authentication before displaying its contents.

BEHIND THE SCENES

Your authentication system is now working as one connected process.

When a visitor creates an account, Supabase stores the user's authentication details securely.

After email verification, Supabase recognises that the account is ready to be used.

When the user signs in successfully, Supabase creates a session.

That session allows your application to recognise the user while they continue using the software.

The dashboard checks for this session every time it opens.

If no valid session exists, the dashboard immediately redirects the visitor back to the login page.

This prevents unauthorised access without requiring you to build your own authentication system from scratch.

THINK LIKE A SOFTWARE DESIGNER

Authentication is much more than creating a login form.

A complete authentication system answers several important questions.

Can someone create an account?

Can they verify ownership of their email address?

Can they sign in?

Can they sign out?

Can unauthorised visitors reach protected pages?

Can authenticated users move smoothly through the application?

Only when every answer is "yes" do you have a complete authentication system.

Professional developers think about the entire user journey, not just individual pages.

CODE-READING QUESTION

Open `register.js`. Find one function that supports building the complete authentication system. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete authentication system
- connect multiple pages into one user journey
- preserve previous functionality while adding new features

- register new users
- verify email addresses
- sign users in
- protect dashboard pages
- display authenticated user information
- log users out safely
- test the complete authentication flow

Congratulations.

Your Expense Tracker is no longer just a website.

It is now a real web application with secure user accounts.

In the next lesson, you will perform a complete audit of the authentication system before building the transactions database.

CHAPTER 3

BUILDING THE AUTHENTICATION SYSTEM

LESSON 2

AUDITING YOUR AUTHENTICATION SYSTEM

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not create any new pages.

Instead, you will perform a complete audit of your authentication system to confirm that every page works correctly before moving on to the database.

Think of this lesson as your quality control stage.

Professional developers never continue building on top of an authentication system until they are confident it is working properly.

WHY THIS MATTERS

The authentication system controls who can access your software.

If it is not working correctly, nothing that comes later can be trusted.

The transactions you build in the next chapter must belong only to the authenticated user.

Before you begin storing financial information, you must first be certain that users are being identified correctly.

BEFORE YOU CONTINUE

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

Confirm that you have successfully created at least one user account.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, carefully test every part of your authentication system.

Complete every test before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by testing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this final authentication check on the deployed HTTPS website.

Do not complete this audit from files opened with:

file://

SUPABASE URL SETTINGS

- ✓ The Site URL contains the Netlify website address.
- ✓ Redirect URLs contains the complete deployed login.html address.

REGISTRATION AND VERIFICATION

- ✓ Registration works from the deployed register.html page.
- ✓ A verification email arrives.
- ✓ The verification link returns to the deployed website.
- ✓ The returned address begins with https:// and uses the correct Netlify website name.

LOGIN AND SESSION

- ✓ Login works from the deployed login.html page.
- ✓ Refreshing a protected page keeps a signed-in user signed in.
- ✓ Logout works.

PROTECTION

- ✓ A signed-out visitor cannot remain on dashboard.html.
- ✓ A signed-in user can reach dashboard.html.
- ✓ A signed-in user who opens login.html returns to dashboard.html.

PAGES

- ✓ The signed-in user's email appears.
- ✓ The Income, Expenses and Balance cards appear.

If every check passes, the authentication system is ready for the next chapter.

FINAL AUTHENTICATION CHECKLIST

You should now be able to answer "Yes" to every question below.

Can someone create an account?

☐ Yes

Can someone verify their email?

☐ Yes

Can someone sign in?

☐ Yes

Can someone sign out?

☐ Yes

Can an unauthenticated visitor access the dashboard?

☐ No

Can an authenticated user reach the dashboard?

☐ Yes

Does the dashboard show the signed-in user's email?

☐ Yes

Does every redirect lead to the correct page?

☐ Yes

CHECKPOINT

Before moving to Chapter 4, confirm that:

- ☒ Registration works.
- ☒ Login works.
- ☒ Logout works.
- ☒ Email verification works.
- ☒ Protected dashboard works.
- ☒ Dashboard redirects correctly.
- ☒ Login redirects correctly.
- ☒ No authentication pages produce errors.

COMMON BEGINNER MISTAKES

Many beginners test only the happy path.

For example, they confirm that login works but never check what happens when someone enters the wrong password.

Good testing means checking both successful actions and unsuccessful ones.

Another common mistake is testing only while signed in.

Always test your application while signed out as well.

A secure application must behave correctly in both situations.

BEHIND THE SCENES

At this stage, Supabase is managing your users and their login sessions.

Every time someone signs in, Supabase remembers who they are until they sign out or their session expires.

Later, when you save expense records, every transaction will be connected to the currently authenticated user.

This is why authentication comes before the database.

The database needs to know exactly who owns each record.

THINK LIKE A SOFTWARE DESIGNER

A good authentication system should almost disappear into the background.

Users should not have to think about it.

They should be able to register, verify their email, sign in, use the application, and sign out without confusion.

When security works well, users rarely notice it.

When security is poorly designed, users become frustrated very quickly.

Professional developers design authentication to be both secure and easy to use.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit an authentication system
- test successful and unsuccessful user actions
- verify page protection
- verify automatic redirects
- confirm that authentication is ready for database work

CHAPTER SUMMARY

Congratulations.

Your Expense Tracker now has a complete authentication system.

Visitors can create accounts, verify their email addresses, sign in securely, access a protected dashboard, and sign out safely.

Every page in the authentication flow now knows exactly where to send the user.

There are no placeholder pages and no broken redirects.

More importantly, your application now knows who the current user is.

That single capability makes everything else possible.

The next stage of your project is storing information that belongs only to that authenticated user.

CHAPTER MILESTONE

By the end of Chapter 3, you have successfully built:

- ✓ User registration
- ✓ Email verification
- ✓ User login
- ✓ Protected dashboard
- ✓ User logout
- ✓ Authentication validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Automatic redirects
- ✓ Shared authentication styling
- ✓ Secure authentication flow

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

|

register.js

|

login.js

|

dashboard.js

TRANSITION TO CHAPTER 4

Your Expense Tracker can now identify every signed-in user.

The next step is to give each user a private place to store their financial records.

In Chapter 4, you will build the transactions database.

You will create the transactions table, add the required columns, enable Row Level Security, and create security policies that ensure every user can view and manage only their own transactions.

By the end of the next chapter, your database will be fully prepared for adding, viewing, editing, and deleting expense records securely.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

CHAPTER INTRODUCTION

Your Expense Tracker can now recognise who is signed in.

The next step is to give each user a secure place to store their own financial records.

That is the purpose of the database.

A database is where software stores information permanently.

Whenever a user adds an income or expense, the information will be saved inside the database so it can be viewed again later.

However, storing information is only part of the job.

The more important responsibility is protecting that information.

If two people create accounts, each person should see only their own transactions.

They should never be able to view, edit, or delete another person's records.

Supabase provides a feature called Row Level Security, often shortened to RLS.

Throughout this chapter, you will use Row Level Security to make sure every user can access only their own transactions.

By the end of this chapter, your database will be ready for the Expense Tracker dashboard you will build in Chapter 5.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A transactions table
- The required table columns

- Row Level Security
- SELECT policy
- INSERT policy
- UPDATE policy
- DELETE policy
- Security testing

By the end of this chapter, every authenticated user will have their own private transaction records.

LESSON 1

CREATING THE TRANSACTIONS TABLE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the database table that stores every income and expense entered into the Expense Tracker.

Think of a table as a spreadsheet.

Each row represents one transaction.

Each column stores one piece of information about that transaction.

For example:

Description

Amount

Category

Transaction Date

Owner

Every transaction entered by every user will eventually be stored inside this table.

WHY THIS MATTERS

Without a database table, your application has nowhere to save transactions.

Even if users can successfully sign in, every transaction would disappear as soon as the page is refreshed.

Creating the correct table is the first step towards building a real software application.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ✓ Your authentication system is working.
- ✓ You can sign in successfully.
- ✓ Your Supabase project is open.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the transactions table directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

Table Editor

Click:

Create a new table

Enter the following information.

Table Name

transactions

Description

(Optional)

Expense Tracker transactions

Enable Row Level Security.

Leave Row Level Security turned ON.

Do not disable it.

Now create the following columns.

Column Name

id

Type

uuid

Default Value

gen_random_uuid()

Primary Key

Yes

Column Name

user_id

Type

uuid

Allow NULL

No

Column Name

description

Type

text

Allow NULL

No

Column Name

transaction_type

Type

text

Allow NULL

No

Column Name

amount

Type

numeric

Allow NULL

No

Column Name

category

Type

text

Allow NULL

No

Column Name

transaction_date

Type

date

Allow NULL

No

Column Name

created_at

Type

timestamp with time zone

Default Value

now()

Allow NULL

No

After checking every column carefully, click:

Save

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

After saving the table, return to the Table Editor.

Open:

transactions

Confirm that every column appears correctly.

Your table should contain exactly these columns.

- ☒ id
- ☒ user_id
- ☒ description
- ☒ transaction_type
- ☒ amount
- ☒ category
- ☒ transaction_date
- ☒ created_at

Also confirm that:

- ☒ id is the Primary Key.
- ☒ Row Level Security is enabled.

CHECKPOINT

Before moving on, confirm that:

- ☒ The transactions table exists.
- ☒ Every required column exists.
- ☒ The Primary Key is correct.
- ☒ Row Level Security is enabled.
- ☒ The table has been saved successfully.

COMMON BEGINNER MISTAKES

- One common mistake is forgetting to create the `user_id` column.
- Without `user_id`, your application cannot tell which user owns each transaction.
- Another common mistake is creating the wrong data type.
- For example, storing the amount as text instead of numeric makes calculations much more difficult.
- Always double-check each column before saving the table.
- Finally, never disable Row Level Security to make testing easier.
- Security should be built correctly from the beginning.

BEHIND THE SCENES

Every transaction entered into your Expense Tracker will become one row inside this table.

For example:

Description

Fuel

Transaction Type

Expense

Amount

₦15,000

Category

Transportation

Transaction Date

2026-07-24

User ID

Current signed-in user

When another user signs in, their transactions will also be stored in the same table.

The difference is that each transaction belongs to a different `user_id`.

Later, Row Level Security will use this value to decide which records each user is allowed to access.

THINK LIKE A SOFTWARE DESIGNER

Good database design begins long before users start entering information.

Every column should have a clear purpose.

Ask yourself:

"What information will the software need later?"

The answer to that question determines the structure of your table.

Changing a database after thousands of users have started using it is much harder than designing it correctly from the beginning.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a database table
- choose appropriate column types
- create a Primary Key
- prepare a table for authenticated users
- understand why `user_id` is essential
- keep Row Level Security enabled

In the next lesson, you will learn why Row Level Security is one of the most important security features in your entire application before creating the policies that protect every user's data.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 2

UNDERSTANDING ROW LEVEL SECURITY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you are not creating anything new.

Instead, you will learn how Row Level Security works and why it is one of the most important security features in your Expense Tracker.

Understanding Row Level Security now will make the next four lessons much easier because you will know exactly why each security policy exists.

WHY THIS MATTERS

Imagine your Expense Tracker has one thousand users.

Every user stores their own income and expense records inside the same transactions table.

Without proper security, one user could accidentally or deliberately view another person's financial records.

That would be a serious security problem.

Row Level Security prevents this from happening.

It checks every request made to the database and decides whether the current user is allowed to perform that action.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ The transactions table exists.
- ☒ Row Level Security is enabled.

- ✓ You can still sign in successfully.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, read this lesson carefully before creating your security policies.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

There is no practical test for this lesson.

Instead, answer these questions.

Question 1

Should every signed-in user be able to see every transaction in the database?

Answer:

No.

Question 2

Should every user see only their own transactions?

Answer:

Yes.

Question 3

Can Row Level Security decide who is allowed to access each row?

Answer:

Yes.

If you answered all three correctly, you are ready for the next lesson.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ One table stores everyone's transactions.
- ☒ Each transaction belongs to one user.
- ☒ Row Level Security protects every row individually.

COMMON BEGINNER MISTAKES

Some beginners think that creating a login page automatically protects the database.

It does not.

Authentication answers one question:

"Who is this user?"

Row Level Security answers a different question:

"What is this user allowed to do?"

Both are necessary.

Another common misunderstanding is believing that separate users need separate database tables.

They do not.

A well-designed application usually stores similar information inside one table.

Security determines which rows each user can access.

BEHIND THE SCENES

Suppose two people use your Expense Tracker.

User A signs in.

User B signs in.

Both users save transactions.

The transactions table may look like this.

User ID

A123

Description

Salary

Amount

₦250,000

User ID

A123

Description

Fuel

Amount

₦18,000

User ID

B456

Description

Rent

Amount

₦450,000

User ID

B456

Description

Groceries

Amount

₦25,000

All four transactions exist inside the same table.

When User A signs in, Row Level Security hides every row that belongs to User B.

When User B signs in, Row Level Security hides every row that belongs to User A.

Neither user knows the other person's records even exist.

THINK LIKE A SOFTWARE DESIGNER

Security should never depend on users behaving correctly.

Good software assumes that someone may accidentally or deliberately try to access information they should not see.

Instead of trusting users, professional developers build security into the application itself.

That way, even if someone tries to access another person's data, the database refuses the request.

WHAT YOU LEARNED

In this lesson you learned:

- what Row Level Security is
- why authentication alone is not enough
- how multiple users share one database table
- why every transaction must belong to a specific user
- how Row Level Security protects individual records

In the next lesson, you will create your first Row Level Security policy so users can safely view only their own transactions.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 3

CREATING THE SELECT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create your first Row Level Security policy.

This policy controls which transactions users are allowed to view.

When completed, every signed-in user will be able to see only their own transactions.

WHY THIS MATTERS

Without a SELECT policy, users cannot safely retrieve information from the database.

The SELECT policy acts like a security guard.

Every time someone requests transactions, the policy checks who they are before returning any data.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The transactions table exists.
- ☒ Row Level Security is enabled.
- ☒ You understand the purpose of Row Level Security.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the SELECT policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

Authentication

Then click:

Policies

Locate the:

transactions

table.

Create a new policy.

Choose:

SELECT

Policy Name

Users can view only their own transactions

Use the following policy expression exactly:

`user_id = auth.uid()`

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your transactions table now contains a SELECT policy.

Verify that:

☒ The policy applies to SELECT.

☒ The policy name is correct.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The SELECT policy exists.

☒ The policy has been saved successfully.

☒ The policy compares `user_id` with `auth.uid()`.

COMMON BEGINNER MISTAKES

Some beginners accidentally compare the wrong columns.

The comparison should always be between:

`user_id`

and

`auth.uid()`

Do not compare description, email address, or any other field.

Another common mistake is creating the policy but forgetting to save it.

Always confirm that the policy appears in the list before continuing.

BEHIND THE SCENES

Every authenticated user has a unique identifier.

Supabase makes that identifier available through:

`auth.uid()`

Every transaction also stores the owner's identifier inside:

`user_id`

When someone requests transactions, Supabase compares these two values.

If they match, the row is returned.

If they do not match, the row remains hidden.

This comparison happens automatically every time a `SELECT` query is performed.

THINK LIKE A SOFTWARE DESIGNER

Good security should happen automatically.

Your application should not need to remember to filter every database query.

Instead, the database itself should enforce the rules consistently.

That is exactly what Row Level Security does.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a `SELECT` policy
- compare `user_id` with `auth.uid()`
- allow users to view only their own transactions
- let the database enforce security automatically

In the next lesson, you will create the `INSERT` policy so authenticated users can securely save their own transactions.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 4

CREATING THE INSERT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the policy that allows authenticated users to save their own transactions.

This policy controls every new row added to the transactions table.

Without it, your Expense Tracker will not be able to store income or expense records.

WHY THIS MATTERS

Allowing users to insert data without restrictions would be a serious security risk.

Your application should allow users to save only their own transactions.

They should never be able to create a transaction that belongs to someone else.

The INSERT policy ensures that every new transaction is linked to the currently signed-in user.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The transactions table exists.
- ☒ Row Level Security is enabled.
- ☒ The SELECT policy has been created.
- ☒ You can still sign in successfully.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the INSERT policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

- Authentication

Then click:

- Policies

Locate the:

- transactions

table.

Create a new policy.

Choose:

INSERT

Policy Name

Users can insert only their own transactions

Use the following policy expression exactly:

```
user_id = auth.uid()
```

Review the policy carefully.

Click:

- Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your transactions table now contains:

☒ One SELECT policy

☒ One INSERT policy

Open the INSERT policy.

Verify that:

☒ The policy applies to INSERT.

☒ The policy name is correct.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The INSERT policy exists.

☒ The policy has been saved successfully.

☒ The policy compares `user_id` with `auth.uid()`.

COMMON BEGINNER MISTAKES

Some beginners think that because a user is signed in, they can automatically insert records into the database.

That is not how Row Level Security works.

Every action must be allowed explicitly.

Without an INSERT policy, the database will reject attempts to save new transactions.

Another common mistake is using the wrong comparison.

Always compare:

`user_id`

with:

`auth.uid()`

BEHIND THE SCENES

When your application saves a new transaction, it will include the authenticated user's unique ID.

Before the database accepts the new row, Supabase checks the INSERT policy.

If:

`user_id = auth.uid()`

the transaction is accepted.

If the values do not match, the transaction is rejected.

This happens automatically every time a new transaction is created.

THINK LIKE A SOFTWARE DESIGNER

Good security should not rely on the honesty of the user or the correctness of the web page.

Instead, the database should verify every important action for itself.

Even if someone tries to bypass your website and communicate directly with the database, the INSERT policy will still protect your data.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an INSERT policy
- allow users to save only their own transactions
- understand how new records are protected
- let the database verify ownership automatically

In the next lesson, you will create the UPDATE policy so users can edit only their own transactions.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 5

CREATING THE UPDATE POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the policy that controls how transactions are updated.

This policy ensures that users can edit only the transactions they own.

WHY THIS MATTERS

Editing financial records is a sensitive operation.

Without proper security, one user could change another person's transactions.

The UPDATE policy prevents this by checking who owns the transaction before allowing any changes.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ The INSERT policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the UPDATE policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

Authentication

Then click:

Policies

Locate the:

transactions

table.

Create a new policy.

Choose:

UPDATE

Policy Name

Users can update only their own transactions

Use the following policy expression exactly:

```
user_id = auth.uid()
```

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your transactions table now contains:

☒ SELECT

☒ INSERT

☒ UPDATE

Open the UPDATE policy.

Verify that:

☒ The policy applies to UPDATE.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The UPDATE policy exists.

☒ The UPDATE policy has been saved.

☒ The correct policy expression is being used.

COMMON BEGINNER MISTAKES

Some beginners believe that checking the transaction ID alone is enough.

It is not.

The transaction must also belong to the currently authenticated user.

Later, when your JavaScript updates a transaction, it should identify both:

- the transaction ID

and

- the authenticated user's ID

This provides two layers of protection.

The Row Level Security policy protects the database, while your JavaScript makes sure it is requesting the correct record.

BEHIND THE SCENES

Whenever someone edits a transaction, Supabase checks the UPDATE policy before saving the changes.

If the transaction belongs to the authenticated user, the update succeeds.

If it belongs to another user, the request is rejected automatically.

Your application does not need to perform this security check manually because the database already enforces it.

THINK LIKE A SOFTWARE DESIGNER

Never assume that because someone can view a record, they should also be allowed to change it.

Reading and updating are two separate permissions.

Professional applications treat them independently.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an UPDATE policy
- protect existing transactions
- understand why editing requires separate permission
- combine application logic with database security

In the next lesson, you will create the DELETE policy so users can remove only their own transactions while protecting every other record in the database.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 6

CREATING THE DELETE POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the final Row Level Security policy for your transactions table.

This policy controls which transactions users are allowed to delete.

Once this lesson is complete, every important database action will be protected by its own security policy.

WHY THIS MATTERS

Deleting information is permanent.

If someone could delete another user's financial records, it would be a serious security problem.

The DELETE policy ensures that users can remove only the transactions they own.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ The INSERT policy exists.
- ☒ The UPDATE policy exists.
- ☒ Row Level Security is still enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the DELETE policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

- Authentication

Then click:

- Policies

Locate the:

- transactions

table.

Create a new policy.

Choose:

DELETE

Policy Name

Users can delete only their own transactions

Use the following policy expression exactly:

```
user_id = auth.uid()
```

Review the policy carefully.

Click:

- Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your transactions table now contains:

☒ SELECT

☒ INSERT

☒ UPDATE

☒ DELETE

Open the DELETE policy.

Verify that:

☒ The policy applies to DELETE.

☒ The policy name is correct.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The DELETE policy exists.

☒ The DELETE policy has been saved successfully.

☒ The policy compares `user_id` with `auth.uid()`.

COMMON BEGINNER MISTAKES

Some beginners believe that because a user created a transaction, they can automatically delete it.

That is not how Row Level Security works.

Every database action must be permitted explicitly.

Without a DELETE policy, the database will reject delete requests.

Another common mistake is relying only on JavaScript to decide which transaction should be deleted.

Your JavaScript should identify the correct transaction, but the database must still verify that the transaction belongs to the authenticated user.

BEHIND THE SCENES

When your application requests that a transaction be deleted, Supabase checks the DELETE policy before removing the row.

If:

`user_id = auth.uid()`

the delete request is allowed.

If the transaction belongs to another user, the database rejects the request immediately.

Even if someone tries to bypass your website and send requests directly to Supabase, the DELETE policy continues protecting the data.

THINK LIKE A SOFTWARE DESIGNER

Deleting information should always be treated as a sensitive operation.

Professional software never assumes that a request is legitimate simply because it came from the application.

The database should always perform its own security checks before removing data.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a DELETE policy
- protect users from unauthorised deletion
- understand why deleting records requires its own permission
- complete the four essential Row Level Security policies

In the next lesson, you will test the complete security model to confirm that every policy is working together correctly.

CHAPTER 4

BUILDING THE TRANSACTIONS DATABASE

LESSON 7

TESTING YOUR DATABASE SECURITY

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will verify that your database security has been configured correctly.

You are not creating new tables or policies.

Instead, you will review everything you have built so far before moving on to the Expense Tracker dashboard.

WHY THIS MATTERS

Database security is not something you should assume is correct.

It should always be checked carefully.

Finding a security problem now is much easier than discovering it after users have started storing important information.

BEFORE YOU CONTINUE

Confirm that your transactions table contains these policies:

☒ SELECT

☒ INSERT

☒ UPDATE

☒ DELETE

Also confirm that Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete the following review inside Supabase.

Open:

Table Editor

Open:

transactions

Review every column.

Then open:

Authentication

Policies

Review every policy.

Confirm that:

SELECT

uses:

`user_id = auth.uid()`

INSERT

uses:

`user_id = auth.uid()`

UPDATE

uses:

`user_id = auth.uid()`

DELETE

uses:

`user_id = auth.uid()`

Finally, confirm that Row Level Security is still enabled for the table.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Transactions Table

- ☒ id
- ☒ user_id
- ☒ description
- ☒ transaction_type
- ☒ amount
- ☒ category
- ☒ transaction_date
- ☒ created_at

Security

- ☒ Row Level Security enabled

Policies

- ☒ SELECT
- ☒ INSERT
- ☒ UPDATE
- ☒ DELETE

Every policy should compare:

```
user_id
```

```
with
```

```
auth.uid()
```

CHECKPOINT

Before moving on, confirm that:

- ☒ The transactions table is complete.
- ☒ Every required column exists.
- ☒ Every required policy exists.
- ☒ Row Level Security remains enabled.
- ☒ No policy has been deleted accidentally.

COMMON BEGINNER MISTAKES

Some students accidentally disable Row Level Security while experimenting with Supabase.

If this happens, your security policies stop protecting the table.

Always confirm that Row Level Security remains enabled.

Another common mistake is creating only some of the policies.

Your Expense Tracker requires all four policies because users need to:

- View transactions
- Add transactions
- Edit transactions
- Delete transactions

BEHIND THE SCENES

At this stage, your database is ready to support the Expense Tracker dashboard.

The dashboard does not need to worry about deciding whether users can access another person's transactions.

The database already enforces those rules automatically.

This allows your application code to remain simpler while keeping your users' information protected.

THINK LIKE A SOFTWARE DESIGNER

Security should be built into the foundation of your software, not added as an afterthought.

By creating your table first and then protecting every database action with Row Level Security, you have followed the same approach used in many professional web applications.

WHAT YOU LEARNED

In this lesson you learned how to:

- review a secure database
- verify every Row Level Security policy
- confirm that your database is ready for application development

CHAPTER SUMMARY

Congratulations.

Your Expense Tracker database is now complete.

You created the transactions table with all the required columns.

You enabled Row Level Security and created four separate policies that protect every important database action.

Every authenticated user will now be able to:

- ☒ View only their own transactions.
- ☒ Add only their own transactions.
- ☒ Update only their own transactions.
- ☒ Delete only their own transactions.

The database will reject requests that do not meet these security rules.

CHAPTER MILESTONE

By the end of Chapter 4, you have successfully built:

- ✓ Transactions table
- ✓ Primary Key
- ✓ user_id column
- ✓ Description column
- ✓ Transaction Type column
- ✓ Amount column
- ✓ Category column
- ✓ Transaction Date column
- ✓ Created At column
- ✓ Row Level Security
- ✓ SELECT policy
- ✓ INSERT policy
- ✓ UPDATE policy
- ✓ DELETE policy
- ✓ Secure database foundation

TRANSITION TO CHAPTER 5

Your Expense Tracker now has everything it needs behind the scenes.

Users can register, sign in, and your database is ready to store secure transaction records.

The next step is to bring everything together.

In Chapter 5, you will transform your protected dashboard into a fully functional Expense Tracker.

Users will be able to:

- Add new transactions

- View their transactions
- See their total income
- See their total expenses
- View their current balance

For the first time, your application will begin storing and displaying real financial information.

CHAPTER 5

BUILDING THE EXPENSE TRACKER DASHBOARD

CHAPTER INTRODUCTION

Your Expense Tracker has reached an important milestone.

Users can create accounts.

They can verify their email addresses.

They can sign in securely.

Your database is ready.

The only thing missing is the feature that gives the software its real purpose.

In this chapter, you will transform the dashboard from a simple placeholder into a fully working Expense Tracker.

By the end of this chapter, users will be able to add income and expense transactions, view their own records, and see a live summary of their finances.

Unlike the authentication system, this chapter will gradually improve the dashboard over several lessons.

Each lesson will add one important capability while preserving everything already built.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Transaction entry form
- Add Transaction feature
- Input validation
- Friendly success messages

- Friendly error messages
- Loading indicators
- Disabled button states
- Live transaction list
- Total income
- Total expenses
- Current balance

By the end of this chapter, your Expense Tracker will begin storing and displaying real financial information.

LESSON 1

BUILDING THE TRANSACTION ENTRY FORM

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will update your dashboard so users can enter new financial transactions.

The dashboard already exists from Chapter 3.

You are now adding the form that allows users to record income and expenses.

This lesson creates the user interface only.

The next lesson will connect the form to Supabase so transactions are saved permanently.

WHY THIS MATTERS

A dashboard without a way to enter information is not very useful.

Before users can view reports or calculate balances, they first need a way to record their financial activity.

A well-designed transaction form makes data entry quick, clear, and accurate.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ You can register successfully.
- ☒ You can log in successfully.
- ☒ dashboard.html opens correctly.
- ☒ The transactions table exists.
- ☒ Row Level Security is enabled.

Your Expense Tracker folder should currently contain:

```
index.html
styles.css
script.js
supabase-config.js
test-connection.html
register.html
login.html
dashboard.html
auth.css
register.js
login.js
dashboard.js
```

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- User registration
- User login
- Protected dashboard
- Logout
- Supabase connection
- Transactions database

Do not remove or break any existing functionality.

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

auth.css

dashboard.js

DASHBOARD.HTML

Keep everything that already exists.

Preserve:

- Logo linking to index.html
- Welcome heading
- User email display
- Summary cards
- Logout button

Below the summary cards, add a professional transaction form.

The form should include:

Description

- Text input

Transaction Type

- Dropdown

Options:

Income

Expense

Amount

- Number input

Category

- Dropdown

Include at least:

Food

Transport

Shopping

Salary

Utilities

Health

Entertainment

Education

Other

Transaction Date

- Date picker

Add a button labelled:

Add Transaction

Below the button include an empty message area for:

- Success messages
- Error messages

Below the form create an empty section titled:

Recent Transactions

Leave a placeholder where transactions will appear in later lessons.

AUTH.CSS

Return the complete updated stylesheet.

Keep every existing style.

Add professional styling for:

- Transaction form
- Labels
- Inputs
- Dropdowns
- Date picker
- Add Transaction button
- Form layout
- Responsive layout
- Success message
- Error message

- Placeholder transaction area

DASHBOARD.JS

Return the complete updated file.

Keep every existing feature working.

Do NOT save transactions yet.

Continue protecting the dashboard.

Continue displaying the logged-in user's email.

Continue supporting Logout.

For the transaction form:

Validate:

- Description
- Transaction Type
- Amount
- Category
- Transaction Date

Display friendly validation messages.

Disable the Add Transaction button while validation is running.

Re-enable it afterwards.

Do not insert anything into Supabase yet.

Instead:

If validation succeeds:

Display:

Transaction is ready to be saved.

If validation fails:

Display clear messages explaining what needs to be corrected.

Prevent the page from refreshing when the form is submitted.

IMPORTANT

Everything already built must continue working.

Do not remove dashboard protection.

Do not remove Logout.

Do not remove user email display.

Do not rename IDs.

Do not rename existing classes unless absolutely necessary.

Do not create new JavaScript files.

Do not create another Supabase client.

Continue using:

supabaseClient

from:

supabase-config.js

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

- ✓ dashboard.html
- ✓ auth.css
- ✓ dashboard.js

The dashboard should now contain:

- ✓ Transaction form
- ✓ Validation
- ✓ Success messages
- ✓ Error messages
- ✓ Placeholder transaction list
- ✓ Existing authentication features

Nothing that already worked should stop working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open each updated file in Notepad.

Replace its entire contents with the complete updated version returned by ChatGPT.

Click File.

Click Save.

Repeat this for:

dashboard.html

auth.css

dashboard.js

Do not create duplicate files.

Do not rename existing files.

TEST YOUR WORK

Sign in to your Expense Tracker.

After reaching the dashboard, confirm that you can see:

☒ Logo

☒ Welcome heading

☒ Your email address

☒ Income summary card

☒ Expense summary card

☒ Balance summary card

☒ Transaction form

☒ Recent Transactions heading

Complete the form.

Leave one field empty.

Click:

Add Transaction

A friendly validation message should appear.

Now complete every field correctly.

Click:

Add Transaction

The page should display:

Transaction is ready to be saved.

The page should not refresh.

The Logout button should still work correctly.

CHECKPOINT

Before moving on, confirm that:

☒ The transaction form appears.

☒ Every field validates correctly.

☒ Friendly messages appear.

☒ The Add Transaction button behaves correctly.

☒ The dashboard still requires users to be signed in.

☒ Logout still works.

COMMON BEGINNER MISTAKES

Some beginners immediately try to save information to Supabase.

Do not do that yet.

This lesson focuses only on building a reliable form and validating the information entered by the user.

Saving data becomes much easier when you know the form is already working correctly.

Another common mistake is accidentally removing the authentication code while updating dashboard.js.

Remember that every update should preserve everything already built.

BEHIND THE SCENES

Professional developers often separate validation from data storage.

The application first checks that the information is complete and sensible.

Only after validation succeeds does it attempt to save the data.

This approach reduces unnecessary database requests and provides faster feedback to users.

THINK LIKE A SOFTWARE DESIGNER

Good software helps users avoid mistakes before information is saved.

A well-designed form should guide users towards entering correct information instead of simply reporting errors afterwards.

When building forms, always ask yourself:

"What can I do to help the user succeed the first time?"

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building the transaction entry form. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- expand an existing dashboard safely
- add a professional transaction form
- validate user input
- display friendly success messages
- display friendly error messages
- preserve existing authentication features

In the next lesson, you will connect this form to Supabase so that every valid transaction is saved securely to the database belonging to the authenticated user.

CHAPTER 5

BUILDING THE EXPENSE TRACKER DASHBOARD

LESSON 2

SAVING TRANSACTIONS TO SUPABASE

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In the previous lesson, you built a professional transaction form and validated the information entered by the user.

In this lesson, you will connect that form to your Supabase database.

When a user completes the form successfully and clicks the Add Transaction button, the transaction will be saved permanently in the transactions table.

Every transaction will automatically belong to the currently signed-in user.

WHY THIS MATTERS

Until now, your Expense Tracker could collect information but it could not store it.

After this lesson, users will no longer lose their information when they refresh the page.

Every valid transaction will be saved securely inside the database and will be available whenever the user signs in again.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ The transaction form is working.
- ☒ Validation messages appear correctly.
- ☒ You can sign in successfully.
- ☒ The transactions table exists.

✓ Row Level Security is enabled.

✓ The SELECT, INSERT, UPDATE and DELETE policies all exist.

Do not continue until every item above is working correctly.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- User registration
- User login
- Protected dashboard
- Logout
- Shared Supabase connection
- Transactions table
- Row Level Security
- Transaction form
- Form validation

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Do not remove:

- Logo
- Welcome heading
- User email display
- Summary cards
- Transaction form
- Message area
- Recent Transactions section

Keep all existing IDs and class names unless absolutely necessary.

AUTH.CSS

Keep every existing style.

If necessary, improve the styling for:

- Loading messages
- Success messages
- Error messages

Do not remove any existing styling.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js

Do not create another Supabase client.

When the Add Transaction form is submitted:

1.

Prevent the page from refreshing.

2.

Validate every field.

3.

If validation fails:

Display a friendly error message.

Do not continue.

4.

Retrieve the currently authenticated user using Supabase.

If no authenticated user exists:

Redirect immediately to:

login.html

5.

Insert a new record into the transactions table.

Insert these values:

user_id

The authenticated user's ID.

description

The Description field.

transaction_type

The selected transaction type.

amount

The Amount field.

Convert the amount into a number before saving it.

category

The selected category.

transaction_date

The selected transaction date.

6.

Display a friendly success message after the transaction has been saved.

Example:

Transaction saved successfully.

7.

Clear the form after a successful save.

8.

Reset the Add Transaction button.

9.

Handle unexpected errors gracefully.

Display friendly error messages.

10.

Disable the Add Transaction button while the save is taking place.

Re-enable it afterwards.

IMPORTANT

Do not load transactions yet.

Do not calculate totals yet.

Do not display transactions yet.

Those features will be built in the next lessons.

DATABASE SECURITY

Every transaction inserted into the database must include:

`user_id`

taken from:

the authenticated user.

Do not accept a `user_id` entered by the visitor.

Always use the authenticated user's ID returned by Supabase.

IMPORTANT

Everything already built must continue working.

Keep:

Dashboard protection

Logout

User email display

Validation

Loading states

Friendly messages

Do not rename files.

Do not rename IDs.

Do not remove existing functionality.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

☒ dashboard.html

☒ dashboard.js

☒ auth.css

Everything previously built should continue working.

The Add Transaction button should now save validated transactions into the transactions table.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open each updated file in Notepad.

Replace the existing contents with the complete updated version returned by ChatGPT.

Click File.

Click Save.

Repeat this process for:

dashboard.html

dashboard.js

auth.css

Do not create duplicate files.

TEST YOUR WORK

Sign in to your Expense Tracker.

Open the dashboard.

Complete every field in the transaction form.

Click:

Add Transaction

The button should become disabled while the transaction is being saved.

A loading message should appear.

After a few moments, you should see:

Transaction saved successfully.

The form should clear automatically.

The page should not refresh.

Now open your Supabase project.

Click:

Table Editor

Open:

transactions

You should see a new row.

Confirm that:

☒ description has been saved.

☒ transaction_type has been saved.

☒ amount has been saved.

☒ category has been saved.

☒ transaction_date has been saved.

✓ `user_id` contains your authenticated user's ID.

✓ `created_at` has been filled automatically.

Repeat the test by adding a second transaction.

Confirm that both transactions appear inside the table.

CHECKPOINT

Before moving on, confirm that:

✓ Transactions are saved successfully.

✓ Validation still works.

✓ Friendly messages still appear.

✓ The Add Transaction button is disabled while saving.

✓ The form clears after saving.

✓ The page does not refresh.

✓ Every new row contains the authenticated user's ID.

COMMON BEGINNER MISTAKES

One common mistake is taking the `user_id` from the form or allowing the visitor to choose it.

Never do this.

The application should always obtain the user ID from the authenticated session.

Another common mistake is saving the amount as text.

Convert the value to a number before inserting it into the database.

Some beginners also forget to clear the form after saving.

Leaving old values in the form makes it easy for users to submit duplicate transactions by mistake.

BEHIND THE SCENES

When you click the Add Transaction button, several things happen in a very short time.

The application validates the form.

It checks that the user is signed in.

It retrieves the authenticated user's unique ID.

It sends the transaction to Supabase.

The database checks the INSERT policy.

If the transaction belongs to the authenticated user, the database accepts it and stores it permanently.

If not, the request is rejected automatically.

Even though this process feels simple to the user, several security checks are taking place behind the scenes.

THINK LIKE A SOFTWARE DESIGNER

Whenever software saves information, reliability is more important than speed.

Users expect confidence that their information has been stored correctly.

That is why good applications provide loading indicators, disable buttons during processing, display clear success messages, and recover gracefully from unexpected errors.

A professional application tells the user exactly what is happening instead of leaving them to guess.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports saving transactions to supabase. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- connect a form to Supabase
- save transactions securely

- retrieve the authenticated user's ID
- insert records into the database
- display loading and success messages
- clear a form after a successful save
- preserve every feature already built

In the next lesson, you will retrieve the authenticated user's transactions from the database and display them inside the dashboard.

CHAPTER 5

BUILDING THE EXPENSE TRACKER DASHBOARD

LESSON 3

DISPLAYING TRANSACTIONS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In the previous lesson, users were able to save transactions into the database.

Those transactions are currently invisible.

In this lesson, you will retrieve the authenticated user's transactions from Supabase and display them inside the dashboard.

Every user should see only their own records.

WHY THIS MATTERS

Saving information is only half of the job.

Users also need to see what they have already entered.

A useful Expense Tracker should immediately display newly added transactions so users can review their financial activity.

Because your database already has Row Level Security enabled, every user will automatically see only their own transactions.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ You can register successfully.
- ☒ You can sign in successfully.
- ☒ You can save transactions.

✓ At least two transactions exist in your database.

✓ The dashboard opens correctly.

Do not continue until everything above is working correctly.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- User registration
- User login
- Protected dashboard
- Logout
- Shared Supabase connection
- Transactions table
- Row Level Security
- Transaction form
- Form validation
- Saving transactions to Supabase

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Preserve:

- Logo
- Welcome heading
- User email display
- Summary cards
- Transaction form
- Message area

Replace the placeholder under:

Recent Transactions

with an empty container that JavaScript can populate dynamically.

The container should have an appropriate ID.

Above the transaction list, display column headings for:

Description

Type

Category

Amount

Date

Actions

Display a friendly empty-state message when no transactions exist.

AUTH.CSS

Keep every existing style.

Add professional styling for:

- Transaction table
- Transaction rows
- Table headings
- Empty state
- Loading state
- Responsive transaction list

On smaller screens, ensure the transaction list remains easy to read.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

`supabaseClient`

from:

`supabase-config.js`

Do not create another Supabase client.

After confirming that the user is authenticated:

Retrieve only that user's transactions from the:

`transactions`

`table`.

Order the transactions by:

`transaction_date`

in descending order so the newest transaction appears first.

Display each transaction inside the Recent Transactions section.

For every transaction, display:

Description

Transaction Type

Category

Amount

Transaction Date

For now, include placeholder Edit and Delete buttons.

These buttons do not need to work yet.

Display them because they will be implemented in a later lesson.

After a new transaction is saved successfully:

Refresh the transaction list automatically.

Do not require the user to refresh the page manually.

Display a friendly loading message while transactions are being retrieved.

If no transactions exist:

Display a friendly message such as:

You haven't added any transactions yet.

Add your first income or expense above to get started.

Handle unexpected database errors gracefully.

Display friendly error messages.

IMPORTANT

Do not calculate totals yet.

Do not implement Edit.

Do not implement Delete.

Those features will be added later.

DATABASE SECURITY

Do not request another user's records.

Retrieve transactions using the authenticated user.

Everything should continue relying on Row Level Security.

IMPORTANT

Everything already built must continue working.

Keep:

Dashboard protection

Logout

User email display

Transaction saving

Validation

Loading states

Friendly messages

Do not rename files.

Do not rename IDs unless absolutely necessary.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

✓ dashboard.html

✓ dashboard.js

✓ auth.css

The dashboard should now:

✓ Display transactions.

✓ Refresh automatically after saving.

✓ Display loading states.

✓ Display empty states.

✓ Continue protecting user data.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open each updated file in Notepad.

Replace the existing contents with the complete updated version returned by ChatGPT.

Click File.

Click Save.

Repeat this process for:

dashboard.html

dashboard.js

auth.css

Do not create duplicate files.

TEST YOUR WORK

Sign in to your Expense Tracker.

Open the dashboard.

If you already have transactions in your database, they should appear automatically.

Confirm that:

☒ The newest transaction appears first.

☒ Description is displayed.

☒ Transaction Type is displayed.

☒ Category is displayed.

☒ Amount is displayed.

☒ Transaction Date is displayed.

☒ Edit button appears.

☒ Delete button appears.

Now add another transaction.

After saving successfully:

☒ The transaction list refreshes automatically.

☒ The new transaction appears at the top.

Now create another user account.

Sign in using the second account.

The transaction list should be empty.

You should see the empty-state message.

The transactions created using the first account should not appear.

This confirms that Row Level Security is working correctly.

CHECKPOINT

Before moving on, confirm that:

- ✓ Transactions are displayed.
- ✓ Only the signed-in user's transactions appear.
- ✓ New transactions appear automatically after saving.
- ✓ Empty-state messages work.
- ✓ Loading messages work.
- ✓ Edit and Delete buttons are visible.
- ✓ The dashboard remains protected.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to refresh the transaction list after saving a new transaction.

If you do not reload the list, users may think their transaction was not saved.

Another common mistake is retrieving every transaction from the table instead of relying on the authenticated user's session.

Your application should always retrieve data as the authenticated user.

The Row Level Security policies will ensure that only authorised records are returned.

Finally, do not implement the Edit or Delete buttons yet.

Although the buttons are visible, their functionality belongs in a later lesson.

BEHIND THE SCENES

When the dashboard loads, it sends a request to Supabase asking for transactions from the transactions table.

Before returning any data, Supabase checks the SELECT policy.

Only the rows that belong to the authenticated user are returned.

Your JavaScript simply displays the records that Supabase provides.

This means your application does not need to manually separate one user's transactions from another user's transactions.

The database performs that security check automatically.

THINK LIKE A SOFTWARE DESIGNER

Good dashboards provide immediate feedback.

When users save information, they expect to see it appear without refreshing the browser.

Refreshing the transaction list automatically creates a smoother experience and reassures users that their information has been stored successfully.

Small improvements like this make software feel faster and more professional.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports displaying transactions. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- retrieve transactions from Supabase
- display database records on the dashboard
- refresh the transaction list automatically
- create loading and empty states
- prepare the dashboard for editing and deleting transactions
- rely on Row Level Security to protect user data

In the next lesson, you will calculate and display the user's total income, total expenses, and current balance using the transactions already loaded into the dashboard.

CHAPTER 5

BUILDING THE EXPENSE TRACKER DASHBOARD

LESSON 4

CALCULATING TOTAL INCOME, TOTAL EXPENSES AND CURRENT BALANCE

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

Your dashboard already displays the user's transactions.

In this lesson, you will make the three summary cards at the top of the dashboard display real information.

Whenever the dashboard loads, it will calculate:

- Total Income
- Total Expenses
- Current Balance

Whenever a new transaction is added, these figures should update automatically without requiring the user to refresh the page.

WHY THIS MATTERS

Most users do not want to read through every transaction before understanding their financial position.

A dashboard should summarise important information immediately.

These summary cards allow users to see how much they have earned, how much they have spent, and how much money remains.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Transactions are displayed correctly.

- ✓ At least three transactions exist.
- ✓ The transaction list refreshes automatically after saving.
- ✓ The summary cards still display placeholder values.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- User registration
- User login
- Protected dashboard
- Logout
- Shared Supabase connection
- Transactions database
- Row Level Security
- Transaction form
- Saving transactions
- Displaying transactions

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Do not remove:

- Logo
- Welcome heading
- User email
- Transaction form
- Recent Transactions
- Edit button placeholders
- Delete button placeholders

Ensure the summary cards have clear IDs so JavaScript can update:

Total Income

Total Expenses

Current Balance

AUTH.CSS

Keep every existing style.

Improve the appearance of the summary cards if necessary.

Make sure they remain responsive.

Do not remove existing styles.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js

Do not create another Supabase client.

After retrieving the authenticated user's transactions:

Calculate:

```
Total Income
Add together every transaction where:
transaction_type
equals:
Income
Calculate:
Total Expenses
Add together every transaction where:
transaction_type
equals:
Expense
Calculate:
Current Balance
Total Income
minus
Total Expenses
Display all three values inside their summary cards.
Format every amount as Nigerian Naira.
Display two decimal places.
After a new transaction is saved:
Refresh the transaction list.
Recalculate all three summary cards automatically.
If no transactions exist:
Display:
₦0.00
for all three cards.
Handle unexpected errors gracefully.
Continue displaying friendly messages.
```

IMPORTANT

```
Everything already built must continue working.
Keep:
```

Dashboard protection

Logout

User email

Transaction saving

Transaction display

Loading states

Validation

Friendly messages

Do not rename IDs unless absolutely necessary.

Do not remove any existing functionality.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

- ☒ dashboard.html
- ☒ dashboard.js
- ☒ auth.css

The dashboard should now calculate:

- ☒ Total Income
- ☒ Total Expenses
- ☒ Current Balance

The summary cards should update automatically whenever transactions change.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open each updated file in Notepad.

Replace the existing contents with the complete updated version returned by ChatGPT.

Click File.

Click Save.

Repeat this process for:

dashboard.html

dashboard.js

auth.css

Do not create duplicate files.

TEST YOUR WORK

Sign in to your Expense Tracker.

Open the dashboard.

Confirm that the summary cards display values instead of placeholders.

Now add an Income transaction.

For example:

Salary

Income

₦500,000

Save the transaction.

The Total Income card should increase automatically.

The Current Balance should also increase.

Now add an Expense transaction.

For example:

Fuel

Expense

₦15,000

Save the transaction.

The Total Expenses card should increase.

The Current Balance should decrease.

Finally, compare the displayed totals with your transaction list.

Confirm that:

- ☒ Total Income equals the sum of all Income transactions.
- ☒ Total Expenses equals the sum of all Expense transactions.
- ☒ Current Balance equals:

Total Income

minus

Total Expenses.

CHECKPOINT

Before moving on, confirm that:

- ☒ Total Income calculates correctly.
- ☒ Total Expenses calculates correctly.
- ☒ Current Balance calculates correctly.
- ☒ The summary cards refresh automatically.
- ☒ Empty dashboards display ₦0.00.
- ☒ Existing dashboard features continue working.

COMMON BEGINNER MISTAKES

One common mistake is treating every transaction as income.

Always check the value of:

transaction_type

before adding the amount to a total.

Another common mistake is forgetting to recalculate the summary cards after saving a new transaction.

If you update the transaction list but not the summary cards, the dashboard displays outdated information.

Some beginners also calculate the Current Balance by adding every transaction together.

Income should increase the balance.

Expenses should reduce it.

BEHIND THE SCENES

Every time the dashboard retrieves transactions, JavaScript loops through the records one by one.

For each transaction, it checks whether the transaction type is Income or Expense.

Income values are added to the Total Income.

Expense values are added to the Total Expenses.

After every transaction has been processed, the Current Balance is calculated by subtracting the Total Expenses from the Total Income.

Because these calculations happen inside the browser, the results update almost instantly whenever the transaction list changes.

THINK LIKE A SOFTWARE DESIGNER

Dashboards should help users understand information quickly.

Most people do not want to calculate totals in their heads.

Good software performs repetitive calculations automatically and presents the results clearly.

Whenever you build a dashboard, ask yourself:

"What information would help the user make a decision immediately?"

Those are usually the numbers that deserve a prominent place at the top of the page.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports calculating total income, total expenses and current balance. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- calculate totals from database records
- separate income from expenses
- calculate a running balance
- update dashboard summary cards automatically
- keep calculated values in sync with the transaction list

In the next lesson, you will add search and filtering so users can quickly find specific transactions without scrolling through the entire list.

CHAPTER MILESTONE

By the end of Chapter 5, you should have successfully built:

- ✓ A professional transaction form
- ✓ Description field
- ✓ Transaction Type field
- ✓ Amount field
- ✓ Category field
- ✓ Transaction Date field
- ✓ Strong form validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Disabled button states
- ✓ Secure transaction saving
- ✓ Authenticated user ownership
- ✓ Recent Transactions section
- ✓ Transaction loading
- ✓ Empty state
- ✓ Total Income calculation
- ✓ Total Expenses calculation
- ✓ Current Balance calculation
- ✓ Automatic dashboard refresh after saving

Your Expense Tracker folder should still contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

No new files were required during this chapter.

BEFORE MOVING TO CHAPTER 6

Sign in to your Expense Tracker and complete this review.

Authentication

- ✓ The dashboard opens only when you are signed in.
- ✓ Your email address appears.
- ✓ Logout redirects to login.html.

Adding Transactions

- ✓ The form validates every field.
- ✓ Invalid entries are rejected.
- ✓ The Add Transaction button becomes disabled while saving.
- ✓ A loading message appears.
- ✓ A success message appears after saving.
- ✓ The form clears after a successful save.

Database

- ✓ Every saved record contains the authenticated user's ID.

- ✓ The amount is stored as a number.
- ✓ The transaction appears in the transactions table.

Dashboard

- ✓ Saved transactions appear in Recent Transactions.
- ✓ Newly added transactions appear without manually refreshing the browser.
- ✓ Total Income is correct.
- ✓ Total Expenses is correct.
- ✓ Current Balance is correct.
- ✓ A new account cannot see another user's transactions.

Do not continue until every item above works correctly.

TRANSITION TO CHAPTER 6

Your dashboard can now save and display transaction records.

In the next chapter, you will improve how those transactions are presented.

You will make the list easier to read, strengthen the loading and empty states, confirm the correct transaction order, and ensure the layout works properly on narrow screens.

The goal is not to add editing or deletion yet.

Those features belong in Chapter 7.

CHAPTER 6

VIEWING TRANSACTIONS

CHAPTER INTRODUCTION

A transaction list should do more than place database records on the page.

It should help users understand their financial activity quickly.

Descriptions should be easy to identify. Income and expenses should look different. Dates and amounts should be formatted consistently. The layout should remain useful on both wide and narrow screens.

In this chapter, you will improve the Recent Transactions section without changing how records are stored.

Everything already built must continue working.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will improve:

- Transaction loading
- Transaction ordering
- Amount formatting
- Date formatting
- Income and expense labels
- Empty states
- Loading states
- Error states
- Responsive transaction display
- Automatic refresh after saving

By the end of the chapter, users will be able to review their financial records clearly on different screen sizes.

LESSON 1

IMPROVING THE TRANSACTION LIST

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will improve the way transactions appear inside the dashboard.

The database connection and transaction-saving feature already work.

You are not creating a new table or changing the database.

You will update the existing dashboard files so that the transaction list becomes clearer, more consistent, and easier to use.

WHY THIS MATTERS

A database may contain correct information while the page displaying it remains difficult to understand.

Users should be able to identify the description, type, category, amount, and date of each transaction without confusion.

Clear presentation becomes increasingly important as the number of saved transactions grows.

BEFORE YOU CONTINUE

Confirm that:

- ☒ You can sign in successfully.
- ☒ You can save transactions.
- ☒ Saved transactions appear on the dashboard.
- ☒ Total Income is correct.
- ☒ Total Expenses is correct.
- ☒ Current Balance is correct.

Add at least four transactions before using the Build Prompt.

Include both Income and Expense records.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project currently includes:

- Public landing page
- Responsive navigation
- Supabase connection
- User registration
- Email verification
- User login
- Protected dashboard
- Logout
- Transactions table
- Row Level Security
- Secure transaction saving
- Recent Transactions list
- Total Income
- Total Expenses
- Current Balance

Do not remove or break any existing functionality.

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Preserve:

- Logo linking to index.html
- Welcome heading
- Authenticated user's email
- Logout button
- Summary cards
- Transaction form
- Message area
- Recent Transactions section

The Recent Transactions section must include clear headings for:

Description

Type

Category

Amount

Date

Actions

Keep placeholder Edit and Delete buttons for every transaction.

The buttons do not need to work yet.

Give the transaction container a clear ID that matches the JavaScript.

Include a loading area that JavaScript can update.

Include an empty-state area that JavaScript can update.

Do not create duplicate transaction containers.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js

Do not create another Supabase client.

Continue protecting dashboard.html.

Continue displaying the authenticated user's email.

Continue supporting Logout.

Continue validating and saving transactions.

Continue calculating Total Income, Total Expenses, and Current Balance.

TRANSACTION RETRIEVAL

Retrieve transactions only after confirming that the user is authenticated.

Retrieve records from:

transactions

Filter the request using both:

user_id

and the authenticated user's ID.

Row Level Security must remain enabled and will provide additional protection.

Order records by:

transaction_date

in descending order.

When two transactions have the same transaction_date, order them by:

created_at

in descending order.

TRANSACTION DISPLAY

For every transaction, display:

- Description
- Transaction Type
- Category
- Amount
- Transaction Date
- Edit button
- Delete button

Format every amount as Nigerian Naira with two decimal places.

Display Income and Expense using clear visual labels.

Format the transaction date so it is easy to read.

Do not display raw database timestamps.

LOADING STATE

Before requesting transactions, display a clear loading message such as:

Loading transactions...

Remove the loading message when the request finishes.

EMPTY STATE

If the authenticated user has no transactions, display:

You haven't added any transactions yet.

Add your first income or expense using the form above.

Do not display an empty table underneath this message.

ERROR STATE

If transactions cannot be loaded, display a friendly message.

Do not expose technical database details to the user.

You may record the full error using:

```
console.error()
```

but the visible message should use simple language.

AUTOMATIC REFRESH

After a transaction is saved successfully:

- Reload the transactions.
- Update the Recent Transactions section.
- Recalculate Total Income.
- Recalculate Total Expenses.
- Recalculate Current Balance.

The user should not need to refresh the browser manually.

IMPORTANT

- Do not implement editing yet.
- Do not implement deletion yet.
- Do not add search.
- Do not add filters.
- Do not add sorting controls.
- These features belong in later chapters.

AUTH.CSS

- Keep every existing style.
- Add or improve professional styling for:
 - Recent Transactions section
 - Transaction headings
 - Transaction rows
 - Income labels
 - Expense labels
 - Amounts
 - Dates
 - Action buttons
 - Loading state
 - Empty state
 - Error state
 - Responsive transaction layout
- On wide screens, the transaction information should align clearly under its headings.
- On narrow screens, each transaction may change into a stacked card layout.
- Make sure every transaction remains readable without horizontal scrolling.
- Do not remove existing dashboard, authentication, form, message, or summary-card styles.

TECHNICAL REQUIREMENTS

- Everything already built must continue working.

- Return the complete updated dashboard.html.
- Return the complete updated dashboard.js.
- Return the complete updated auth.css.
- Do not return partial sections.
- Do not rename files.
- Do not create new files.
- Do not create another Supabase client.
- Keep the correct Supabase script loading order inside dashboard.html:
 1. Supabase CDN
 2. supabase-config.js
 3. dashboard.js
- Ensure every HTML ID used by dashboard.js exists in dashboard.html.
- Ensure every class used for styling exists where required.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

dashboard.html

dashboard.js

auth.css

The updated files should preserve all previous functionality while improving the transaction list.

The response should not contain incomplete code, changed snippets, or omitted sections.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Start with dashboard.html.

Open dashboard.html in Notepad.

Select all the existing code.

Delete it.

Paste the complete updated dashboard.html code returned by ChatGPT.

Click File.

Click Save.

Close Notepad.

Next, open dashboard.js in Notepad.

Replace its complete contents with the updated dashboard.js code.

Click File.

Click Save.

Close Notepad.

Finally, open auth.css in Notepad.

Replace its complete contents with the updated auth.css code.

Click File.

Click Save.

Close Notepad.

Do not use Save As when updating an existing file unless the file is missing.

Confirm that the filenames remain:

dashboard.html

dashboard.js

auth.css

Make sure none of them has become:

dashboard.html.txt

dashboard.js.txt

auth.css.txt

TEST YOUR WORK

Open login.html in your browser.

Sign in.

After reaching the dashboard, watch the Recent Transactions section.

A loading message should appear briefly while the transactions are being retrieved.

When loading finishes, your saved transactions should appear.

Confirm that:

- ☒ The newest transaction date appears first.
- ☒ Transactions with the same date are arranged with the most recently created one first.
- ☒ Descriptions are easy to read.
- ☒ Income and Expense labels look different.
- ☒ Categories appear correctly.
- ☒ Amounts display in Nigerian Naira.
- ☒ Amounts include two decimal places.
- ☒ Dates are easy to read.
- ☒ Edit buttons appear.
- ☒ Delete buttons appear.

The Edit and Delete buttons should not perform their full actions yet.

Now add a new transaction.

After saving:

- ☒ The transaction appears without refreshing the browser manually.
- ☒ The loading state does not remain visible.
- ☒ The financial summary cards update correctly.

Test the empty state using an account that has no transactions.

The dashboard should display:

You haven't added any transactions yet.

Add your first income or expense using the form above.

Make the browser window narrower.

Confirm that:

- ✓ Transaction records remain readable.
- ✓ Text does not run outside the page.
- ✓ The page does not develop unnecessary horizontal scrolling.
- ✓ Buttons remain easy to select.

CHECKPOINT

Before moving on, confirm that:

- ✓ Transactions load only after authentication is confirmed.
- ✓ Transactions are filtered using the authenticated user's ID.
- ✓ Row Level Security remains enabled.
- ✓ Records are ordered correctly.
- ✓ Amounts are formatted correctly.
- ✓ Dates are formatted clearly.
- ✓ Loading states work.
- ✓ Empty states work.
- ✓ Error messages are friendly.
- ✓ New transactions appear automatically.
- ✓ Summary cards remain correct.
- ✓ The transaction list works on narrow screens.

COMMON BEGINNER MISTAKES

A common mistake is changing only a small part of `dashboard.js` while leaving the surrounding code unchanged.

This may create duplicate functions, conflicting variable names, or multiple event listeners.

Always replace the complete contents of every affected file with the complete updated version returned by ChatGPT.

Another mistake is formatting the amount before using it in calculations.

The application should calculate with the numeric amount first. Formatting should happen only when the value is displayed.

Do not store values such as:

₦15,000.00

inside the amount column.

The database should store the number.

JavaScript should add the currency symbol and commas when displaying it.

Some beginners also remove the transaction headings when adding the empty state.

The headings should appear when transactions exist. The friendly empty state should appear when there are no records to display.

BEHIND THE SCENES

The database stores information in a form that is useful for calculations and sorting.

The browser then presents that information in a form that is easier for people to understand.

For example, the database may store an amount as:

15000

The dashboard can display it as:

₦15,000.00

The stored value remains a number, while the visible value is formatted for the user.

The same idea applies to dates.

The database stores a reliable date value. JavaScript changes how that date appears on the page without changing the saved record.

THINK LIKE A SOFTWARE DESIGNER

Users judge software partly by how clearly it presents information.

Correct data is important, but the user should not struggle to interpret it.

A good transaction list creates a clear visual order. The description identifies the transaction, the type explains whether money came in or went out, and the amount shows the financial effect.

When the screen becomes narrow, the layout should adjust rather than forcing the user to move sideways across the page.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports improving the transaction list. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- improve an existing transaction list
- retrieve records in a reliable order
- filter database requests using the authenticated user's ID
- format Naira amounts
- format transaction dates
- display loading, empty, and error states
- refresh dashboard information after saving
- create a responsive transaction layout

In the next lesson, you will audit the full transaction-viewing experience before moving to Chapter 7, where the Edit and Delete buttons will become fully functional.

CHAPTER 6

VIEWING TRANSACTIONS

LESSON 2

AUDITING THE TRANSACTION VIEW

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not create new files or add new features.

You will carefully test the way transactions are loaded, displayed, ordered, and refreshed.

This audit confirms that the Recent Transactions section is reliable before you begin editing and deleting records in the next chapter.

WHY THIS MATTERS

Editing and deleting depend on the correct transaction being identified.

If records are displayed incorrectly, mixed between users, or not refreshed properly, later features may affect the wrong transaction.

A careful audit now gives you a stable foundation for the next stage of the project.

BEFORE YOU CONTINUE

Confirm that your dashboard currently supports:

- ☒ Adding transactions
- ☒ Saving transactions to Supabase
- ☒ Displaying saved transactions
- ☒ Total Income
- ☒ Total Expenses
- ☒ Current Balance

☒ Loading state

☒ Empty state

☒ Responsive transaction layout

You should also have access to two registered and verified user accounts.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test below before moving to Chapter 7.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by testing the existing project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete the tests in order.

TEST 1

AUTHENTICATED ACCESS

Open:

login.html

Sign in successfully.

Confirm that you are redirected to:

dashboard.html

The dashboard should display the signed-in user's email address.

Log out.

Try to open:

dashboard.html

You should be redirected to:

login.html

TEST 2

LOADING STATE

Sign in again.

Watch the Recent Transactions section as the dashboard opens.

A loading message should appear while records are being retrieved.

The loading message should disappear after the request finishes.

It should not remain permanently on the page.

TEST 3

TRANSACTION ORDER

Add three transactions using different dates.

For example:

Transaction 1

Description:

Salary

Type:

Income

Date:

A recent date

Transaction 2

Description:

Transport

Type:

Expense

Date:

An older date

Transaction 3

Description:

Food

Type:

Expense

Date:

Today's date

The newest transaction date should appear first.

The oldest transaction date should appear last.

Now add two transactions using the same transaction date.

The transaction created most recently should appear above the earlier one.

TEST 4

TRANSACTION DETAILS

Review each transaction.

Confirm that every row or card displays:

- ☒ Description
- ☒ Transaction Type
- ☒ Category
- ☒ Amount
- ☒ Transaction Date
- ☒ Edit button
- ☒ Delete button

The Edit and Delete buttons may still be inactive at this stage.

That is expected.

TEST 5

AMOUNT FORMATTING

Add a transaction with this amount:

15000

After saving, it should display in a clear Naira format such as:

₦15,000.00

Add another transaction with this amount:

25000.

It should display with two decimal places.

The amount stored in Supabase should remain numeric.

It should not include the Naira symbol or commas inside the database column.

TEST 6

INCOME AND EXPENSE LABELS

Confirm that Income and Expense records are easy to distinguish.

Their labels should use clear wording and different visual treatment.

Do not depend only on colour.

The words:

Income

and

Expense

should remain visible.

TEST 7

AUTOMATIC REFRESH

Add a new transaction.

After it is saved:

☒ The form should clear.

☒ A success message should appear.

☒ The transaction list should reload.

☒ The new transaction should appear.

☒ Total Income, Total Expenses, and Current Balance should update.

You should not need to refresh the browser manually.

TEST 8

EMPTY STATE

Log out.

Sign in using a second account that has no transactions.

The dashboard should not display the first user's records.

Instead, it should display:

You haven't added any transactions yet.

Add your first income or expense using the form above.

The transaction headings or empty table should not create a confusing blank area.

TEST 9

USER PRIVACY

While signed in with the second account, add one transaction.

Confirm that only this new transaction appears.

Log out.

Sign in using the first account.

The transaction created by the second user should not appear.

Log out again.

Sign in with the second account.

The first user's transactions should not appear.

This confirms that each account can retrieve only its own records.

TEST 10

RESPONSIVE LAYOUT

Open the dashboard on a wide browser window.

Confirm that transaction information lines up clearly.

Make the browser window narrower.

The layout should adjust.

Confirm that:

- ☒ Every transaction remains readable.
- ☒ Buttons remain visible.
- ☒ Text does not overlap.
- ☒ Amounts do not run outside their containers.
- ☒ The page does not require unnecessary horizontal scrolling.
- ☒ The transaction form remains usable.
- ☒ The summary cards remain readable.

CHECKPOINT

Before moving to Chapter 7, confirm that:

- ☒ The dashboard remains protected.
- ☒ Loading states appear and disappear correctly.
- ☒ Transactions are ordered by `transaction_date`.
- ☒ Transactions with matching dates are ordered by `created_at`.
- ☒ Every required transaction value appears.
- ☒ Amounts display as Nigerian Naira.
- ☒ Stored amounts remain numeric.

- ✓ Income and Expense records are clearly identified.
- ✓ New transactions appear automatically.
- ✓ Summary cards update automatically.
- ✓ Empty states work correctly.
- ✓ Two accounts cannot see each other's transactions.
- ✓ The transaction layout works on narrow screens.

COMMON BEGINNER MISTAKES

- Some students test the transaction list using only one record.
- That is not enough to confirm ordering, totals, empty states, or privacy.
- Use several transactions with different types, categories, amounts, and dates.
- Another mistake is testing only one account.
- You cannot properly test user privacy without signing in with at least two different accounts.
- Do not rely only on what you see inside the Supabase Table Editor. Test the application itself while signed in as each user.
- A third mistake is ignoring small display problems on narrow screens.
- A dashboard may look correct on a wide screen and still be difficult to use when the browser window becomes smaller.
- Test both layouts before continuing.

BEHIND THE SCENES

- Your transaction display depends on several parts of the application working together.
- Supabase confirms the current user.
- The SELECT policy protects the records.
- The database returns authorised transactions.
- JavaScript orders and formats those transactions.
- HTML provides the display structure.
- CSS adjusts the layout for different screen sizes.

A problem in any one of these areas can affect what the user sees.

This is why complete testing matters.

THINK LIKE A SOFTWARE DESIGNER

Reliable software should behave correctly when there is a lot of data, very little data, or no data at all.

It should also behave correctly for every user, not only the first account used during development.

When testing a feature, consider several conditions instead of checking only whether it works once.

That habit helps you find problems before real users experience them.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit a transaction display
- test record ordering
- verify currency formatting
- confirm automatic refresh
- test loading and empty states
- test user privacy with two accounts
- review a responsive transaction layout

CHAPTER SUMMARY

Congratulations.

Your Expense Tracker now presents saved financial records clearly and consistently.

Transactions load only after the user has been authenticated.

They are retrieved securely, ordered correctly, and displayed with readable descriptions, types, categories, amounts, and dates.

The dashboard also handles several important conditions properly.

It displays a loading state while records are being retrieved.

It displays a helpful empty state when no transactions exist.

It provides friendly feedback if records cannot be loaded.

New transactions appear automatically after saving, and the summary cards remain in sync with the complete transaction list.

You also confirmed that separate accounts cannot see each other's financial records.

CHAPTER MILESTONE

By the end of Chapter 6, you should have successfully completed:

- ✓ Secure transaction retrieval
- ✓ Filtering requests by authenticated user ID
- ✓ Row Level Security protection
- ✓ Transaction ordering by date
- ✓ Secondary ordering by creation time
- ✓ Naira amount formatting
- ✓ Clear date formatting
- ✓ Income labels
- ✓ Expense labels
- ✓ Loading state
- ✓ Empty state
- ✓ Friendly error state
- ✓ Automatic list refresh
- ✓ Automatic summary refresh
- ✓ Responsive transaction display
- ✓ Two-account privacy testing

Your project folder should still contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

No new files were required.

TRANSITION TO CHAPTER 7

Users can now add transactions and review them clearly.

However, people sometimes enter the wrong amount, choose the wrong category, or save a transaction they no longer need.

In Chapter 7, you will make the Edit and Delete buttons fully functional.

Users will be able to update their own records and remove unwanted transactions.

Both actions will be protected carefully.

Every update and delete request will check:

- The transaction ID
- The authenticated user's ID

Row Level Security will remain enabled and continue protecting the database.

CHAPTER 7

EDITING AND DELETING TRANSACTIONS

CHAPTER INTRODUCTION

Financial records sometimes need to be corrected.

A user may enter the wrong amount, select the wrong transaction type, choose an incorrect category, or use the wrong date.

Users may also need to remove duplicate or unwanted records.

In this chapter, you will add both capabilities.

Editing and deleting are sensitive actions because they change information that already exists.

The application must identify the correct transaction and confirm that it belongs to the authenticated user.

You will not rely on the transaction ID alone.

Every update and delete query will check both the record ID and the logged-in user's ID.

Row Level Security will remain enabled throughout the chapter.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Working Edit buttons
- Transaction edit mode
- Form population for editing
- Update Transaction button state
- Cancel Edit option
- Secure update queries

- Working Delete buttons
- Delete confirmation
- Secure delete queries
- Friendly update messages
- Friendly delete messages
- Loading and disabled button states
- Automatic list and summary refresh
- Ownership testing with two accounts

By the end of the chapter, users will be able to manage the complete life cycle of their own transactions.

LESSON 1

EDITING TRANSACTIONS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In this lesson, you will make the Edit button work.

When a user selects Edit, the chosen transaction's information will be placed inside the existing transaction form.

The form will change from Add mode to Edit mode.

After the user saves the changes, the correct database record will be updated and the dashboard will refresh automatically.

WHY THIS MATTERS

People make mistakes when entering information.

A useful application should allow them to correct those mistakes without deleting the record and starting again.

Editing improves both accuracy and convenience.

Because financial information is private, the update must also be protected carefully.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Transactions save correctly.
- ☒ Transactions display correctly.
- ☒ Edit buttons appear.
- ☒ Delete buttons appear.
- ☒ The dashboard calculates totals correctly.
- ☒ Two accounts cannot see each other's transactions.

Add at least three transactions to the account you will use for testing.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project includes:

- Public landing page
- Responsive navigation
- Supabase connection
- Registration
- Email verification
- Login
- Protected dashboard
- Logout
- Transactions table
- Row Level Security
- SELECT, INSERT, UPDATE and DELETE policies
- Transaction form
- Secure transaction saving
- Transaction display
- Total Income
- Total Expenses
- Current Balance
- Loading states
- Empty states
- Responsive transaction layout
- Placeholder Edit and Delete buttons

Do not remove or break any existing functionality.

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Preserve:

- Logo linking to index.html
- Welcome heading
- Authenticated user's email
- Logout button
- Summary cards
- Transaction form
- Message area
- Recent Transactions section
- Search-free transaction display

The transaction form must continue using these fields:

- Description
- Transaction Type
- Amount
- Category
- Transaction Date

Prepare the form to support two modes:

1. Add mode
2. Edit mode

Add a visible heading or label that JavaScript can update to show whether the user is adding or editing a transaction.

The main form button should be controlled by JavaScript.

In Add mode, it should display:

Add Transaction

In Edit mode, it should display:

Update Transaction

Add a Cancel Edit button.

Hide the Cancel Edit button during normal Add mode.

Display it only when a transaction is being edited.

Ensure all IDs used by dashboard.js exist in dashboard.html.

Do not create a second transaction form.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js

Do not create another Supabase client.

Continue supporting:

- Dashboard protection
- User email display
- Logout
- Transaction validation
- Adding transactions
- Loading transactions
- Displaying transactions
- Financial summary calculations
- Loading state
- Empty state
- Error state

EDIT BUTTON

Make every Edit button functional.

Store the transaction ID safely using a data attribute.

When an Edit button is selected:

- Find the correct transaction from the transactions already stored in memory.
- Confirm that the transaction exists.
- Place its description inside the Description field.
- Select its Transaction Type.
- Place its amount inside the Amount field.
- Select its category.
- Place its transaction date inside the Date field.
- Store the transaction ID in an editing state variable.
- Change the form heading to show that the user is editing a transaction.
- Change the main form button text to:

Update Transaction

- Display the Cancel Edit button.
- Move the user to the transaction form if necessary.

UPDATING THE RECORD

When the form is submitted while Edit mode is active:

- Validate every field.
- Confirm that the user is still authenticated.
- Disable the Update Transaction button while processing.
- Display a friendly loading message.

Update the:

transactions

table.

Update only:

- description
- transaction_type
- amount
- category
- transaction_date

The UPDATE query must check both:

- The transaction ID

- The authenticated user's ID

Use both conditions in the update query.

Do not update:

- id
- user_id
- created_at

Do not accept a user_id from the form.

After a successful update:

- Display a friendly success message.
- Clear the form.
- Return the form to Add mode.
- Change the main button text back to:

Add Transaction

- Hide the Cancel Edit button.
- Clear the editing state variable.
- Reload transactions.
- Recalculate Total Income.
- Recalculate Total Expenses.
- Recalculate Current Balance.

CANCEL EDIT

When the Cancel Edit button is selected:

- Do not update the database.
- Clear the form.
- Clear the editing state variable.
- Return the form to Add mode.
- Change the main button text to:

Add Transaction

- Hide the Cancel Edit button.
- Clear any old editing message where appropriate.

ERROR HANDLING

If the update fails:

- Display a friendly message.
- Do not expose technical database details to the user.
- Record the full error using `console.error()`.
- Re-enable the button.
- Keep the form values so the user does not lose their changes.

IMPORTANT

- Do not implement Delete functionality yet.
- Delete buttons should remain visible.
- Do not add search.
- Do not add filters.
- Do not add sorting controls.

AUTH.CSS

- Keep every existing style.
- Add or improve professional styling for:
 - Edit mode indicator
 - Update Transaction button
 - Cancel Edit button
 - Form action buttons
 - Edit buttons inside transaction rows
 - Disabled button states
 - Loading state during updates
- Ensure the buttons remain easy to use on narrow screens.
- Do not remove existing authentication, dashboard, form, summary, transaction, loading, empty, or error styles.

SECURITY REQUIREMENTS

- Row Level Security must remain enabled.
- The UPDATE policy must remain active.
- The update query must check:
 - The record ID

- The logged-in user's ID
- Do not disable Row Level Security.
- Do not use a Service Role Key.
- Do not use a Secret Key.
- Do not use the database password.
- Do not allow the user to supply user_id.

TECHNICAL REQUIREMENTS

- Return complete updated files.
- Do not return snippets.
- Do not create new files.
- Do not rename existing files.
- Do not create another Supabase client.
- Keep the dashboard script loading order:
 1. Supabase CDN
 2. supabase-config.js
 3. dashboard.js
- Every HTML ID expected by JavaScript must exist.
- Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

dashboard.html

dashboard.js

auth.css

The updated dashboard should allow users to edit their own transactions securely.

The Delete buttons should remain visible but should not become functional yet.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open dashboard.html in Notepad.

Replace its complete contents with the updated dashboard.html code.

Click File.

Click Save.

Close Notepad.

Open dashboard.js in Notepad.

Replace its complete contents with the updated dashboard.js code.

Click File.

Click Save.

Close Notepad.

Open auth.css in Notepad.

Replace its complete contents with the updated auth.css code.

Click File.

Click Save.

Close Notepad.

Confirm that the filenames remain:

dashboard.html

dashboard.js

auth.css

TEST YOUR WORK

Sign in to your Expense Tracker.

Select the Edit button beside one transaction.

Confirm that:

- ☒ The correct transaction details appear in the form.
- ☒ The form indicates that you are editing.
- ☒ The main button changes to Update Transaction.
- ☒ The Cancel Edit button appears.

Change the description.

Change the amount.

Click:

Update Transaction

The button should become disabled while processing.

After the update succeeds:

- ☒ A success message should appear.
- ☒ The form should clear.
- ☒ The main button should return to Add Transaction.
- ☒ The Cancel Edit button should disappear.
- ☒ The transaction list should refresh.
- ☒ The updated values should appear.
- ☒ The summary cards should recalculate.

Now select Edit on another transaction.

Change some information.

Select:

Cancel Edit

Confirm that:

- ✓ The database record is not changed.
- ✓ The form clears.
- ✓ The main button returns to Add Transaction.
- ✓ The Cancel Edit button disappears.
- ✓ Normal Add mode works again.

CHECKPOINT

Before moving on, confirm that:

- ✓ Edit buttons work.
- ✓ The correct transaction enters the form.
- ✓ Edit mode is clearly visible.
- ✓ Update validation works.
- ✓ The update query checks the record ID.
- ✓ The update query checks the authenticated user's ID.
- ✓ Row Level Security remains enabled.
- ✓ Successful updates refresh the transaction list.
- ✓ Successful updates refresh the summary cards.
- ✓ Cancel Edit works.
- ✓ Adding new transactions still works.
- ✓ Logout still works.
- ✓ Dashboard protection still works.

COMMON BEGINNER MISTAKES

- A common mistake is building a second form for editing.
- That creates unnecessary duplication and makes the dashboard harder to maintain.
- Use the existing transaction form for both adding and editing.
- Another mistake is updating a record using only its ID.

The query should also check the authenticated user's ID.

This makes the request more precise and adds protection alongside Row Level Security.

Some beginners also forget to clear Edit mode after a successful update.

If the editing state is not reset, the next form submission may update the old transaction instead of creating a new one.

Always return the form to Add mode after updating or cancelling.

BEHIND THE SCENES

When the user selects Edit, the application does not immediately change the database.

It first loads the selected transaction into the form.

The record changes only after the user submits the updated values.

The editing state variable tells JavaScript whether the form should create a new record or update an existing one.

If no transaction ID is stored in the editing state, the form works in Add mode.

If a transaction ID is stored, the form works in Edit mode.

THINK LIKE A SOFTWARE DESIGNER

A good editing experience should always make the current state clear.

Users should know whether they are creating a new record or changing an existing one.

The button label, form heading, and Cancel Edit option help prevent accidental updates.

Software should not force users to guess what will happen when they select a button.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports editing transactions. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- reuse one form for adding and editing
- load an existing transaction into a form
- manage Add mode and Edit mode
- update a database record securely
- check both the transaction ID and authenticated user ID
- cancel an edit safely
- refresh records and totals after an update

In the next lesson, you will make the Delete buttons functional and add a clear confirmation step before any record is removed.

CHAPTER 7

EDITING AND DELETING TRANSACTIONS

LESSON 2

DELETING TRANSACTIONS

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will make the Delete buttons fully functional.

When a user selects Delete, the application will ask for confirmation before removing the transaction.

If the user confirms, the correct record will be deleted from Supabase.

The transaction list and financial summary cards will then refresh automatically.

WHY THIS MATTERS

Deleting information is permanent.

A user may select the wrong button or change their mind after seeing the transaction details.

A confirmation step gives them one final opportunity to cancel before the record is removed.

The delete request must also be secure.

The application must check both the transaction ID and the authenticated user's ID before attempting to remove the record.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Transactions can be added.
- ☒ Transactions can be displayed.
- ☒ Transactions can be edited.
- ☒ Cancel Edit works.
- ☒ Delete buttons appear beside each transaction.

✓ Row Level Security remains enabled.

✓ The DELETE policy exists.

Add at least three transactions before using the Build Prompt.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project includes:

- Public landing page
- Responsive navigation
- Supabase connection
- Registration
- Email verification
- Login
- Protected dashboard
- Logout
- Transactions table
- Row Level Security
- SELECT, INSERT, UPDATE and DELETE policies
- Transaction form
- Secure transaction saving
- Transaction display
- Total Income
- Total Expenses
- Current Balance
- Loading states
- Empty states
- Responsive transaction layout

- Working Edit buttons
 - Add mode
 - Edit mode
 - Cancel Edit
- Do not remove or break any existing functionality.

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Preserve:

- Logo linking to index.html
- Welcome heading
- Authenticated user's email
- Logout button
- Summary cards
- Transaction form
- Add mode
- Edit mode
- Cancel Edit button
- Message area
- Recent Transactions section
- Edit buttons
- Delete buttons

Do not create a second form.

Do not create a second transaction list.

Ensure all IDs and classes used by dashboard.js exist.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

`supabaseClient`

from:

`supabase-config.js`

Do not create another Supabase client.

Continue supporting:

- Dashboard protection
- User email display
- Logout
- Transaction validation
- Adding transactions
- Editing transactions
- Cancel Edit
- Loading transactions
- Displaying transactions
- Financial summary calculations
- Loading state
- Empty state
- Error state

DELETE BUTTON

Make every Delete button functional.

Store the transaction ID safely using a data attribute.

When a Delete button is selected:

- Find the correct transaction from the transactions already stored in memory.
- Confirm that the transaction exists.
- Show a clear confirmation message before deleting.

The confirmation should include the transaction description where possible.

Example:

Are you sure you want to delete "Fuel"?

This action cannot be undone.

Use the browser's built-in confirmation dialog or another simple confirmation method that does not require a new file.

If the user cancels:

- Do not send a delete request.
- Do not change the transaction list.
- Do not change the summary cards.

If the user confirms:

- Confirm that the user is still authenticated.
- Disable the selected Delete button while processing.
- Change its text to show that deletion is taking place.
- Display a friendly loading message where appropriate.

Delete the record from:

transactions

The DELETE query must check both:

- The transaction ID
- The authenticated user's ID

Use both conditions in the delete query.

Do not delete using only the transaction ID.

AFTER SUCCESSFUL DELETION

After the record is deleted successfully:

- Display a friendly success message.

Example:

Transaction deleted successfully.

- Reload transactions.
- Refresh the Recent Transactions section.
- Recalculate Total Income.
- Recalculate Total Expenses.
- Recalculate Current Balance.
- If the deleted transaction was currently open in Edit mode, cancel Edit mode and clear the form.

- If no transactions remain, display the existing empty state.

ERROR HANDLING

If the delete request fails:

- Display a friendly error message.
- Do not expose technical database details to the user.
- Record the full error using `console.error()`.
- Restore the Delete button text.
- Re-enable the Delete button.
- Keep the transaction visible.

BUTTON HANDLING

Do not allow several delete requests for the same transaction while one request is already processing.

Make sure button states return to normal if the request fails.

IMPORTANT

- Do not remove Edit functionality.
- Do not remove Add functionality.
- Do not add search.
- Do not add filters.
- Do not add sorting controls.
- Those features belong in Chapter 8.

AUTH.CSS

Keep every existing style.

Add or improve professional styling for:

- Delete buttons
- Delete button hover state
- Delete button focus state
- Disabled Delete buttons
- Deleting state

- Success messages after deletion

- Error messages after deletion

Make the Delete button visually different from the Edit button.

Do not depend only on colour.

The button text must clearly say:

Delete

Ensure action buttons remain easy to use on narrow screens.

Do not remove any existing authentication, dashboard, form, summary, transaction, loading, empty, error, or edit-mode styles.

SECURITY REQUIREMENTS

Row Level Security must remain enabled.

The DELETE policy must remain active.

The delete query must check:

- The record ID
- The logged-in user's ID

Do not disable Row Level Security.

Do not use a Service Role Key.

Do not use a Secret Key.

Do not use the database password.

Do not accept user_id from the page.

Always retrieve the logged-in user from Supabase.

TECHNICAL REQUIREMENTS

- Return complete updated files.
- Do not return snippets.
- Do not create new files.
- Do not rename existing files.
- Do not create another Supabase client.
- Keep the dashboard script loading order:

1. Supabase CDN

2. supabase-config.js

3. dashboard.js

- Every HTML ID expected by JavaScript must exist.
- Every class used by CSS must match the HTML.
- Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

dashboard.html

dashboard.js

auth.css

The updated dashboard should allow authenticated users to delete only their own transactions after confirming the action.

All existing features must continue working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open dashboard.html in Notepad.

Replace its complete contents with the updated dashboard.html code.

Click File.

Click Save.

Close Notepad.

Open dashboard.js in Notepad.

Replace its complete contents with the updated dashboard.js code.

Click File.

Click Save.

Close Notepad.

Open auth.css in Notepad.

Replace its complete contents with the updated auth.css code.

Click File.

Click Save.

Close Notepad.

Confirm that the filenames remain:

dashboard.html

dashboard.js

auth.css

Make sure they have not been saved as:

dashboard.html.txt

dashboard.js.txt

auth.css.txt

TEST YOUR WORK

Sign in to your Expense Tracker.

Choose one transaction that you do not mind deleting.

Click:

Delete

A confirmation message should appear.

Select Cancel.

Confirm that:

☒ The transaction remains visible.

☒ The summary cards do not change.

☒ No success message appears.

Click Delete again.

This time, confirm the deletion.

The Delete button should become disabled while processing.

Its text should show that deletion is taking place.

After the request succeeds:

☒ A success message should appear.

☒ The transaction should disappear.

☒ Total Income should update if an Income transaction was deleted.

☒ Total Expenses should update if an Expense transaction was deleted.

☒ Current Balance should update.

☒ The browser should not refresh manually.

Now choose another transaction and select Edit.

While that transaction is in Edit mode, delete the same transaction from its Delete button if the layout allows it.

After deletion:

☒ Edit mode should end.

☒ The form should clear.

☒ The main form button should return to Add Transaction.

☒ The Cancel Edit button should disappear.

Finally, delete every remaining transaction in a test account.

When no transactions remain, the empty-state message should appear.

CHECKPOINT

Before moving on, confirm that:

☒ Delete buttons work.

- ✓ A confirmation message appears.
- ✓ Cancelling does not remove the record.
- ✓ Confirming removes the correct record.
- ✓ The delete query checks the transaction ID.
- ✓ The delete query checks the authenticated user's ID.
- ✓ Row Level Security remains enabled.
- ✓ Deleted transactions disappear automatically.
- ✓ Summary cards recalculate automatically.
- ✓ Empty state appears when no records remain.
- ✓ Failed requests restore the Delete button.
- ✓ Add functionality still works.
- ✓ Edit functionality still works.
- ✓ Cancel Edit still works.
- ✓ Logout still works.
- ✓ Dashboard protection still works.

COMMON BEGINNER MISTAKES

- A common mistake is deleting a transaction immediately when the button is selected.
- Always ask for confirmation first.
- Users can select the wrong button, especially on smaller screens.
- Another mistake is deleting by transaction ID alone.
- The query must also check the authenticated user's ID.
- This makes the request more precise and works alongside Row Level Security.
- Some beginners remove the transaction from the page before Supabase confirms that the delete request succeeded.
- Do not do that.
- Wait for the database response first.
- If the request fails, the record should remain visible.

BEHIND THE SCENES

The confirmation step happens before the application contacts Supabase.

If the user cancels, nothing is sent to the database.

If the user confirms, JavaScript retrieves the authenticated user's ID and sends a delete request containing two conditions.

The first condition identifies the transaction.

The second confirms who owns it.

Supabase then applies the DELETE policy before removing the record.

Only after the database confirms success should the dashboard reload the transaction list and recalculate the summary cards.

THINK LIKE A SOFTWARE DESIGNER

Permanent actions deserve more protection than ordinary actions.

Selecting Edit does not immediately change the database.

Selecting Delete can permanently remove information.

The application should make that difference clear.

Good software gives users enough information to understand the consequence of an action before it happens.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports deleting transactions. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- make Delete buttons functional
- ask for confirmation before deleting
- delete a transaction securely
- check both the record ID and authenticated user ID
- handle loading and disabled button states
- refresh records and totals after deletion
- recover gracefully when a delete request fails

In the next lesson, you will test the complete editing and deletion system using two accounts before moving to Chapter 8.

CHAPTER 7

EDITING AND DELETING TRANSACTIONS

LESSON 3

TESTING OWNERSHIP AND RECORD MANAGEMENT

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not create any new features.

You will test the complete Add, Edit, and Delete system to confirm that users can manage their own records without affecting anyone else's transactions.

This is one of the most important security tests in the workbook.

WHY THIS MATTERS

A feature can appear to work correctly while still containing a serious ownership problem.

For example, an Edit button may update the correct record during normal testing but fail to check who owns that record.

Testing with two accounts helps confirm that the application and database security rules work together properly.

BEFORE YOU CONTINUE

Prepare two registered and verified accounts.

For clarity, you can think of them as:

Account A

Account B

Each account should have at least two transactions.

Make sure you can sign in and out successfully before beginning.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete each test carefully.

Do not continue to Chapter 8 until every test passes.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by testing your application.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

TEST 1

ADD MODE

Sign in using Account A.

Add a new Income transaction.

Confirm that:

- ☒ Validation works.
- ☒ The button becomes disabled while saving.
- ☒ A success message appears.

☒ The form clears.

☒ The transaction appears automatically.

☒ Total Income updates.

☒ Current Balance updates.

Now add a new Expense transaction.

Confirm that:

☒ Total Expenses updates.

☒ Current Balance updates.

TEST 2

EDIT MODE

Select Edit beside one of Account A's transactions.

Confirm that:

☒ The correct record enters the form.

☒ The form heading changes.

☒ The main button changes to Update Transaction.

☒ The Cancel Edit button appears.

Change the amount and category.

Save the update.

Confirm that:

☒ The correct record changes.

☒ Other records remain unchanged.

☒ The form returns to Add mode.

☒ The summary cards recalculate.

TEST 3**CANCEL EDIT**

Select Edit beside another transaction.

Change several fields.

Select:

Cancel Edit

Confirm that:

- ☒ No database value changes.
- ☒ The form clears.
- ☒ Add mode returns.
- ☒ The main button says Add Transaction.
- ☒ The Cancel Edit button disappears.

TEST 4**DELETE CANCELLATION**

Select Delete beside one transaction.

Cancel the confirmation.

Confirm that:

- ☒ The record remains visible.
- ☒ The summary cards do not change.
- ☒ No success message appears.

TEST 5**SUCCESSFUL DELETE**

Select Delete again.

Confirm the action.

Confirm that:

- ☒ The correct record disappears.
- ☒ Other records remain visible.
- ☒ The summary cards recalculate.
- ☒ A success message appears.

TEST 6**ACCOUNT SEPARATION**

Write down the descriptions of the transactions visible in Account A.

Log out.

Sign in using Account B.

Confirm that:

- ☒ Account A's transactions do not appear.
- ☒ Only Account B's transactions appear.

Edit one of Account B's transactions.

Confirm that the update succeeds.

Delete one of Account B's transactions.

Confirm that the deletion succeeds.

Log out.

Sign in again using Account A.

Confirm that:

- ☒ Account A's records remain unchanged.
- ☒ Account B's updated record does not appear.
- ☒ Account B's deleted record does not affect Account A.

TEST 7

QUERY CONDITIONS

Open dashboard.js in Notepad.

Find the code used for updating transactions.

Confirm that the update query checks both:

- The transaction ID
- The authenticated user's ID

Now find the delete query.

Confirm that it also checks both:

- The transaction ID
- The authenticated user's ID

Do not continue if either query checks only the transaction ID.

TEST 8

ROW LEVEL SECURITY

Open your Supabase project.

Confirm that Row Level Security remains enabled for:

transactions

Review the policies.

Confirm that all four still exist:

☒ SELECT

☒ INSERT

☒ UPDATE

☒ DELETE

Every policy should compare:

user_id

with:

auth.uid()

TEST 9

EMPTY STATE AFTER DELETION

Use a test account with only one transaction.

Delete that transaction.

Confirm that:

☒ The transaction disappears.

☒ The summary cards display ₦0.00.

☒ The empty-state message appears.

☒ The transaction table does not leave a confusing blank area.

TEST 10

FAILED ACTION RECOVERY

-
- Temporarily disconnect your internet connection.
- Try updating a transaction.
- The application should display a friendly error message.
- The form values should remain available.
- The main button should become usable again.
- Reconnect your internet connection.
- Try deleting a transaction while the connection is unavailable.
- The application should display a friendly error message.
- The transaction should remain visible.
- The Delete button should become usable again after the failed request.
- Reconnect and refresh the browser before continuing.

CHECKPOINT

Before moving to Chapter 8, confirm that:

- ☒ Adding transactions still works.
- ☒ Editing transactions works.
- ☒ Cancel Edit works.
- ☒ Deleting transactions works.
- ☒ Delete confirmation works.
- ☒ Updates check the record ID.
- ☒ Updates check the authenticated user's ID.
- ☒ Deletes check the record ID.
- ☒ Deletes check the authenticated user's ID.

- ✓ Row Level Security remains enabled.
- ✓ All four policies remain active.
- ✓ Two accounts remain completely separate.
- ✓ Failed actions recover properly.
- ✓ Empty states remain correct.
- ✓ Summary cards stay accurate.

COMMON BEGINNER MISTAKES

- Some students test editing and deleting using only one account.
- That confirms basic functionality but does not properly test privacy.
- Always use two accounts.
- Another mistake is checking only whether the visible transaction changed.
- You should also confirm that other records remain unchanged and that the correct row was updated inside Supabase.
- A third mistake is ignoring what happens when the internet connection fails.
- Users should not be left with permanently disabled buttons or lost form values after an unsuccessful request.

BEHIND THE SCENES

- Your application now uses several layers of protection.
- The dashboard checks whether the user is authenticated.
- The JavaScript query includes the transaction ID and authenticated user's ID.
- Row Level Security checks ownership again inside the database.
- These layers work together.
- If one layer contains a mistake, the others still reduce the chance of unauthorised access.
- This approach is often called defence in depth.

THINK LIKE A SOFTWARE DESIGNER

A complete feature includes more than the successful action.

You must also consider cancellation, failed requests, empty results, loading states, and ownership.

Software becomes reliable when it behaves sensibly in all of these situations.

WHAT YOU LEARNED

In this lesson you learned how to:

- test the complete record-management flow
- verify update ownership
- verify delete ownership
- test with two separate accounts
- confirm Row Level Security remains active
- test recovery after failed requests
- confirm that totals remain accurate

CHAPTER SUMMARY

Congratulations.

Your Expense Tracker now supports the complete transaction life cycle.

Authenticated users can:

- Add transactions
- View transactions
- Edit their own transactions
- Delete their own transactions

The application clearly shows whether the form is in Add mode or Edit mode.

Users can cancel an edit without changing the database.

Delete actions require confirmation before a record is removed.

After every successful update or deletion, the transaction list and financial summary cards refresh automatically.

Both update and delete queries check:

- The record ID
- The authenticated user's ID

Row Level Security remains enabled and provides database-level protection.

CHAPTER MILESTONE

By the end of Chapter 7, you should have successfully built:

- ✓ Working Edit buttons
- ✓ Add mode
- ✓ Edit mode
- ✓ Form population
- ✓ Update Transaction state
- ✓ Cancel Edit
- ✓ Secure update queries
- ✓ Working Delete buttons
- ✓ Delete confirmation
- ✓ Secure delete queries
- ✓ Disabled button states
- ✓ Friendly update messages
- ✓ Friendly delete messages
- ✓ Automatic transaction refresh
- ✓ Automatic summary refresh
- ✓ Two-account ownership testing
- ✓ Failed-request recovery

Your project folder should still contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

No new files were required.

TRANSITION TO CHAPTER 8

Your Expense Tracker now supports the main actions users expect from a working financial application.

They can add, view, edit, and delete their own records securely.

As the number of transactions grows, users need faster ways to find specific information.

In Chapter 8, you will make the dashboard more powerful by adding:

- Search
- Category filters
- Transaction Type filters
- Sorting controls
- Responsive improvements
- Final user experience improvements

These features will improve how users work with the records they have already created without changing the database structure.

CHAPTER 8

MAKING THE DASHBOARD MORE POWERFUL

CHAPTER INTRODUCTION

Your Expense Tracker can now handle the complete transaction process.

Users can add records, review them, correct mistakes, and remove transactions they no longer need.

The next step is to make the dashboard easier to use as the number of records grows.

A user with ten transactions may be able to find information by scrolling.

A user with hundreds of transactions needs better tools.

In this chapter, you will add search, filters, and sorting controls.

These tools will work with the transactions already loaded into the browser. They will not repeatedly request the database each time the user types or changes a filter.

The financial summary cards will continue showing totals for all saved transactions.

Search, filtering, and sorting will change only the records shown inside the Recent Transactions section.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Live transaction search
- Category filter
- Transaction Type filter
- Sorting controls
- Combined search and filtering
- Clear no-results state

- Reset controls
- Improved responsive layout
- Better mobile usability
- Final dashboard polish

By the end of the chapter, users will be able to find and organise their transactions quickly without changing the records stored in Supabase.

LESSON 1

ADDING SEARCH AND FILTERS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In this lesson, you will add tools that allow users to search their transaction descriptions and filter records by category or transaction type.

The results will update immediately as the user types or changes a filter.

The application will search and filter the transactions already stored in memory after they have been retrieved from Supabase.

WHY THIS MATTERS

A long transaction list becomes difficult to use without a quick way to narrow the results.

Search helps users find records using words they remember.

Filters help them focus on a category or transaction type.

When these tools work together, users can answer more specific questions.

For example, someone may want to see only Food expenses or find every transaction containing the word school.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Transactions can be added.
- ☒ Transactions can be edited.
- ☒ Transactions can be deleted.
- ☒ Transactions are stored in memory after loading.
- ☒ Summary cards calculate correctly.

✓ At least eight test transactions exist.

Use different descriptions, categories, and transaction types so you can test the new tools properly.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project includes:

- Public landing page
- Responsive navigation
- Supabase connection
- Registration
- Email verification
- Login
- Protected dashboard
- Logout
- Transactions table
- Row Level Security
- SELECT, INSERT, UPDATE and DELETE policies
- Adding transactions
- Viewing transactions
- Editing transactions
- Deleting transactions
- Total Income
- Total Expenses
- Current Balance
- Loading states
- Empty states
- Friendly error states

- Responsive transaction display
- Do not remove or break any existing functionality.
- Return complete updated files only.
- Do not return snippets.
- Do not return explanations.
- Return complete updated versions of:
- dashboard.html
 - dashboard.js
 - auth.css

DASHBOARD.HTML

Keep everything already built.

Preserve:

- Logo linking to index.html
- Welcome heading
- Authenticated user's email
- Logout button
- Summary cards
- Transaction form
- Add mode
- Edit mode
- Cancel Edit
- Message area
- Recent Transactions section
- Edit buttons
- Delete buttons

Above the transaction headings, add a professional search and filter area.

Include:

SEARCH FIELD

Add a text input.

Use this placeholder:

Search transactions...

Give it a clear label for accessibility.

The search field should allow users to search by:

- Description
- Category
- Transaction Type

CATEGORY FILTER

Add a dropdown with these options:

All Categories

Food

Transport

Shopping

Salary

Utilities

Health

Entertainment

Education

Other

TRANSACTION TYPE FILTER

Add a dropdown with these options:

All Types

Income

Expense

RESET BUTTON

Add a button labelled:

Reset

The Reset button should clear:

- The search field
- The Category filter

- The Transaction Type filter

After resetting, all transactions should appear again.

Give every control a clear ID that matches dashboard.js.

Do not create a second transaction list.

Do not create a second set of summary cards.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

supabaseClient

from:

supabase-config.js

Do not create another Supabase client.

Continue supporting:

- Dashboard protection
- User email display
- Logout
- Adding transactions
- Editing transactions
- Cancelling Edit mode
- Deleting transactions
- Loading transactions
- Displaying transactions
- Total Income
- Total Expenses
- Current Balance
- Loading states
- Empty states
- Error states

TRANSACTIONS IN MEMORY

After transactions are loaded successfully, store the complete authenticated user's transaction list in memory.

Do not request Supabase again every time the user types into the search field or changes a filter.

SEARCH

Listen for changes as the user types.

Search should check:

- description
- category
- transaction_type

Search must ignore uppercase and lowercase differences.

Remove unnecessary spaces from the beginning and end of the search text before comparing values.

CATEGORY FILTER

When a specific category is selected, display only transactions from that category.

When:

All Categories

is selected, do not apply a category restriction.

TRANSACTION TYPE FILTER

When Income is selected, display only Income transactions.

When Expense is selected, display only Expense transactions.

When:

All Types

is selected, do not apply a transaction type restriction.

COMBINED RESULTS

Search and both filters must work together.

A transaction should appear only when it matches every active condition.

Example:

If the user searches for:

school

selects:

Education

and selects:

Expense

display only Expense transactions in the Education category that also match the word school.

NO-RESULTS STATE

If the user has saved transactions but none match the active search and filters, display:

No transactions match your search or filters.

Do not display the normal empty-account message in this situation.

The normal empty-account message should still appear when the user has no saved transactions at all.

RESET BUTTON

When Reset is selected:

- Clear the search input.
- Select All Categories.
- Select All Types.
- Display every transaction again.
- Clear the no-results state.

SUMMARY CARDS

Do not recalculate Total Income, Total Expenses, or Current Balance using filtered results.

The summary cards must continue showing totals for all transactions stored by the authenticated user.

ADD, EDIT AND DELETE

After adding, editing, or deleting a transaction:

- Reload the complete transaction list.

- Update the transactions stored in memory.
- Reapply the current search and filter settings where sensible.
- Keep summary cards based on all transactions.

EVENT HANDLING

- Use one clear function to apply the active search and filters.
- Use one clear function to display the resulting transactions.
- Do not create duplicate event listeners every time records are reloaded.

IMPORTANT

- Do not add sorting controls yet.
- Sorting will be added in the next lesson.

AUTH.CSS

- Keep every existing style.
- Add or improve professional styling for:
 - Search and filter area
 - Search label
 - Search input
 - Category filter
 - Transaction Type filter
 - Reset button
 - No-results state
 - Focus states
 - Responsive filter layout
- On wide screens, arrange the controls neatly in one row where space permits.
- On narrower screens, allow the controls to stack without becoming cramped.
- Make every field and button easy to use on a touch screen.
- Prevent the controls from creating horizontal scrolling.
- Do not remove any existing authentication, dashboard, form, summary, transaction, loading, empty, error, edit, or delete styles.

TECHNICAL REQUIREMENTS

- Everything already built must continue working.
- Return complete dashboard.html.
- Return complete dashboard.js.
- Return complete auth.css.
- Do not return partial changes.
- Do not rename files.
- Do not create new files.
- Do not create another Supabase client.
- Keep the dashboard script loading order:
 1. Supabase CDN
 2. supabase-config.js
 3. dashboard.js
- Every HTML ID used by JavaScript must exist.
- Every class used by CSS must match the HTML.
- Do not repeatedly request the database during search or filtering.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

dashboard.html

dashboard.js

auth.css

The updated dashboard should include:

- ☒ Live search
- ☒ Category filtering
- ☒ Transaction Type filtering
- ☒ Combined search and filtering

- ✓ Reset button
- ✓ No-results state
- ✓ Existing Add, Edit, and Delete functionality
- ✓ Existing financial summaries

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

- Before replacing an existing file, copy your complete project folder.
- Save the copy outside your working project folder.
- Name the backup clearly using the current chapter and lesson.
- Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open dashboard.html in Notepad.

Replace its complete contents with the updated dashboard.html code.

Click File.

Click Save.

Close Notepad.

Open dashboard.js in Notepad.

Replace its complete contents with the updated dashboard.js code.

Click File.

Click Save.

Close Notepad.

Open auth.css in Notepad.

Replace its complete contents with the updated auth.css code.

Click File.

Click Save.

Close Notepad.

Confirm that the filenames remain:

dashboard.html

dashboard.js

auth.css

TEST YOUR WORK

Sign in to your Expense Tracker.

Confirm that the search field, two filters, and Reset button appear above the transaction list.

SEARCH TEST

Type part of a transaction description.

The list should update immediately.

Search using uppercase letters.

The same matching records should appear.

Search using lowercase letters.

The results should remain correct.

Search for a category name.

Matching records should appear.

Clear the search field.

CATEGORY FILTER TEST

Select:

Food

Only transactions in the Food category should appear.

Select:

Salary

Only Salary transactions should appear.

Return the filter to:

All Categories

Every transaction should appear again.

TRANSACTION TYPE FILTER TEST

Select:

Income

Only Income transactions should appear.

Select:

Expense

Only Expense transactions should appear.

Return the filter to:

All Types

Every transaction should appear again.

COMBINED TEST

Enter a search word.

Choose a category.

Choose a transaction type.

Confirm that only records matching all three conditions appear.

NO-RESULTS TEST

Enter a search term that does not match any transaction.

The page should display:

No transactions match your search or filters.

The normal empty-account message should not appear.

RESET TEST

Apply a search and both filters.

Select:

Reset

Confirm that:

- ☒ The search field clears.
- ☒ Category returns to All Categories.
- ☒ Transaction Type returns to All Types.
- ☒ Every transaction appears.

SUMMARY TEST

Apply filters that show only one or two records.

Confirm that:

- ☒ Total Income remains based on all Income transactions.
- ☒ Total Expenses remains based on all Expense transactions.
- ☒ Current Balance remains based on all saved transactions.

ADD, EDIT AND DELETE TEST

Keep one filter active.

Add a matching transaction.

Confirm that it appears after saving.

Edit the transaction so it no longer matches the active filter.

Confirm that it disappears from the filtered view after the update.

Reset the controls and confirm that the edited transaction still exists.

Delete a transaction.

Confirm that the stored list, filtered view, and summary cards update correctly.

CHECKPOINT

Before moving on, confirm that:

- ☒ Search works as the user types.
- ☒ Search ignores uppercase and lowercase differences.

- ✓ Category filtering works.
- ✓ Transaction Type filtering works.
- ✓ Search and filters work together.
- ✓ Reset restores the complete list.
- ✓ No-results state works.
- ✓ Empty-account state still works.
- ✓ Search does not repeatedly request Supabase.
- ✓ Summary cards remain based on all transactions.
- ✓ Add still works.
- ✓ Edit still works.
- ✓ Delete still works.
- ✓ Dashboard protection still works.
- ✓ Logout still works.

COMMON BEGINNER MISTAKES

- A common mistake is requesting transactions from Supabase every time the user types a letter.
- This creates unnecessary requests and may make the page feel slow.
- Load the records once, store them in memory, and filter that stored list.
- Another mistake is treating the no-results state as the same thing as the empty-account state.
- These are different situations.
- The empty-account state means the user has no saved transactions.
- The no-results state means transactions exist, but none match the current search or filters.
- Some beginners also recalculate the summary cards from the filtered records.
- Do not do that.
- Search and filters should change only the visible transaction list.
- They should not change the user's overall financial summary.

BEHIND THE SCENES

The complete transaction list remains stored in memory after Supabase returns it.

When the user types or changes a filter, JavaScript creates a temporary filtered list.

The original list remains unchanged.

This makes it possible to reset the controls and display every transaction again without contacting Supabase.

The summary cards also continue using the complete list rather than the temporary filtered results.

THINK LIKE A SOFTWARE DESIGNER

Search and filters should make information easier to find without changing the information itself.

The user should always understand that applying a filter does not delete hidden records.

Clear controls, a visible Reset button, and a helpful no-results message reduce confusion.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports adding search and filters. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- search transactions stored in memory
- filter by category
- filter by transaction type
- combine several conditions

- distinguish an empty state from a no-results state
- reset search and filter controls
- keep financial summaries based on all records

In the next lesson, you will add sorting controls so users can organise transactions by date, amount, or description.

CHAPTER 8

MAKING THE DASHBOARD MORE POWERFUL

LESSON 2

ADDING SORTING CONTROLS

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In the previous lesson, users learned how to search and filter their transactions.

In this lesson, you will allow users to control the order in which those transactions are displayed.

Instead of always showing the newest transactions first, users will be able to choose how they want to organise their records.

Sorting changes only the order in which transactions are displayed.

It does not change the information stored in the database.

WHY THIS MATTERS

Different users look for different kinds of information.

Someone reviewing their recent spending may want to see the newest transactions first.

Another person may want to find the largest expense or the smallest payment.

Giving users control over the display order makes the dashboard much easier to use.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Transactions load correctly.
- ☒ Search works.
- ☒ Category filtering works.
- ☒ Transaction Type filtering works.

- ✓ The Reset button works.
- ✓ At least ten transactions exist with different amounts and dates.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- Registration
- Email verification
- Login
- Protected dashboard
- Logout
- Transaction form
- Add Transaction
- Edit Transaction
- Delete Transaction
- Search
- Category filtering
- Transaction Type filtering
- Reset button
- Financial summary cards

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

DASHBOARD.HTML

Keep everything already built.

Do not remove:

- Search field
- Category filter
- Transaction Type filter
- Reset button

Add a new dropdown labelled:

Sort By

Include these options:

Newest First

Oldest First

Highest Amount

Lowest Amount

Description (A–Z)

Description (Z–A)

Give the dropdown a clear ID that matches dashboard.js.

Arrange the Sort By control neatly beside the existing search and filter controls.

Keep the layout responsive.

AUTH.CSS

Keep every existing style.

Add professional styling for:

- Sort By label
- Sort By dropdown
- Responsive layout including the new control

Ensure the controls wrap neatly onto multiple rows on smaller screens.

DASHBOARD.JS

Keep every existing feature working.

Continue using:

`supabaseClient`

from:

`supabase-config.js`

Do not create another Supabase client.

Continue supporting:

- Dashboard protection
- Logout
- Add Transaction
- Edit Transaction
- Delete Transaction
- Search
- Category filtering
- Transaction Type filtering
- Reset button
- Summary cards

SORTING

After search and filtering have been applied, sort the remaining transactions based on the selected option.

Implement:

Newest First

Sort by:

`transaction_date`

descending.

If two transactions have the same `transaction_date`, sort by:

`created_at`

descending.

Oldest First

Sort by:

transaction_date

ascending.

If two transactions have the same transaction_date, sort by:

created_at

ascending.

Highest Amount

Sort by:

amount

descending.

Lowest Amount

Sort by:

amount

ascending.

Description (A–Z)

Sort alphabetically by:

description

Description (Z–A)

Sort alphabetically in reverse order.

SEARCH, FILTERS AND SORTING

Search, filters and sorting must work together.

Changing the Sort By dropdown should immediately refresh the transaction list.

Changing the search or filters should preserve the selected sort order.

RESET BUTTON

When Reset is selected:

Clear:

- Search
- Category
- Transaction Type

Return Sort By to:

Newest First

Refresh the transaction list.

SUMMARY CARDS

Do not calculate summary cards using the sorted or filtered list.

Continue using every saved transaction.

IMPORTANT

Everything already built must continue working.

Do not request the database again simply because the sort order changed.

Sort only the transactions already stored in memory.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

✓ dashboard.html

✓ dashboard.js

✓ auth.css

The dashboard should now support:

✓ Search

✓ Filters

✓ Sorting

✓ Reset

✓ Existing functionality

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

Replace its complete contents.

Save.

Repeat the same process for:

dashboard.js

auth.css

Do not create duplicate files.

TEST YOUR WORK

Sign in to your Expense Tracker.

Confirm that the Sort By dropdown appears.

Test:

Newest First

The newest transactions should appear first.

Test:

Oldest First

The oldest transactions should appear first.

Test:

Highest Amount

The largest amount should appear first.

Test:

Lowest Amount

The smallest amount should appear first.

Test:

Description (A–Z)

Transactions should appear alphabetically.

Test:

Description (Z–A)

Transactions should appear in reverse alphabetical order.

Now apply:

A search

A category filter

A transaction type filter

Change the sort order.

Confirm that:

☒ Search still works.

☒ Filters still work.

☒ The selected sort order is preserved.

Click:

Reset

Confirm that:

☒ Search clears.

☒ Filters reset.

☒ Sort returns to Newest First.

☒ Every transaction appears.

☒ Summary cards remain unchanged.

CHECKPOINT

Before moving on, confirm that:

- ✓ Every sort option works.
- ✓ Search still works.
- ✓ Filters still work.
- ✓ Sorting updates immediately.
- ✓ Reset restores the default order.
- ✓ Summary cards remain correct.
- ✓ Existing dashboard functionality continues working.

COMMON BEGINNER MISTAKES

A common mistake is requesting transactions from Supabase again whenever the user changes the sort order.

That is unnecessary.

Sorting should happen entirely inside the browser using the transactions already stored in memory.

Another mistake is sorting the original transaction list directly.

Instead, work with the filtered list that will be displayed to the user.

This keeps the original data unchanged and makes resetting much easier.

BEHIND THE SCENES

Think of the transaction list in stages.

First, JavaScript starts with the complete list stored in memory.

Second, it applies the search text.

Third, it applies the category filter.

Fourth, it applies the transaction type filter.

Finally, it sorts the remaining transactions before displaying them.

Keeping these steps separate makes the code easier to understand and maintain.

THINK LIKE A SOFTWARE DESIGNER

Users should never have to fight the software to find information.

Search helps users locate records.

Filters reduce the number of visible records.

Sorting arranges those records in the most useful order.

Together, these three tools transform a long list into something that is much easier to navigate.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports adding sorting controls. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- sort transactions in different ways
- combine sorting with search and filters
- preserve summary calculations
- keep sorting inside the browser
- improve the usability of large transaction lists

In the next lesson, you will perform a final user experience review and polish the dashboard before moving to the final testing chapter.

CHAPTER 8

MAKING THE DASHBOARD MORE POWERFUL

LESSON 3

FINAL USER EXPERIENCE REVIEW

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not add any new features.

Instead, you will review the complete dashboard from the user's point of view.

Professional software is not finished simply because every feature works.

It should also feel clear, responsive, and consistent.

This lesson focuses on polishing the overall experience before carrying out the final project testing in Chapter 9.

WHY THIS MATTERS

A user judges software within a few seconds.

Buttons should be easy to find.

Messages should be easy to understand.

The layout should remain organised.

The application should respond quickly to every action.

Many successful software products stand out because they pay attention to these details.

BEFORE YOU CONTINUE

Confirm that your Expense Tracker currently supports:

☒ Registration

☒ Login

☒ Protected dashboard☒ Logout☒ Add Transaction☒ Edit Transaction☒ Delete Transaction☒ Search☒ Category filter☒ Transaction Type filter☒ Sorting☒ Reset☒ Summary cards☒ Responsive layout

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a complete working Expense Tracker application.

Everything already built must continue working.

The project already includes:

- Public website
- Responsive navigation
- Registration
- Email verification
- Login
- Protected dashboard
- Logout
- Transaction form
- Add Transaction

- Edit Transaction
- Delete Transaction
- Search
- Category filtering
- Transaction Type filtering
- Sorting
- Reset
- Summary cards
- Responsive dashboard

Return complete updated files only.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

dashboard.html

dashboard.js

auth.css

GOAL

Review the complete dashboard and improve the overall user experience without changing the application's existing functionality.

Everything already built must continue working.

DASHBOARD.HTML

Keep every existing feature.

Improve only where necessary.

Review:

- Headings
- Labels
- Button text
- Section spacing
- Dashboard organisation
- Accessibility labels

Do not remove any features.

Do not rename IDs unless absolutely necessary.

AUTH.CSS

Keep every existing style.

Improve where appropriate:

- Consistent spacing
- Better alignment
- Button consistency
- Card spacing
- Form spacing
- Search controls
- Filter controls
- Summary cards
- Transaction list
- Mobile layout

Improve keyboard focus styles where appropriate.

Maintain good colour contrast.

Do not redesign the dashboard completely.

Refine it.

DASHBOARD.JS

Keep every existing feature working.

Review the JavaScript for:

- Duplicate code
- Repeated functions
- Unnecessary event listeners
- Repeated database requests
- Repeated calculations

Simplify the code where possible without changing behaviour.

Continue using:

`supabaseClient`

from:

supabase-config.js

Do not create another Supabase client.

Do not remove existing functionality.

QUALITY REVIEW

Review the complete dashboard and ensure that:

- Messages use consistent wording.
- Loading messages are clear.
- Error messages are friendly.
- Success messages are consistent.
- Buttons become disabled while processing.
- Buttons always return to their normal state.
- Forms return to Add mode correctly.
- Search, filters and sorting continue working together.
- Responsive layout remains clean.

IMPORTANT

Everything already built must continue working.

Do not introduce new features.

Do not remove features.

Do not rename files.

Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return three complete updated files:

dashboard.html

dashboard.js

auth.css

The dashboard should feel more polished while preserving every feature.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

Replace the complete contents.

Save the file.

Repeat the same process for:

dashboard.js

auth.css

Do not rename the files.

Do not create duplicate files.

TEST YOUR WORK

Sign in to your Expense Tracker.

Work through the dashboard from top to bottom.

Confirm that:

Navigation

☒ The logo still links to index.html.

☒ Your email address is displayed.

☒ Logout still works.

Summary Cards

☒ Total Income displays correctly.

☒ Total Expenses displays correctly.

☒ Current Balance displays correctly.

Transaction Form

☒ Every field is aligned correctly.

☒ Validation messages are easy to understand.

☒ Buttons disable while processing.

☒ Buttons return to normal afterwards.

Transactions

☒ Transactions display correctly.

☒ Search works.

☒ Filters work.

☒ Sorting works.

☒ Reset works.

☒ Edit works.

☒ Delete works.

Responsive Layout

Reduce the browser width.

Confirm that:

☒ The dashboard remains readable.

☒ Buttons remain usable.

☒ Search controls fit correctly.

☒ Forms remain easy to complete.

☒ Transactions remain readable.

General Review

Confirm that:

- ☒ No unnecessary spacing appears.
- ☒ No controls overlap.
- ☒ No horizontal scrolling appears.
- ☒ Success messages are consistent.
- ☒ Error messages are consistent.
- ☒ Loading messages are consistent.

CHECKPOINT

Before moving to Chapter 9, confirm that:

- ☒ Every feature still works.
- ☒ The dashboard feels consistent.
- ☒ The layout is professional.
- ☒ Buttons behave correctly.
- ☒ Messages are clear.
- ☒ Responsive behaviour is reliable.
- ☒ The application is ready for final testing.

COMMON BEGINNER MISTAKES

- Many beginners continue adding new features when they should be improving the existing ones.
- Professional software development often spends significant time refining what already exists.
- Another common mistake is making visual changes without testing every feature afterwards.
- Always confirm that your improvements have not accidentally affected existing functionality.

BEHIND THE SCENES

- Small improvements often have a larger impact than major redesigns.

Consistent spacing, predictable button behaviour, and clear messages reduce confusion and help users trust the application.

Many successful software products evolve through continuous refinement rather than dramatic redesigns.

THINK LIKE A SOFTWARE DESIGNER

Ask yourself one question:

"If someone who has never seen this software before opened it today, would they immediately understand how to use it?"

If the answer is yes, you have achieved one of the most important goals of good software design.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports final user experience review. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- review a complete application
- improve consistency
- refine user experience
- simplify existing code
- preserve existing functionality while improving quality

CHAPTER SUMMARY

Congratulations.

Your Expense Tracker dashboard is now feature complete.

Users can:

- ✓ Register
- ✓ Verify their email
- ✓ Log in
- ✓ Log out
- ✓ Add transactions
- ✓ View transactions
- ✓ Edit transactions
- ✓ Delete transactions
- ✓ Search
- ✓ Filter
- ✓ Sort
- ✓ Monitor Total Income
- ✓ Monitor Total Expenses
- ✓ Monitor Current Balance

Every major feature now works together as one complete application.

CHAPTER MILESTONE

By the end of Chapter 8, you have successfully built:

- ✓ Live search
- ✓ Category filtering
- ✓ Transaction Type filtering
- ✓ Sorting controls
- ✓ Reset controls
- ✓ Combined search, filtering and sorting
- ✓ Responsive improvements
- ✓ User experience improvements
- ✓ Dashboard refinement
- ✓ Complete feature integration

TRANSITION TO CHAPTER 9

Your Expense Tracker is now complete from a feature perspective.

The final chapter is about verification.

You will perform a complete end-to-end audit of the application.

You will test every page, every workflow, every security rule, every file, and every major feature.

You will also test the application using two different user accounts to confirm that privacy and security work exactly as intended.

Finally, you will prepare the project for submission as part of your Prompt to Profit™ portfolio.

CHAPTER 9

FINAL TESTING AND PROJECT COMPLETION

CHAPTER INTRODUCTION

Congratulations.

You have now built a complete Expense Tracker application from the ground up.

Your project includes:

- A professional public website
- Secure user registration
- Email verification
- User login
- Protected dashboard
- Secure database
- Transaction management
- Financial summaries
- Search
- Filtering
- Sorting
- Responsive design

The final chapter is not about adding more features.

Instead, it is about confirming that every feature works together exactly as expected.

Professional developers call this end-to-end testing.

Rather than testing one feature in isolation, you will test the complete application from the moment a visitor opens the website until they safely manage their financial records.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will complete:

- Full system audit
- Folder review
- Authentication testing
- Dashboard testing
- Database testing
- Security testing
- Two-account privacy testing
- Final project review
- Submission preparation
- Portfolio description

By the end of this chapter, your Expense Tracker will be ready for submission.

LESSON 1

COMPLETE PROJECT AUDIT

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

In this lesson, you are not creating new files.

Instead, you will test every important part of your application.

This is your final quality assurance review before submitting your project.

WHY THIS MATTERS

Many software bugs are discovered only after different parts of an application begin working together.

A complete audit helps you identify those problems before your users do.

Professional software teams spend significant time testing completed systems before releasing them.

BEFORE YOU CONTINUE

Your Expense Tracker folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

auth.css

register.js

login.js

dashboard.js

You should also have two verified user accounts available for testing.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete every test in this chapter carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

TEST 1

PUBLIC WEBSITE

Open:

index.html

Confirm that:

☒ The page opens.

☒ The layout is professional.

- ✓ Responsive navigation works.
- ✓ Hamburger menu works.
- ✓ Hero section displays correctly.
- ✓ Problem section displays correctly.
- ✓ Features section displays correctly.
- ✓ How It Works section displays correctly.
- ✓ Dashboard preview displays correctly.
- ✓ Call-to-action section displays correctly.
- ✓ Footer displays correctly.
- ✓ Login opens login.html.
- ✓ Register opens register.html.

TEST 2

USER REGISTRATION

Open:

register.html

Create a brand-new account.

Confirm that:

- ✓ Validation works.
- ✓ Password confirmation works.
- ✓ Loading state appears.
- ✓ Button becomes disabled.
- ✓ Success message appears.
- ✓ Verification email is received.
- ✓ Verification succeeds.

- ✓ User reaches login.html.

TEST 3

LOGIN

Open:

login.html

Attempt to sign in using an incorrect password.

Confirm that:

- ✓ A friendly error message appears.

Now sign in correctly.

Confirm that:

- ✓ Loading state appears.
- ✓ Button becomes disabled.
- ✓ Dashboard opens.

TEST 4

DASHBOARD

Confirm that:

- ✓ User email is displayed.
- ✓ Logout works.
- ✓ Summary cards appear.
- ✓ Transaction form appears.
- ✓ Search appears.

- ☒ Filters appear.
- ☒ Sorting appears.
- ☒ Transaction list appears.

TEST 5

ADDING TRANSACTIONS

-
- Add:
- Two Income transactions.
- Add:
- Three Expense transactions.
- Confirm that:
- ☒ All records save successfully.
 - ☒ Transactions appear immediately.
 - ☒ Summary cards update automatically.
-

TEST 6

EDITING TRANSACTIONS

-
- Edit one transaction.
- Confirm that:
- ☒ Form changes to Edit mode.
 - ☒ Transaction updates successfully.
 - ☒ Summary cards refresh.
 - ☒ Cancel Edit still works.

TEST 7**DELETING TRANSACTIONS**

Delete one transaction.

Confirm that:

- ☒ Confirmation appears.
 - ☒ Transaction disappears.
 - ☒ Summary cards refresh.
 - ☒ Empty state still works when appropriate.
-

TEST 8**SEARCH**

Search using:

Description

Category

Transaction Type

Confirm that:

- ☒ Matching transactions appear immediately.
 - ☒ Search ignores uppercase and lowercase differences.
-

TEST 9**FILTERS**

Test:

Category

Transaction Type

Confirm that:

- ☒ Filters work independently.
- ☒ Filters work together.
- ☒ Reset restores the complete list.

TEST 10**SORTING**

Test every sorting option.

Confirm that:

- ☒ Newest First
- ☒ Oldest First
- ☒ Highest Amount
- ☒ Lowest Amount
- ☒ Description (A–Z)
- ☒ Description (Z–A)

All work correctly.

TEST 11**RESPONSIVE DESIGN**

Reduce the browser width.

Confirm that:

- ☒ Navigation remains usable.
- ☒ Forms remain usable.
- ☒ Buttons remain usable.
- ☒ Search controls remain usable.
- ☒ Transactions remain readable.
- ☒ No horizontal scrolling appears unnecessarily.

TEST 12**USER PRIVACY**

Sign in using Account A.

Record several transactions.

Log out.

Sign in using Account B.

Confirm that:

- ☒ Account A's records are hidden.

Create transactions using Account B.

Log out.

Sign in using Account A.

Confirm that:

- ✓ Account B's records remain hidden.

This is the most important security test in the workbook.

CHECKPOINT

Before moving on, confirm that:

- ✓ Every feature works.
- ✓ Every page loads.
- ✓ Every workflow completes successfully.
- ✓ Two users remain completely isolated.
- ✓ Dashboard totals remain accurate.
- ✓ Responsive layout works.
- ✓ No major errors remain.

COMMON BEGINNER MISTAKES

- One common mistake is testing only the features you recently built.
- Always test the complete application from beginning to end.
- Sometimes a small change in one area accidentally affects another feature.
- Another common mistake is forgetting to test with two separate accounts.
- Security should always be verified using more than one user.

BEHIND THE SCENES

- By now, your application contains many connected systems.
- Registration depends on authentication.
- Authentication depends on Supabase.
- The dashboard depends on authentication.
- Transactions depend on the database.
- Financial summaries depend on transactions.

Search, filtering and sorting depend on the transactions already loaded into memory.

A complete audit confirms that every one of these systems continues working together.

THINK LIKE A SOFTWARE DESIGNER

Professional software is judged by consistency.

Every button should behave predictably.

Every page should guide the user naturally.

Every action should produce clear feedback.

When users trust that your software behaves consistently, they become confident using it.

WHAT YOU LEARNED

In this lesson you learned how to:

- perform a complete software audit
- test connected systems
- verify security
- verify user privacy
- confirm overall application quality

In the next chapter, you will carry out a complete end-to-end audit of your Expense Tracker, prepare it for deployment, and complete Workbook 01.

CHAPTER 9

FINAL TESTING AND PROJECT COMPLETION

LESSON 2

GOING LIVE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will prepare your Expense Tracker for real users.

Until now, you have been running the application from files stored on your computer.

The next step is to publish your project so that it can be opened from anywhere using a web browser.

You will deploy your project to Netlify.

Once the deployment is complete, your Expense Tracker will have its own public web address that you can share with anyone.

WHY THIS MATTERS

Software becomes much more valuable when other people can use it.

Deploying your project allows you to:

- Demonstrate your work.
- Build your portfolio.
- Share your project with employers.
- Show clients what you can build.
- Continue improving your application over time.

Many beginner developers build projects that never leave their computer.

Professional developers publish their work.

BEFORE YOU CONTINUE

Before deploying your project, confirm that:

- ✓ Registration works.
- ✓ Email verification works.
- ✓ Login works.
- ✓ Dashboard protection works.
- ✓ Transactions can be added.
- ✓ Transactions can be edited.
- ✓ Transactions can be deleted.
- ✓ Search works.
- ✓ Filters work.
- ✓ Sorting works.
- ✓ Summary cards calculate correctly.
- ✓ Responsive layout works.

Do not deploy a project that still contains major problems.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, deploy your Expense Tracker.

Follow these steps.

1.

Open your web browser.

2.

Visit:

<https://www.netlify.com>

3.

Create a free account if you do not already have one.

4.

Sign in.

5.

Click:

Add new site

6.

Choose:

Deploy manually

7.

Open your Expense Tracker folder.

8.

Select every project file.

9.

Compress the files into a ZIP file if required by your operating system, or drag the project folder if Netlify accepts it.

10.

Upload the project.

Wait while Netlify publishes your website.

After deployment, Netlify will provide a public web address.

Open the new website.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed using your web browser.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Open your live website.

Work through the complete application exactly as a new visitor would.

Landing Page

☒ The website opens correctly.

☒ Navigation works.

☒ Responsive menu works.

☒ Login link works.

☒ Register link works.

Authentication

☒ Registration works.

☒ Email verification works.

☒ Login works.

☒ Logout works.

Dashboard

☒ Dashboard opens only after login.

☒ User email appears.

☒ Transaction form works.

☒ Summary cards work.

Transactions

☒ Add Transaction works.

☒ Edit Transaction works.

☒ Delete Transaction works.

☒ Search works.

☒ Filters work.

☒ Sorting works.

Responsive Design

Open the website on a mobile phone if possible.

If that is not possible, reduce the browser width.

Confirm that:

☒ The layout remains usable.

- ✓ Buttons remain easy to press.
- ✓ Forms remain easy to complete.
- ✓ Transactions remain readable.

CHECKPOINT

Before moving on, confirm that:

- ✓ Your project is live.
- ✓ Every page opens correctly.
- ✓ Authentication works.
- ✓ Dashboard features work.
- ✓ Search works.
- ✓ Filters work.
- ✓ Sorting works.
- ✓ Responsive layout works.

COMMON BEGINNER MISTAKES

Some students assume that because the project works on their computer, it will automatically work online.

Always test the live version after deployment.

Another common mistake is testing only the landing page.

Walk through the complete application from registration to logout.

The live version should behave exactly like the local version.

BEHIND THE SCENES

When you deploy your project to Netlify, your HTML, CSS and JavaScript files are copied to servers that can be accessed over the internet.

Anyone with your website address can open the public pages.

Your authentication and transaction data continue to be stored securely in Supabase.

The website and the database work together even though they are hosted separately.

This is a common architecture used by many modern web applications.

THINK LIKE A SOFTWARE DESIGNER

Deploying software is not the end of development.

It is the beginning of real-world use.

After publishing a project, continue testing it, improving it, and learning from the way people use it.

Professional software improves continuously.

WHAT YOU LEARNED

In this lesson you learned how to:

- deploy a web application
- publish a project online
- test a live deployment
- verify that local and live behaviour match
- prepare a project for real users

In the next lesson, you will write a professional portfolio description for your Expense Tracker, complete your final reflection, and finish Workbook 01.

CHAPTER 9

FINAL TESTING AND PROJECT COMPLETION

LESSON 3

PORTFOLIO DESCRIPTION

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will prepare a professional description of the project you have built.

A portfolio description explains what your application does, the technologies you used, and the skills you demonstrated while building it.

A good portfolio description helps other people understand your work quickly.

WHY THIS MATTERS

Building software is only one part of becoming a developer.

You should also be able to explain what you built.

Whether you are applying for a job, bidding for freelance work, speaking to a client, or showcasing your projects on your own website, people need to understand the value of your work.

Learning how to describe your projects professionally is an important skill.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Your Expense Tracker has been deployed.
- ☒ The live website works correctly.
- ☒ Every major feature has been tested.

BUILD PROMPT

PROMPT STARTS HERE

Generate a professional portfolio description for my Expense Tracker project.

The description should be written in simple, professional English.

It should be suitable for:

- My personal portfolio website
- LinkedIn
- My CV
- Job applications
- Freelance proposals

The description should explain:

- What the application does.
- The technologies used.
- The key features.
- The software development skills demonstrated.

Do not exaggerate.

Do not invent features that do not exist.

Use the following technologies:

- HTML
- CSS
- Vanilla JavaScript
- Supabase

The application includes:

- Responsive public website
- User registration
- Email verification
- Secure login
- Protected dashboard
- Logout
- Transaction management
- Total Income

- Total Expenses
 - Current Balance
 - Search
 - Category filtering
 - Transaction Type filtering
 - Sorting
 - Responsive dashboard
 - Row Level Security
- Return only the completed portfolio description.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one professional project description.

It should be ready to copy into:

- A portfolio website
- LinkedIn
- A CV
- A project showcase

SAVE YOUR FILES

You may save the portfolio description in a document if you wish.

This is optional.

TEST YOUR WORK

Read the completed portfolio description.

Ask yourself:

- ☒ Does it accurately describe the project?
- ☒ Does it explain what the application does?

- ☒ Does it mention the technologies used?
- ☒ Does it avoid exaggeration?
- ☒ Could another person understand the project after reading it?

If the answer is yes to every question, your portfolio description is complete.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your portfolio description is complete.
- ☒ It accurately describes your project.
- ☒ It is written professionally.

COMMON BEGINNER MISTAKES

- Some beginners focus only on listing technologies.
- Instead, explain what the software actually does.
- Another common mistake is exaggerating the complexity of the project.
- Your portfolio should accurately reflect your work.
- Honest, well-explained projects build more trust than exaggerated descriptions.

BEHIND THE SCENES

Experienced software developers maintain portfolios because projects demonstrate practical ability far better than simply listing programming languages or tools.

A portfolio allows people to see what you have built rather than just reading what you claim to know.

THINK LIKE A SOFTWARE DESIGNER

- Every completed project tells a story.
- Your portfolio description should answer three simple questions.
- What problem does this application solve?

How does it solve that problem?

What did I learn while building it?

Clear answers to those questions make your work much easier to appreciate.

WHAT YOU LEARNED

In this lesson you learned how to:

- describe a software project professionally
- explain the technologies used
- communicate your work clearly
- prepare a project for your portfolio

REFLECTION QUESTIONS

Take a few minutes to think about what you have accomplished.

1.

What part of this workbook challenged you the most?

2.

Which feature taught you the most about how software works?

3.

What debugging problem took the longest to solve?

4.

If you built this project again, what would you do differently?

5.

Which part of the project makes you most proud?

6.

How has your understanding of AI-assisted software development changed since beginning this workbook?

There are no right or wrong answers.

The purpose of these questions is to help you recognise how much you have learned.

EXTENSION CHALLENGES

Congratulations on completing the core Expense Tracker.

If you would like to continue improving the project, try adding one or more of these features on your own.

- Monthly reports
- Export transactions as CSV
- Export transactions as PDF
- Dashboard charts
- Budget planning
- Spending alerts
- Dark mode
- Recurring transactions
- Notes for each transaction
- Receipt image uploads
- Multiple currencies
- User profile page

Try to build these features without step-by-step guidance.

This is where real learning begins.

NEXT WORKBOOK

Congratulations on completing Prompt to Profit™ Workbook 01.

You have successfully built a complete Expense Tracker using:

- HTML
- CSS
- Vanilla JavaScript
- Supabase
- ChatGPT
- Notepad

Along the way, you learned how to build a modern web application one step at a time while preserving everything already working.

You also learned how to think more like a software designer by testing carefully, protecting user data, and improving software through small, deliberate changes.

In Workbook 02, you will build another practical software application using the same structured approach, strengthening your confidence and expanding your ability to build useful digital tools with AI.

Well done.